CSE 489 · Mobile Application Development · BRAC University

অ্যাপের ফিচার ও তাদের কার্যপদ্ধতি

Assignment 2: Fragments, Navigation Drawer ও Jetpack
বাংলায় সহজ ও তত্ত্বভিত্তিক ব্যাখ্যা

A **Broadcast Receiver**
নিজের ব্রডকাস্ট পাঠানো ও গ্রহণ, আর ফোনের ব্যাটারির ব্রডকাস্ট শোনা

B **Image Scale**
ইন্টারনেট থেকে ছবি নামানো, পিঞ্চ জুম, সরানো ও ডাবল-ট্যাপ

C **Video**
VideoView ও MediaController দিয়ে অ্যাপের ভেতরে ভিডিও চালানো

D **Audio**
MediaPlayer দিয়ে গান: Play/Pause, Stop, স্লাইডার ও সময়

এই সংস্করণ সম্পর্কে

- এটি ইংরেজি নির্দেশিকা `CSE489_Assignment2_Explained.pdf`-এর বাংলা সংস্করণ। এখানে কোনো ছবি বা কোডের তালিকা নেই; আছে প্রতিটি ফিচার কী করে আর ভেতরে কীভাবে কাজ করে, তার সহজ ব্যাখ্যা।
- প্রথম অধ্যায়ে কঠিন শব্দ ছাড়া পুরো অ্যাসাইনমেন্ট; আর প্রতিটি অধ্যায়ের শুরুতে হলুদ "সহজ কথায়" বাক্সে সেই অধ্যায়ের মূল কথা।
- মেসেজ কিউ আর পাবলিশ/সাবস্ক্রাইব দুটি অধ্যায়ে গোড়া থেকে, দৈনন্দিন জীবন আর এই অ্যাপের উদাহরণসহ; শেষে ভাইভা প্রশ্নোত্তর আর পরিভাষা।
- কোড ব্যাখ্যা করা প্রতিটি অংশে "কোডে কোথায়" বাক্স ফাইল, ফাংশন আর লাইন নম্বর বলে দেয়, যেমন `CustomMessageReceiver.kt › onReceive() লাইন 23–28`। ইংরেজি রাখা হয়েছে মূলত কোড আর ক্লাসের নামে; নতুন পারিভাষিক শব্দের পাশে বন্ধনীতে ইংরেজি দেওয়া আছে।

সূচিপত্র

১ সহজ কথায় পুরো অ্যাসাইনমেন্ট ৫

১.১ অ্যাসাইনমেন্টে কী করতে বলা হয়েছে ৫

১.২ অ্যাপ খুললে কী দেখা যায় ৫

১.৩ Broadcast Receiver সহজ কথায় ৫

১.৪ Image Scale সহজ কথায় ৬

১.৫ Video ও Audio সহজ কথায় ৬

১.৬ দুটি মূল ধারণা সহজ কথায় ৬

১.৭ এই বই কীভাবে পড়া যায় ৭

২ অ্যাপটি এক নজরে ৮

২.১ অ্যাপের ফিচারসমূহ ৮

২.২ ব্যবহৃত প্রযুক্তি ৮

২.৩ কোডের মানচিত্র ও "কোডে কোথায়" বাক্স ৯

৩ মেসেজ কিউ (Message Queue): ধারণা ও উদাহরণ ১১

৩.১ সরাসরি ডাকার সমস্যা ১১

৩.২ মেসেজ কিউ কী ১১

৩.৩ দৈনন্দিন উদাহরণ: রেস্তোরাঁ ও ব্যাংক ১২

৩.৪ সফটওয়্যারের উদাহরণ: অনলাইন দোকান ১৩

৩.৫ মেসেজ কিউয়ের গুরুত্বপূর্ণ বৈশিষ্ট্য ১৩

৩.৬ Android-এর মেইন থ্রেড: অ্যাপের ভেতরের মেসেজ কিউ ১৪

৩.৭ এই অ্যাপে মেসেজ কিউয়ের উদাহরণ ১৫

৩.৮ একটি বার্তার যাত্রা: Play চাপা থেকে স্লাইডার চলা ১৫

৪ পাবলিশ/সাবস্ক্রাইব (Publish/Subscribe): ধারণা ও উদাহরণ ১৭

৪.১ পাবলিশ/সাবস্ক্রাইব কী ১৭

৪.২ দৈনন্দিন উদাহরণ: YouTube, রেডিও ও নোটিশ বোর্ড ১৭

৪.৩ ধাপে ধাপে: একটি ক্রিকেট স্কোর অ্যাপ ১৮

৪.৪ টপিক ও ফিল্টার ১৯

৪.৫ তিন দিক থেকে আলাদা থাকা (decoupling) ১৯

৪.৬ দেরিতে আসা সাবস্ক্রাইবার: তিন রকম পৌঁছানো ১৯

৪.৭ অবজারভার প্যাটার্ন: একই প্রোগ্রামের ভেতরে পাবলিশ/সাবস্ক্রাইব ২০

৪.৮ Android-এর ব্রডকাস্ট: একটি পাবলিশ/সাবস্ক্রাইব ব্যবস্থা ২০

৪.৯ উদাহরণ ১: নিজের ব্রডকাস্টের পুরো যাত্রা ২১

৪.১০ উদাহরণ ২: ব্যাটারি, শেষ বার্তা রাখা একটি ব্রডকাস্ট ২২

৫ দুই ব্যবস্থার তুলনা ও এই অ্যাপের মানচিত্র ২৪

৫.১ মেসেজ কিউ বনাম পাবলিশ/সাবস্ক্রাইব ২৪

৫.২ Explicit Intent: এক-থেকে-এক বার্তা ২৪

৫.৩ বড় মানচিত্র: ধারণা থেকে কোড ২৫
৫.৪ কোন ফিচারে কোন ব্যবস্থা ২৬

৬ লাইফসাইকেল ও স্টেট ২৭
৬.১ লাইফসাইকেল: কখন সাবস্ক্রাইব, কখন আনসাবস্ক্রাইব ২৭
৬.২ কনফিগারেশন পরিবর্তন, ViewModel ও সংরক্ষিত অবস্থা ২৮

৭ অ্যাপের কাঠামো: ড্রয়ার ও নেভিগেশন ২৯
৭.১ একটি Activity, চারটি Fragment ২৯
৭.২ নেভিগেশন গ্রাফ ও ড্রয়ার মেনুর সংযোগ ২৯
৭.৩ Fragment ও ViewBinding ৩০
৭.৪ ব্যাক বাটন চাপলে আগে ড্রয়ার বন্ধ হয় ৩১
৭.৫ পুরো স্ক্রিনজুড়ে নকশা (এজ-টু-এজ) ও ইনসেট ৩১

৮ ফিচার A: Broadcast Receiver ৩৩
৮.১ পুরো প্রবাহ এক নজরে ৩৩
৮.২ প্রথম স্ক্রিন: স্পিনার ও Proceed ৩৪
৮.৩ Explicit Intent: এক Activity থেকে আরেক Activity-তে ৩৪
৮.৪ নিজের ব্রডকাস্টের পথ, Activity 2: বার্তা নেওয়া ৩৫
৮.৫ CustomMessageReceiver: অ্যাপের নিজের সাবস্ক্রাইবার ৩৫
৮.৬ Activity 3: রেজিস্টার, প্রকাশ, গ্রহণ ৩৬
৮.৭ হারিয়ে যাওয়া বার্তার সমস্যা ৩৭
৮.৮ ফোন ঘোরালে: বার্তা দুবার যায় না, কিছু হারায়ও না ৩৮
৮.৯ ব্যাটারির পথ: সিস্টেমের ব্রডকাস্ট ৩৮
৮.১০ BatteryStatusLiveData: শুধু দেখা গেলে শোনা ৩৯
৮.১১ ব্যাটারির Intent থেকে শতাংশ ও অবস্থা ৩৯
৮.১২ ব্যাটারির পথ, Activity 3: কিছুই না করা ৪০
৮.১৩ নিরাপত্তা: কে পাঠাতে পারে, কে শুনতে পারে ৪০

৯ ফিচার B: Image Scale ৪২
৯.১ ইন্টারনেট থেকে ছবি: Coil 3 ..... ৪২
৯.২ স্ক্রিনে স্টেট দেখানো ৪৩
৯.৩ ছবির Matrix ও “ফিট” ৪৩
৯.৪ পিঞ্চ জুম: ScaleGestureDetector ৪৪
৯.৫ এক আঙুলে সরানো ও কিনারা সামলানো ৪৪
৯.৬ ডাবল-ট্যাপ ৪৫
৯.৭ ছোঁয়ার ঘটনা: একটি বার্তার ধারা ৪৫
৯.৮ ফোন ঘোরালে জুম ধরে রাখা ৪৬

১০ ফিচার C: Video ৪৭
১০.১ VideoView ও MediaController ৪৭
১০.২ প্লেয়ারের তিনটি ঘটনা ৪৭

১০.৩ থামানো ও মনে রাখা: onPause-এ কেন ৪৮
১০.৪ VideoViewModel: অবস্থান কোথায় থাকে ৪৮
১০.৫ কন্ট্রোল ও ড্রয়ারের সমন্বয় ৪৯

১১ ফিচার D: Audio ৫০
১১.১ MediaPlayer একটি স্টেট মেশিন ৫০
১১.২ প্লেয়ার কেন ViewModel-এ ৫১
১১.৩ স্টেট ওপরে ওঠে, নির্দেশ নিচে নামে ৫১
১১.৪ অগ্রগতি: কোরুটিনের একটি লুপ ৫২
১১.৫ স্লাইডার ও সময়ের লেখা ৫২
১১.৬ Stop, গান শেষ ও ভুল ৫৩
১১.৭ অডিও আর ভিডিও কখনো একসঙ্গে বাজে না ৫৩

১২ সব স্ক্রিনে প্রযোজ্য বিষয় ৫৪
১২.১ ফোন ঘোরালে: কোন স্ক্রিন কী ধরে রাখে ৫৪
১২.২ থিম, রিসোর্স ও অ্যাক্সেসিবিলিটি ৫৫

১৩ দ্রুত পুনরালোচনা ৫৬
১৩.১ মেসেজ কিউ ও পাবলিশ/সাবস্ক্রাইবের মূল কথা ৫৬
১৩.২ প্রতিটি ফিচারের মূল কথা ৫৭

১৪ ভাইভা প্রশ্নোত্তর ৫৯
১৪.১ অ্যাপের কাঠামো ও নেভিগেশন ৫৯
১৪.২ মেসেজ কিউ ও পাবলিশ/সাবস্ক্রাইব ৬০
১৪.৩ Broadcast Receiver ৬১
১৪.৪ Image Scale ৬২
১৪.৫ Video ও Audio ৬৩
১৪.৬ লাইফসাইকেল ও স্টেট ৬৪

১৫ পরিভাষা ৬৬
১৫.১ বার্তা ব্যবস্থার শব্দ ৬৬
১৫.২ Android-এর শব্দ ৬৭

১ সহজ কথায় পুরো অ্যাসাইনমেন্ট

> **সহজ কথায়**
> এই অধ্যায়টি পুরো বইয়ের ছোট রূপ। কোনো কোড বা কঠিন শব্দ ছাড়া এখানে বলা হয়েছে অ্যাপটি কী করে আর কেন এভাবে বানানো। সময় কম থাকলে শুধু এই অধ্যায়, দ্রুত পুনরালোচনা আর ভাইভা প্রশ্নোত্তর পড়লেও মূল বিষয়গুলো জানা হয়ে যাবে।

১.১ অ্যাসাইনমেন্টে কী করতে বলা হয়েছে

অ্যাসাইনমেন্টে একটি Android অ্যাপ বানাতে বলা হয়েছে, যার বাঁ পাশে একটি ড্রয়ার (টেনে বের করা মেনু) থাকবে। ড্রয়ারে চারটি আইটেম, আর প্রতিটি আইটেম একটি আলাদা স্ক্রিন খোলে:

- **Broadcast Receiver:** একটি তালিকা (স্পিনার) থেকে দুই ধরনের ব্রডকাস্টের একটি বেছে নিয়ে পরের ধাপে যাওয়া। প্রথম ধরনে ব্যবহারকারীর লেখা একটি বার্তা অ্যাপের নিজের বানানো রিসিভারে পৌঁছাতে হবে; দ্বিতীয় ধরনে ফোনের ব্যাটারির শতাংশ দেখাতে হবে।
- **Image Scale:** ইন্টারনেট থেকে একটি ছবি এনে দুই আঙুলের চিমটি (pinch) দিয়ে বড়-ছোট করা।
- **Video:** অ্যাপের ভেতরে একটি ভিডিও চালানো।
- **Audio:** অ্যাপের ভেতরে একটি গান চালানো।

এর উদ্দেশ্য Fragment, ড্রয়ার আর Android Jetpack-এর নিয়মকানুন হাতে-কলমে শেখা।

১.২ অ্যাপ খুললে কী দেখা যায়

অ্যাপ খুললেই প্রথমে Broadcast Receiver স্ক্রিন আসে। ওপরের বাঁ কোণে তিন দাগের হ্যামবার্গার আইকন (☰) চাপলে, বা স্ক্রিনের বাঁ কিনারা থেকে আঙুল টানলে, ড্রয়ার বেরিয়ে আসে। সেখান থেকে যেকোনো আইটেম বাছলে সেই স্ক্রিন খোলে, আর ওপরের শিরোনামও বদলে যায়। ড্রয়ার খোলা অবস্থায় ফোনের ব্যাক বাটন চাপলে অ্যাপ বন্ধ হয় না, শুধু ড্রয়ারটি বন্ধ হয়।

পেছনের কথা: পুরো অ্যাপে একটিই মূল Activity (একটি পূর্ণ স্ক্রিনের ধারক), আর চারটি স্ক্রিন হলো চারটি Fragment (Activity-র ভেতরে বসানো স্ক্রিন-অংশ)। ড্রয়ার আর ওপরের টুলবার একবারই তৈরি হয়; আইটেম বাছলে শুধু মাঝের Fragment বদলায়।

১.৩ Broadcast Receiver সহজ কথায়

ব্রডকাস্ট মানে এমন একটি ঘোষণা, যা একজন দেয় আর যারা শুনতে চেয়েছে তারা সবাই পায়, ঠিক স্কুলের মাইকে দেওয়া ঘোষণা বা রেডিওর মতো। Android-এ ফোন নিজেও এমন ঘোষণা দেয়, যেমন ব্যাটারির অবস্থা বদলালে; আর অ্যাপও নিজের ঘোষণা দিতে পারে। যে অংশ ঘোষণা শোনে, তার নাম রিসিভার (BroadcastReceiver)।

নিজের বার্তা পাঠানোর পথ

১. প্রথম স্ক্রিনে “Custom broadcast receiver” বেছে Proceed চাপা হয়।

২. দ্বিতীয় স্ক্রিনে (Activity 2) একটি বার্তা লেখা হয়, যেমন "Hello CSE489", তারপর "Send to receiver" চাপা হয়। লেখা ফাঁকা থাকলে অ্যাপ ভুলের বার্তা দেখায়।

৩. বার্তাটি তৃতীয় স্ক্রিনে (Activity 3) চলে যায়। সেখানে প্রথমে একটি রিসিভার তৈরি হয়ে শোনা শুরু করে, তারপর বার্তাটি ব্রডকাস্ট হিসেবে ঘোষণা করা হয়।

৪. রিসিভার ঘোষণাটি শুনে বার্তাটি স্ক্রিনে দেখায়: "Received via broadcast: Hello CSE489"।

কেন তৃতীয় স্ক্রিনই ঘোষণা দেয়? কারণ সাধারণ ব্রডকাস্ট রেডিওর মতো: ঘোষণার সময় যে শুনছে না, সে পরে আর পায় না। দ্বিতীয় স্ক্রিন ঘোষণা দিলে তখনো তৃতীয় স্ক্রিনের রিসিভার তৈরিই হয়নি, তাই বার্তা হারিয়ে যেত। তাই আগে শোনার ব্যবস্থা, তারপর ঘোষণা।

ব্যাটারির পথ

প্রথম স্ক্রিনে "System battery notification receiver" বেছে Proceed চাপলে দ্বিতীয় স্ক্রিন ব্যাটারির শতাংশ, চার্জ হচ্ছে কি না, আর শেষ কখন খবর এসেছে তা দেখায়। চার্জার লাগালে বা খুললে সংখ্যাগুলো সঙ্গে সঙ্গে বদলায়। ফোন ব্যাটারির শেষ খবরটি নোটিশ বোর্ডের মতো ধরে রাখে, তাই স্ক্রিন খোলামাত্র সংখ্যা দেখা যায়। "Next" চাপলে তৃতীয় স্ক্রিন খোলে, যা অ্যাসাইনমেন্টের কথামতো কিছুই করে না।

১.৪ Image Scale সহজ কথায়

স্ক্রিন খুললে অ্যাপ ইন্টারনেট থেকে একটি বড় ছবি নামায়; ততক্ষণ একটি ঘুরন্ত চিহ্ন দেখা যায়। ইন্টারনেট না থাকলে ভুলের বার্তা আর Retry বাটন আসে।

ছবি এলে দুই আঙুল ছড়িয়ে বড় করা যায় (সর্বোচ্চ ৫ গুণ), আঙুল কাছে আনলে ছোট হয় (সবচেয়ে ছোট মানে পুরো ছবি)। বড় অবস্থায় এক আঙুলে ছবি সরানো যায়, কিন্তু কিনারায় কখনো ফাঁকা জায়গা দেখা যায় না। দুবার দ্রুত ট্যাপ করলে ২.৫ গুণ বড় হয়, আবার দুবার ট্যাপ করলে আগের অবস্থায় ফেরে। ফোন ঘোরালেও জুম আর দেখা অংশ একই থাকে।

পেছনের কথা: ছবিটি আঁকা হয় একটি Matrix দিয়ে, যা একটি গাণিতিক নিয়ম: ছবির প্রতিটি বিন্দু স্ক্রিনের কোথায় বসবে আর কত বড় হবে, তা বলে দেয়। আঙুলের নড়াচড়া থেকে Android-এর ScaleGestureDetector বের করে দেয় কতটা বড় বা ছোট করতে হবে।

১.৫ Video ও Audio সহজ কথায়

Video: অ্যাপের ভেতরে রাখা একটি ভিডিও চলে; ভিডিওতে ট্যাপ করলে নিচে play/pause, সামনে-পেছনে যাওয়ার দাগ আর সময় দেখা যায়। ফোন ঘোরালে ভিডিও থামে না, একই জায়গা থেকে চলতে থাকে। অন্য স্ক্রিনে গেলে ভিডিও থেমে যায়, আর ফিরে এলে ঠিক সেই জায়গায় অপেক্ষা করে।

Audio: অ্যাপের ভেতরে রাখা একটি গান বাজে: Play/Pause, Stop (থামিয়ে শুরুতে ফেরা), টেনে সামনে-পেছনে যাওয়ার স্লাইডার, আর মিনিট:সেকেন্ডে সময়। ফোন ঘোরালেও গান চলতে থাকে, কারণ প্লেয়ারটি স্ক্রিনের বাইরে একটি ViewModel-এ রাখা, যা ঘোরানোর সময় নষ্ট হয় না। অন্য স্ক্রিনে গেলে গান থেমে যায়, তাই গান আর ভিডিও কখনো একসঙ্গে বাজে না।

১.৬ দুটি মূল ধারণা সহজ কথায়

এই অ্যাপ বুঝতে দুটি ধারণা জানা দরকার। দুটিই আসলে বার্তা পাঠানোর নিয়ম।

- **মেসেজ কিউ (Message Queue), অর্থাৎ বার্তার সারি:** ব্যাংকের টোকেনের মতো। সবাই সারিতে অপেক্ষা করে, কাউন্টার একে একে ডাকে, আর প্রতিটি টোকেন একটিই কাউন্টারে সেবা পায়। Android-এর প্রতিটি অ্যাপের ভেতরে ঠিক এমন একটি সারি আছে: আঙুলের ছোঁয়া, বাটন চাপা, ব্রডকাস্ট আসা, সবকিছু সারিতে দাঁড়ায়, আর মেইন থ্রেড একে একে সেগুলো সামলায়।
- **পাবলিশ/সাবস্ক্রাইব (Publish/Subscribe):** YouTube চ্যানেলের মতো। চ্যানেল ভিডিও দিলে সব সাবস্ক্রাইবার খবর পায়, অথচ চ্যানেল তাদের চেনেও না। Android-এর ব্রডকাস্ট ঠিক এই নিয়মে চলে: যে ঘোষণা দেয়, সে জানে না কে শুনছে।

এই দুটি ধারণা অধ্যায় ৩ ও ৪-এ উদাহরণসহ বিস্তারিত আছে।

১.৭ এই বই কীভাবে পড়া যায়

- **সময় খুব কম হলে:** এই অধ্যায়, তারপর অধ্যায় ১৩ (দ্রুত পুনরালোচনা) আর অধ্যায় ১৪ (ভাইভা প্রশ্নোত্তর)।
- **ধারণাগুলো ভালো করে বুঝতে:** অধ্যায় ৩ থেকে ৬ (মেসেজ কিউ, পাবলিশ/সাবস্ক্রাইব, দুইয়ের তুলনা, লাইফসাইকেল)।
- **কোড মিলিয়ে পড়তে:** অধ্যায় ৭ থেকে ১২; প্রতিটি অংশের "কোডে কোথায়" বাক্স ফাইল আর লাইন নম্বর বলে দেয়।
- প্রতিটি অধ্যায়ের শুরুতে হলুদ "সহজ কথায়" বাক্সে সেই অধ্যায়ের মূল কথা কয়েক লাইনে দেওয়া আছে।
- অচেনা শব্দ পেলে শেষের পরিভাষা অধ্যায় (অধ্যায় ১৫) দেখা যায়।

২ অ্যাপটি এক নজরে

> **সহজ কথায়**
> এখানে আছে অ্যাপের সব ফিচারের একটি তালিকা, অ্যাপটি কোন প্রযুক্তি দিয়ে বানানো, আর প্রজেক্টের কোন ফাইলে কী আছে। কোড মিলিয়ে পড়তে চাইলে শেষের "কোডের মানচিত্র" অংশটি কাজে লাগবে।

২.১ অ্যাপের ফিচারসমূহ

তৈরি অ্যাপটি অ্যাসাইনমেন্টের প্রতিটি শর্ত পূরণ করে, আর বাস্তব অ্যাপের মতো আচরণের জন্য কিছু বাড়তি খুঁটিনাটিও যোগ করে। শেষ কলামে লেখা অধ্যায়ে প্রতিটি ফিচারের বিস্তারিত ব্যাখ্যা আছে।

ফিচার	কী করে	অধ্যায়
ড্রয়ার ও নেভিগেশন	চারটি স্ক্রিনের মধ্যে যাতায়াত; প্রতিটি স্ক্রিনে হ্যামবার্গার আইকন (☰); ওপরের শিরোনাম আর ড্রয়ারের চিহ্নিত আইটেম বর্তমান স্ক্রিন অনুযায়ী বদলায়; ড্রয়ার খোলা থাকলে ব্যাক বাটন আগে ড্রয়ার বন্ধ করে।	৭
নিজের ব্রডকাস্ট (Custom)	দ্বিতীয় স্ক্রিনে লেখা বার্তা একটি ব্রডকাস্ট হয়ে তৃতীয় স্ক্রিনের নিজস্ব রিসিভারে পৌঁছায়। রিসিভার রেজিস্টার করা আছে কি না, কী পেল, কখন পেল, সবই স্ক্রিনে দেখা যায়।	৮
ব্যাটারির ব্রডকাস্ট	ফোনের `ACTION_BATTERY_CHANGED` ব্রডকাস্ট থেকে ব্যাটারির শতাংশ, চার্জ হচ্ছে কি না আর শেষ খবরের সময় সরাসরি (লাইভ) দেখায়; শুধু স্ক্রিন দেখা গেলেই শোনে।	৮
Image Scale	ইন্টারনেট থেকে বড় ছবি নামায়; দুই আঙুলে পিঞ্চ জুম (১ থেকে ৫ গুণ), জুম অবস্থায় এক আঙুলে সরানো, ডাবল-ট্যাপে জুম বা শুরুর অবস্থা; লোডিং আর ভুলের (Retry) অবস্থা।	৯
Video	অ্যাপে রাখা একটি ভিডিও VideoView ও MediaController দিয়ে চলে; ফোন ঘোরালে থামে না; স্ক্রিন ছেড়ে গেলে থেমে একই জায়গায় অপেক্ষা করে।	১০
Audio	অ্যাপে রাখা একটি MP3 গান MediaPlayer দিয়ে বাজে; Play/Pause, Stop, স্লাইডার আর মিনিট:সেকেন্ডে সময়; ফোন ঘোরালে গান চলে, স্ক্রিন ছাড়লে থামে।	১১
সব স্ক্রিনে	ফোন ঘোরালে কোনো তথ্য হারায় না; আড়াআড়ি (landscape) লেআউট; হালকা ও গাঢ় থিম; পুরো স্ক্রিনজুড়ে নকশা (এজ-টু-এজ)।	১২

এই ফিচারগুলোর পেছনে দুটি ধারণা বারবার কাজ করে: মেসেজ কিউ আর পাবলিশ/সাবস্ক্রাইব। তাই ফিচারের আগে অধ্যায় ৩ ও ৪-এ এ দুটি উদাহরণসহ ব্যাখ্যা করা হয়েছে।

২.২ ব্যবহৃত প্রযুক্তি

- **ভাষা ও স্ক্রিন:** Kotlin ভাষা; স্ক্রিনগুলো XML লেআউটে বানানো আর ViewBinding দিয়ে কোডের সঙ্গে যুক্ত। Jetpack Compose ব্যবহার করা হয়নি।

- **Material 3:** MaterialToolbar, NavigationView, TextInputLayout, Slider, প্রগ্রেস ইন্ডিকেটরের মতো তৈরি অংশ (component); হালকা ও গাঢ় দুই থিম।
- **Jetpack (AndroidX):** Navigation (নেভিগেশন গ্রাফ, NavHostFragment, NavigationUI), Fragment, Lifecycle, ViewModel, SavedStateHandle ও LiveData; সঙ্গে Kotlin-এর কোরুটিন ও StateFlow।
- **Coil 3:** ইন্টারনেট থেকে ছবি নামানো ও ডিকোড করার লাইব্রেরি (নেটওয়ার্কের কাজ করে OkHttp)।
- **Android-এর মিডিয়া API:** `android.media.MediaPlayer`, VideoView ও MediaController; আলাদা কোনো প্লেয়ার লাইব্রেরি নেই।
- **SDK:** minSdk 24 (Android 7.0), targetSdk ও compileSdk 37।

২.৩ কোডের মানচিত্র ও "কোডে কোথায়" বাক্স

সব Kotlin ফাইল আছে `app/src/main/java/com/example/labassignment2_cse489/` ফোল্ডারে, আর লেআউট, মেনু, নেভিগেশন ও লেখার (string) মতো রিসোর্স ফাইল আছে `app/src/main/res/`-এ। নিচের তালিকায় পথগুলো এই ফোল্ডারের পর থেকে লেখা।

ফাইল	কী আছে	অধ্যায়
`MainActivity.kt`	একমাত্র মূল Activity: ড্রয়ার, টুলবার, নেভিগেশন, ব্যাক বাটন ও ইনসেট	৭
`broadcast/BroadcastFragment.kt`	ব্রডকাস্টের প্রথম স্ক্রিন: স্পিনার ও Proceed	৮
`broadcast/BroadcastViewModel.kt`	স্পিনারের বাছাই মনে রাখা	৮
`broadcast/BroadcastMode.kt`	দুটি ধরন: CUSTOM ও BATTERY	৮
`broadcast/BroadcastInputActivity.kt`	Activity 2: বার্তা নেওয়া, অথবা লাইভ ব্যাটারি দেখানো	৮
`broadcast/CustomReceiverActivity.kt`	Activity 3: রিসিভার রেজিস্টার, ব্রডকাস্ট পাঠানো ও গ্রহণ	৮
`broadcast/CustomMessageReceiver.kt`	নিজস্ব রিসিভার, অর্থাৎ সাবস্ক্রাইবার	৮
`broadcast/CustomReceiverViewModel.kt`	পাওয়া বার্তা ও "পাঠানো হয়েছে" ফ্ল্যাগ	৮
`broadcast/BatteryStatusLiveData.kt`	স্ক্রিন দেখা গেলেই ব্যাটারির ব্রডকাস্ট শোনে	৮
`broadcast/BatteryViewModel.kt`	ব্যাটারির LiveData ধরে রাখে	৮
`broadcast/BatteryStatus.kt`	শতাংশের হিসাব ও চার্জের অবস্থা	৮
`image/ImageScaleFragment.kt`	Image Scale স্ক্রিন: লোডিং, ভুলের বার্তা, জুমের লেবেল	৯
`image/ImageScaleViewModel.kt`	Coil দিয়ে ছবি নামানো	৯
`image/ZoomableImageView.kt`	পিঞ্চ, সরানো, ডাবল-ট্যাপ ও Matrix	৯
`image/ZoomMath.kt`	জুমের শুধু গণিতের অংশ	৯
`video/VideoFragment.kt`	VideoView ও MediaController	১০
`video/VideoViewModel.kt`	ভিডিওর অবস্থান মনে রাখা	১০

ফাইল	কী আছে	অধ্যায়
`audio/AudioFragment.kt`	অডিওর বাটন, স্লাইডার ও সময়	১১
`audio/AudioPlayerViewModel.kt`	MediaPlayer-এর মালিক	১১
`util/TimeFormat.kt`, `util/WindowInsetsExt.kt`	মিনিট:সেকেন্ড ও ঘড়ির সময়; সিস্টেম বারের জন্য প্যাডিং	১১, ৭
`AndroidManifest.xml`, `res/navigation/nav_graph.xml`, `res/menu/drawer_menu.xml`	পারমিশন ও Activity-র তালিকা; চারটি স্ক্রিনের নেভিগেশন গ্রাফ; ড্রয়ারের আইটেম	৭

কোড ব্যাখ্যা করা প্রতিটি অংশের শুরুতে একটি **"কোডে কোথায়"** বাক্স আছে। এর প্রতিটি চিপে তিনটি তথ্য থাকে: ফাইলের নাম, ফাংশন বা ভেরিয়েবলের নাম, আর লাইন নম্বর। যেমন `CustomMessageReceiver.kt › onReceive() লাইন 23–28` মানে `CustomMessageReceiver.kt` ফাইলের `onReceive()` ফাংশন, লাইন 23 থেকে 28। লেখার ভেতরে শুধু লাইন নম্বরের ছোট চিপও আছে, যেমন `লাইন 34`; তখন ফাইলটি আশপাশের লেখা বা বাক্স থেকে বোঝা যায়।

> **কোড মিলিয়ে পড়ার নিয়ম**
>
> VS Code বা Android Studio-তে ফাইলটি খুলে Ctrl+G চেপে লাইন নম্বর দিলে সরাসরি সেই লাইনে যাওয়া যায়। লাইন নম্বরগুলো কোডের বর্তমান সংস্করণ (git commit `50afc1c`) অনুযায়ী; পরে কোড বদলালে নম্বর একটু সরে যেতে পারে, তখন ফাংশনের নাম দিয়ে খুঁজলেই হবে।

৩ মেসেজ কিউ (Message Queue): ধারণা ও উদাহরণ

> **সহজ কথায়**
>
> মেসেজ কিউ মানে বার্তার সারি। একজন বার্তা সারিতে রেখে নিজের কাজে চলে যায়, আরেকজন সারি থেকে একটি একটি করে বার্তা নিয়ে কাজ করে, ঠিক ব্যাংকের টোকেনের মতো। প্রতিটি বার্তা একজনই সামলায়, আর সামলানোর আগে পর্যন্ত বার্তা হারায় না। Android-এর প্রতিটি অ্যাপের ভেতরেও এমন একটি সারি আছে, যা মেইন থ্রেড একে একে সামলায়।

এই অধ্যায়ে মেসেজ কিউ বা বার্তার সারির ধারণাটি গোড়া থেকে দেখা হবে: কেন দরকার, কীভাবে কাজ করে, দৈনন্দিন জীবনে আর সফটওয়্যারে এর উদাহরণ, আর শেষে এই অ্যাপের ভেতরে এটি কোথায় কাজ করছে।

৩.১ সরাসরি ডাকার সমস্যা

ধরা যাক প্রোগ্রামের একটি অংশ A অন্য অংশ B-কে দিয়ে একটি কাজ করাতে চায়। সবচেয়ে সহজ উপায় হলো A সরাসরি B-এর ফাংশন ডাকবে। একে সরাসরি ডাকা বা সিঙ্ক্রোনাস কল (synchronous call) বলে। এতে চার রকম সমস্যা হতে পারে:

- **অপেক্ষা:** B কাজ শেষ না করা পর্যন্ত A আটকে থাকে। B ধীর হলে A-ও ধীর হয়ে যায়।
- **উপস্থিতি:** ডাকার মুহূর্তে B-কে চালু ও প্রস্তুত থাকতে হয়। B বন্ধ থাকলে কাজটি ব্যর্থ হয়।
- **চাপ:** একসঙ্গে অনেক অনুরোধ এলে B সামলাতে পারে না, আর অনুরোধগুলো জমিয়ে রাখার জায়গাও নেই।
- **শক্ত বাঁধন (tight coupling):** A-কে জানতে হয় B কোথায় আছে আর তাকে কীভাবে ডাকতে হয়। B বদলালে A-কেও বদলাতে হয়।

দৈনন্দিন জীবনে এটি ফোন কলের মতো: অপর পক্ষ ফোন না ধরলে কথা হয় না, আর কথা শেষ না হওয়া পর্যন্ত দুজনেই ব্যস্ত। বিকল্প হলো SMS বা চিঠি: প্রেরক বার্তা পাঠিয়েই নিজের কাজে ফিরে যান, আর প্রাপক সুবিধামতো সময়ে পড়ে নেন। সফটওয়্যারে এই "বার্তা পাঠানো"-র দুটি প্রধান রূপ হলো মেসেজ কিউ আর পাবলিশ/সাবস্ক্রাইব। এই অধ্যায়ে প্রথমটি, পরের অধ্যায়ে দ্বিতীয়টি।

৩.২ মেসেজ কিউ কী

মেসেজ কিউ হলো একটি সারি, যেখানে এক পক্ষ বার্তা রেখে যায় আর অন্য পক্ষ সেখান থেকে একটি একটি করে বার্তা তুলে নিয়ে কাজ করে। এর চারটি অংশ:

- **বার্তা (message):** যে কাজ করতে হবে বা যে খবর দিতে হবে, তার একটি প্যাকেট। সাধারণত দুই ভাগ থাকে: পরিচয়ের অংশ বা header (কী ধরনের বার্তা, কখন তৈরি) আর মূল অংশ বা body (আসল তথ্য)।
- **প্রেরক (producer):** যে বার্তা তৈরি করে সারিতে রাখে। সারিতে রাখার এই কাজকে enqueue বলে।

- **কিউ বা সারি (queue):** বার্তাগুলো যেখানে অপেক্ষা করে; প্রেরক আর গ্রাহকের মাঝখানের একটি জমা রাখার জায়গা (বাফার)।
- **গ্রাহক (consumer):** যে সারি থেকে বার্তা তুলে নেয় (dequeue) আর কাজটি করে।

সারি সাধারণত FIFO (First In, First Out) নিয়মে চলে: যে বার্তা আগে এসেছে, সেটি আগে বের হয়। ধরা যাক প্রেরক পরপর তিনটি বার্তা পাঠাল: M1, M2, M3। তখন সারির অবস্থা এরকম:

```
প্রেরক → [ M3 | M2 | M1 ] → গ্রাহক
```

গ্রাহক প্রথমে M1 নেবে, তারপর M2, তারপর M3। গ্রাহক যতক্ষণ M1 নিয়ে ব্যস্ত, ততক্ষণ M2 ও M3 সারিতে অপেক্ষা করে; হারিয়ে যায় না। এর মধ্যে প্রেরক চাইলে M4, M5 পাঠাতে পারে; সেগুলো সারির পেছনে যোগ হয়।

> **মূল নিয়ম**
> মেসেজ কিউতে প্রতিটি বার্তা ঠিক একজন গ্রাহক সামলায়, আর কাজ শেষ হলে বার্তাটি সারি থেকে মুছে যায়। একে এক-থেকে-এক (point-to-point) যোগাযোগ বলে: একটি বার্তা, একজন প্রাপক।

৩.৩ দৈনন্দিন উদাহরণ: রেস্তোরাঁ ও ব্যাংক

উদাহরণ ১: রেস্তোরাঁর রান্নাঘর

একটি ব্যস্ত রেস্তোরাঁয় ওয়েটার টেবিল থেকে অর্ডার নিয়ে একটি স্লিপে লেখেন আর রান্নাঘরের তারে স্লিপটি ঝুলিয়ে দেন। রাঁধুনি সেই তার থেকে সবচেয়ে পুরোনো স্লিপটি নেন, রান্না করেন, তারপর পরের স্লিপ। এখানে প্রতিটি অংশ মেসেজ কিউয়ের একটি অংশের সঙ্গে মেলে:

রেস্তোরাঁয়	মেসেজ কিউতে
অর্ডারের স্লিপ	বার্তা
ওয়েটার	প্রেরক
রান্নাঘরের তার, যেখানে স্লিপ ঝোলে	কিউ বা সারি
রাঁধুনি	গ্রাহক

এবার দেখা যাক এই ব্যবস্থায় কী কী ঘটে, আর প্রতিটি ঘটনার নাম কী:

১. ওয়েটার স্লিপ ঝুলিয়েই পরের টেবিলে চলে যান; রান্না শেষের অপেক্ষা করেন না। **প্রেরক অপেক্ষা করে না (অ্যাসিঙ্ক্রোনাস, asynchronous)।**

২. দুপুরের ভিড়ে একসঙ্গে দশটি অর্ডার এলেও কোনোটি হারায় না; স্লিপগুলো তারে জমে থাকে, আর রাঁধুনি নিজের গতিতে একটি একটি করে রান্না করেন। **বাফারিং, বা ভিড়ের চাপ সমান করা (load leveling)।**

৩. যে অর্ডার আগে এসেছে, সেটি আগে রান্না হয়। **FIFO ক্রম।**

৪. দুজন রাঁধুনি থাকলে তাঁরা স্লিপগুলো ভাগ করে নেন; একই স্লিপ দুজনে রান্না করেন না। **প্রতিযোগী গ্রাহক (competing consumers): প্রতিটি বার্তা একজনই সামলায়।**

৫. রান্না শেষ হলে স্লিপটি সেই তার থেকে নামিয়ে ফেলা হয়। **কাজ শেষে বার্তা সারি থেকে মুছে যায়।**

৬. একজন রাঁধুনি একটি অর্ডারে আধা ঘণ্টা আটকে থাকলে পেছনের সব অর্ডার দেরি হয়ে যায়। **গ্রাহক একজন হলে একটি ধীর বার্তা পুরো সারিকে আটকে দেয়।** এই শেষ বিষয়টি Android-এর জন্য খুব গুরুত্বপূর্ণ (৩.৬)।

উদাহরণ ২: ব্যাংকের টোকেন

ব্যাংকে ঢুকে একজন মানুষ মেশিন থেকে একটি টোকেন নেন; অর্থাৎ একটি বার্তা সারিতে ঢুকল। কাউন্টারগুলো একে একে পরের নম্বর ডাকে; এখানে কাউন্টারই কিউয়ের গ্রাহক। মানুষটি বসে অপেক্ষা করেন, লাইন ভাঙে না, আর প্রতিটি টোকেন একটিমাত্র কাউন্টারে সেবা পায়। কাউন্টার বাড়লে সারি দ্রুত ছোট হয়, কিন্তু নিয়ম একই থাকে: এক টোকেন, এক কাউন্টার।

৩.৪ সফটওয়্যারের উদাহরণ: অনলাইন দোকান

একটি অনলাইন দোকানের কথা ভাবা যাক। ক্রেতা "Order" চাপলে সার্ভারকে তিনটি কাজ করতে হয়: অর্ডার সংরক্ষণ, নিশ্চিতকরণ ইমেইল পাঠানো, আর গুদামে প্যাকিংয়ের নির্দেশ দেওয়া। ইমেইল পাঠাতে কয়েক সেকেন্ড লাগতে পারে, আর ইমেইল সার্ভার মাঝে মাঝে বন্ধও থাকে।

- **সারি ছাড়া:** সার্ভার ইমেইল পাঠানো শেষ না হওয়া পর্যন্ত ক্রেতাকে অপেক্ষা করায়। ইমেইল সার্ভার বন্ধ থাকলে পুরো অর্ডারই ব্যর্থ হতে পারে, যদিও অর্ডারের আসল কাজে কোনো সমস্যা ছিল না।
- **সারি দিয়ে:** সার্ভার অর্ডার সংরক্ষণ করে "OrderPlaced #1023" নামের একটি বার্তা ইমেইলের সারিতে রাখে, আর সঙ্গে সঙ্গে ক্রেতাকে জানায় "অর্ডার হয়েছে"। আলাদা একটি ইমেইল-কর্মী প্রোগ্রাম (গ্রাহক) সারি থেকে বার্তা নিয়ে ইমেইল পাঠায়। ইমেইল সার্ভার বন্ধ থাকলে বার্তাগুলো সারিতে অপেক্ষা করে; চালু হলে কর্মী প্রোগ্রাম সেগুলো একে একে পাঠিয়ে দেয়।

ফল: কোনো অর্ডারের ইমেইল হারায় না, ক্রেতাকেও অপেক্ষা করতে হয় না, আর ঈদের সেলের মতো ভিড়ের সময় আরও কয়েকটি কর্মী প্রোগ্রাম চালু করলেই কাজ ভাগ হয়ে যায়। বাস্তবে এ কাজে RabbitMQ, Amazon SQS বা ActiveMQ-এর মতো সফটওয়্যার ব্যবহার হয়।

৩.৫ মেসেজ কিউয়ের গুরুত্বপূর্ণ বৈশিষ্ট্য

- **ক্রম (ordering):** সাধারণ সারি FIFO। কিছু সারি বার্তাকে অগ্রাধিকার বা নির্ধারিত সময় অনুযায়ী সাজায়; তখন জরুরি বার্তা, কিংবা যার নির্ধারিত সময় আগে, সেটি আগে বের হয়।
- **বাফারিং ও চাপ সমান করা (load leveling):** প্রেরক দ্রুত আর গ্রাহক ধীর হলে বার্তা সারিতে জমে; গ্রাহক নিজের গতিতে কাজ করে। ভিড়ের সময়ের চাপ সময়ের ওপর ছড়িয়ে যায়।
- **প্রতিটি বার্তা একজন গ্রাহক:** একাধিক গ্রাহক থাকলে তারা কাজ ভাগ করে নেয় (competing consumers); গ্রাহক বাড়িয়ে কাজের গতি বাড়ানো যায়।
- **প্রাপ্তিস্বীকার (acknowledgement বা ack) ও আবার চেষ্টা:** গ্রাহক কাজ শেষে সারিকে জানায় "শেষ হয়েছে", তবেই বার্তা মোছা হয়। জানানোর আগে গ্রাহক বিকল হলে সারি বার্তাটি আরেকজনকে আবার দেয়।
- **পৌঁছানোর নিশ্চয়তা:** at-most-once মানে বার্তা হারাতে পারে কিন্তু দুবার আসবে না; at-least-once মানে হারাবে না কিন্তু দুবার আসতে পারে, তাই গ্রাহককে একই বার্তা দুবার সামলানো ঠেকাতে হয়; exactly-once মানে ঠিক একবার, যা অর্জন করা সবচেয়ে কঠিন।

- **দেরিতে চলার বার্তা (delayed message):** বার্তা এখনই নয়, নির্দিষ্ট সময় পরে কাজের জন্য রাখা যায়; যেমন "৩০ মিনিট পরে মনে করিয়ে দাও"।
- **স্থায়িত্ব (durability):** সারি যদি বার্তা ডিস্কে লিখে রাখে, তবে সার্ভার বন্ধ-চালু হলেও বার্তা থাকে। শুধু মেমরিতে থাকলে প্রোগ্রাম বন্ধ হলে অপেক্ষমাণ বার্তা হারায়।

সুবিধা	সীমাবদ্ধতা
প্রেরক অপেক্ষা করে না; সাড়া দ্রুত হয়	ফল সঙ্গে সঙ্গে পাওয়া যায় না (কিছুটা দেরি হয়)
ভিড় সামলায়; কোনো বার্তা হারায় না	সারি নিজেই একটি বাড়তি অংশ, যা চালাতে ও দেখভাল করতে হয়
প্রেরক ও গ্রাহক আলাদাভাবে বদলানো যায়	একাধিক গ্রাহক থাকলে কঠোর ক্রম বজায় রাখা কঠিন

৩.৬ Android-এর মেইন থ্রেড: অ্যাপের ভেতরের মেসেজ কিউ

কোডে কোথায় এটি Android-এর নিজস্ব ব্যবস্থা; প্রজেক্টে এটি আলাদা করে লেখা নেই। পরের অংশে (**৩.৭**) এই ব্যবস্থার প্রতিটি ব্যবহার কোডের অবস্থানসহ দেখানো হয়েছে।

Android নিজেই প্রতিটি অ্যাপের ভেতরে একটি মেসেজ কিউ চালায়। অ্যাপের স্ক্রিনের সব কাজ একটিমাত্র থ্রেডে (কাজের ধারায়) হয়: মেইন থ্রেড বা UI থ্রেড। এই থ্রেডে চারটি ক্লাস মিলে একটি পূর্ণ মেসেজ কিউ তৈরি করে:

সাধারণ ধারণা	Android-এর ক্লাস	কাজ
বার্তা	Message	একটি কাজ, সঙ্গে সেটি কখন চলবে তার সময়
সারি	MessageQueue	বার্তাগুলো চলার সময় অনুযায়ী সাজিয়ে রাখে; দেরিতে চলার বার্তা তার সময় পর্যন্ত অপেক্ষা করে
প্রেরক	Handler	`post()` বা `sendMessageDelayed()` দিয়ে সারিতে বার্তা রাখে; সিস্টেম নিজেও বার্তা রাখে
গ্রাহক	Looper	একটি অবিরাম লুপ: পরের বার্তা তুলে নিয়ে চালায়, তারপর আবার পরেরটা

এই সারির গ্রাহক একজনই, Looper। তাই বার্তাগুলো একটির পর একটি চলে, দুটি কখনো একসঙ্গে নয়। এর দুটি গুরুত্বপূর্ণ ফল:

- **লক লাগে না:** একসঙ্গে দুটি কলব্যাক (কোনো ঘটনা ঘটলে সিস্টেম যে ফাংশন ডেকে খবর দেয়) কখনো চলে না, তাই স্ক্রিনের তথ্য পড়তে-লিখতে লক বা বাড়তি সাবধানতা লাগে না।
- **ধীর বার্তা সব আটকে দেয়:** কোনো বার্তার কাজে বেশি সময় লাগলে পেছনের সব বার্তা অপেক্ষা করে; ছোঁয়ায় সাড়া মেলে না, স্ক্রিন জমে যায়, আর কয়েক সেকেন্ড চললে সিস্টেম ANR (Application Not Responding, অর্থাৎ "অ্যাপ সাড়া দিচ্ছে না") দেখায়। এটি সেই আধা ঘণ্টা আটকে থাকা রাঁধুনির ঘটনা। তাই ইন্টারনেট থেকে নামানো বা ফাইল প্রস্তুত করার মতো ধীর কাজ মেইন থ্রেডে করা চলে না।

আরেকটি বিষয়: এই সারি শুধু মেমরিতে থাকে। অ্যাপের প্রসেস বন্ধ হলে অপেক্ষমাণ বার্তাগুলোও হারায়। এ কারণেই দরকারি তথ্য আলাদাভাবে সংরক্ষণ করতে হয় (**৬.২**)।

৩.৭ এই অ্যাপে মেসেজ কিউয়ের উদাহরণ

এই অ্যাপের প্রায় প্রতিটি কলব্যাক আসলে মেইন থ্রেডের সারি থেকে আসা একটি বার্তা। কয়েকটি স্পষ্ট উদাহরণ, কোডের অবস্থানসহ:

১. **ছোঁয়ার ঘটনা (touch event):** আঙুলের প্রতিটি নড়াচড়া একটি MotionEvent বার্তা হয়ে সারিতে আসে, আর Looper সেটি ভিউয়ের (স্ক্রিনে দেখা একটি অংশ) `onTouchEvent()`-এ পৌঁছে দেয়। Image Scale স্ক্রিন এই বার্তার ধারা থেকেই পিঞ্চ আর সরানো চেনে।

`ZoomableImageView.kt` › **`onTouchEvent()`** লাইন 170–187

২. **ব্রডকাস্টের `onReceive()`:** সিস্টেম থেকে আসা ব্রডকাস্ট প্রথমে আমাদের প্রসেসের মেইন সারিতে জমা হয়; Looper পৌঁছালে `onReceive()` চলে। তাই সেখান থেকে সরাসরি LiveData-র মান বসানো নিরাপদ; কোডের মন্তব্যেও সে কথা লেখা আছে।

`BatteryStatusLiveData.kt` › **`receiver`** লাইন 22–27

৩. **MediaPlayer-এর onPrepared:** `prepareAsync()` অনুরোধ পাঠিয়েই ফিরে আসে; ফাইল প্রস্তুত হলে প্লেয়ার মেইন সারিতে একটি "প্রস্তুত" বার্তা রাখে, আর তখন onPrepared চলে। একে অনুরোধ-উত্তর (request/reply) ধরন বলে: অনুরোধ এখন, উত্তর পরে বার্তা হিসেবে।

`AudioPlayerViewModel.kt` › **`init (লিসেনার ও prepareAsync)`** লাইন 64–69

৪. **দেরিতে চলার বার্তা, `delay(200)`:** অডিওর অগ্রগতির লুপ প্রতি ২০০ মিলিসেকেন্ডে গানের অবস্থান হালনাগাদ করে। লুপটি চলে একটি কোরুটিনে (Kotlin-এর হালকা কাজ, যা থ্রেড না আটকে থামতে আর আবার চলতে পারে)। `delay()` আসলে একটি দেরিতে চলার বার্তা রেখে কোরুটিনকে থামিয়ে দেয়; ২০০ মিলিসেকেন্ড পরে বার্তাটি চলার উপযোগী হলে লুপ আবার চলে, আর মাঝের সময়ে মেইন থ্রেড অন্য বার্তা সামলায়।

`AudioPlayerViewModel.kt` › **`startProgressUpdates()`** লাইন 125–133

৫. **অ্যানিমেশনের ফ্রেম:** ডাবল-ট্যাপের জুম অ্যানিমেশন প্রতিটি ফ্রেমে একটি হালনাগাদ কলব্যাক পায়, যা একটি করে বার্তা হিসেবে আসে।

`ZoomableImageView.kt` › **`animateZoomTo()`** লাইন 158–168

৬. **লাইফসাইকেল কলব্যাক:** onStart, onStop ইত্যাদিও সিস্টেম থেকে আসা বার্তা হিসেবে মেইন থ্রেডে চলে; যেমন Activity 3-এর রিসিভার রেজিস্টার হয় onStart-এ।

`CustomReceiverActivity.kt` › **`onStart()`** লাইন 79–103

৩.৮ একটি বার্তার যাত্রা: Play চাপা থেকে স্লাইডার চলা

কোডে কোথায় `AudioFragment.kt` › **`onViewCreated()`** লাইন 30–44
`AudioPlayerViewModel.kt` › **`togglePlayPause(), play()`** লাইন 72–81
`AudioPlayerViewModel.kt` › **`startProgressUpdates()`** লাইন 125–133
`AudioFragment.kt` › **`render()`** লাইন 76–103

Audio স্ক্রিনে ব্যবহারকারী Play চাপলে ভেতরে কোন কোন বার্তা চলাচল করে, ধাপে ধাপে দেখা যাক:

১. আঙুল স্ক্রিন ছোঁয়া আর তোলার সঙ্গে সঙ্গে দুটি ছোঁয়ার বার্তা (ACTION_DOWN ও ACTION_UP) মেইন সারিতে আসে। Looper এগুলো Play বাটনে পৌঁছে দেয়, আর বাটন বুঝে নেয় এটি একটি ক্লিক।

২. ক্লিকের লিসেনার (যে ফাংশন ক্লিকের খবর পায়) ViewModel-এর `togglePlayPause()` ডাকে লাইন 34।

৩. ViewModel দেখে গান চলছে না, তাই `play()` চালায়: প্লেয়ার চালু করে, স্টেটে (state, অর্থাৎ স্ক্রিনের বর্তমান অবস্থা) "চলছে" লেখে, আর অগ্রগতির লুপ চালু করে লাইন 76–81।

৪. স্টেট বদলানোয় StateFlow নতুন মান প্রকাশ করে। Fragment-এর সংগ্রাহক (collector), যা নিজেও মেইন থ্রেডের একটি বার্তা হিসেবে চলে, `render()` চালায়, আর বাটনের লেখা "Pause" হয়ে যায়।

৫. অগ্রগতির লুপ প্লেয়ারের বর্তমান অবস্থান পড়ে স্টেটে বসায়, তারপর `delay(200)`। ২০০ মিলিসেকেন্ড পরে দেরিতে চলার বার্তাটি চলার উপযোগী হয়, Looper লুপটিকে আবার চালায়, আর স্লাইডার এক ধাপ এগোয়। গান চলার পুরো সময় এটি চলতে থাকে।

৬. এর ফাঁকে ফাঁকে ব্যবহারকারীর অন্য ছোঁয়া, স্ক্রিন আঁকার ফ্রেম ইত্যাদি বার্তাও একই সারি দিয়ে চলে। কোনো বার্তাই বেশি সময় নেয় না, তাই অ্যাপ মসৃণ থাকে।

> **মনে রাখার বিষয়**
>
> মেইন থ্রেড একটি মেসেজ কিউ, যার গ্রাহক একজন: Looper। অ্যাপের প্রায় সব কোড কোনো না কোনো বার্তার উত্তরে চলে, তাই প্রতিটি কলব্যাক দ্রুত শেষ করা জরুরি। ধীর কাজ (ডাউনলোড, ফাইল প্রস্তুত) অন্য থ্রেডে পাঠিয়ে ফল আবার বার্তা হিসেবে মেইন থ্রেডে আনতে হয়, ঠিক যেমন Coil আর `prepareAsync()` করে।

৪ পাবলিশ/সাবস্ক্রাইব (Publish/Subscribe): ধারণা ও উদাহরণ

সহজ কথায়

পাবলিশ/সাবস্ক্রাইব মানে একজন একটি বিষয়ে (টপিকে) খবর প্রকাশ করে, আর যারা আগে থেকে সেই বিষয়ে আগ্রহ জানিয়ে রেখেছে, তারা সবাই খবরটি পায়, ঠিক YouTube চ্যানেলের সাবস্ক্রাইবারদের মতো। প্রকাশক জানে না কে শুনছে। Android-এর ব্রডকাস্ট এই নিয়মেই চলে। মনে রাখার মূল প্রশ্ন: খবর প্রকাশের সময় যে শুনছিল না, সে কি পরে পাবে? সাধারণ ব্রডকাস্টে পায় না, ব্যাটারির ব্রডকাস্টে পায়।

মেসেজ কিউতে একটি বার্তা যায় একজন প্রাপকের কাছে। কিন্তু অনেক সময় একটি খবর একসঙ্গে অনেকের জানা দরকার, আর প্রেরক জানেনও না কারা জানতে চায়। এর সমাধান পাবলিশ/সাবস্ক্রাইব। এই অধ্যায়ে এর ধারণা, উদাহরণ, আর Android ও এই অ্যাপে এর প্রয়োগ দেখা হবে।

৪.১ পাবলিশ/সাবস্ক্রাইব কী

পাবলিশ/সাবস্ক্রাইব (সংক্ষেপে Pub/Sub) এমন একটি বার্তা ব্যবস্থা, যেখানে প্রেরক নির্দিষ্ট কাউকে বার্তা পাঠায় না; একটি বিষয় বা টপিকে (topic) বার্তা প্রকাশ (publish) করে। যারা আগে থেকে সেই টপিকে আগ্রহ জানিয়ে রেখেছে (subscribe), তাদের প্রত্যেকে বার্তার একটি করে কপি পায়। এর অংশগুলো:

- **প্রকাশক (publisher):** যে বার্তা প্রকাশ করে। সে জানে না কে বা কতজন শুনছে।
- **সাবস্ক্রাইবার (subscriber):** যে এক বা একাধিক টপিকে সাবস্ক্রাইব করে আর সেসব টপিকের বার্তা পায়।
- **টপিক (topic):** বার্তার শ্রেণির নাম, যেমন "খেলার খবর" বা "ব্যাটারির অবস্থা"। প্রকাশক আর সাবস্ক্রাইবার শুধু টপিকের নামে একমত হয়।
- **ব্রোকার (broker):** মাঝখানের ব্যবস্থা, যে কে কোন টপিকে সাবস্ক্রাইব করেছে তার তালিকা রাখে আর প্রতিটি বার্তা মিলিয়ে সঠিক সাবস্ক্রাইবারদের কাছে পৌঁছে দেয়।
- **সাবস্ক্রিপশন (subscription):** "এই সাবস্ক্রাইবার এই টপিক চায়", এই নিবন্ধন। আনসাবস্ক্রাইব করলে নিবন্ধন মুছে যায়, আর বার্তা আসা বন্ধ হয়।

মূল নিয়ম

মেসেজ কিউতে একটি বার্তা যায় একজনের কাছে; পাবলিশ/সাবস্ক্রাইবে একটি বার্তার কপি যায় টপিকের সব সাবস্ক্রাইবারের কাছে। একে ফ্যান-আউট (fan-out) বলে। আর প্রকাশককে সাবস্ক্রাইবারদের চিনতে হয় না, শুধু টপিকের নাম জানলেই চলে।

৪.২ দৈনন্দিন উদাহরণ: YouTube, রেডিও ও নোটিশ বোর্ড

উদাহরণ ১: YouTube চ্যানেল

কেউ একটি YouTube চ্যানেলে Subscribe চাপলে তিনি ওই চ্যানেলের সাবস্ক্রাইবার হয়ে যান। চ্যানেল নতুন ভিডিও আপলোড করলে (প্রকাশ) YouTube (ব্রোকার) চ্যানেলের সব সাবস্ক্রাইবারকে নোটিফিকেশন পাঠায়।

চ্যানেলের মালিক জানেন না ঠিক কোন কোন মানুষ নোটিফিকেশন পেলেন; তিনি শুধু আপলোড করেন। নতুন সাবস্ক্রাইবার এলে বা কেউ আনসাবস্ক্রাইব করলে মালিককে কিছুই বদলাতে হয় না।

YouTube-এ	পাবলিশ/সাবস্ক্রাইবে
চ্যানেল	টপিক
ভিডিও আপলোড করা	প্রকাশ (publish)
Subscribe বাটন চাপা	সাবস্ক্রাইব
YouTube-এর নোটিফিকেশন ব্যবস্থা	ব্রোকার
দর্শক	সাবস্ক্রাইবার
প্রত্যেকের ফোনে আসা নোটিফিকেশন	প্রত্যেক সাবস্ক্রাইবারের কাছে পৌঁছানো বার্তার কপি

উদাহরণ ২: রেডিও

রেডিও স্টেশন একটি নির্দিষ্ট ফ্রিকোয়েন্সিতে (টপিক) অনুষ্ঠান প্রচার করে। যার রেডিও সেই ফ্রিকোয়েন্সিতে টিউন করা (সাবস্ক্রাইব), তিনিই শোনেন। কিন্তু কেউ দেরিতে রেডিও চালু করলে আগের খবর আর শুনতে পান না; যা প্রচার হয়ে গেছে, তা চলে গেছে। একে বলে ক্ষণস্থায়ী (non-durable) পৌঁছানো: শুধু সেই মুহূর্তে উপস্থিত শ্রোতাই বার্তা পান।

উদাহরণ ৩: স্কুলের নোটিশ বোর্ড

নোটিশ বোর্ডে সর্বশেষ নোটিশটি ঝুলে থাকে, যতক্ষণ না নতুন নোটিশ এসে তার জায়গা নেয়। যে ছাত্র পরে আসে, সে-ও বোর্ড দেখেই সর্বশেষ খবর জেনে যায়। একে বলে শেষ বার্তা রাখা (retained বা last-value message): ব্রোকার প্রতিটি টপিকের শেষ বার্তাটি রেখে দেয়, আর নতুন সাবস্ক্রাইবার যুক্ত হওয়ামাত্র সেটি পায়।

পরে দেখা যাবে, এই অ্যাপের নিজের ব্রডকাস্ট রেডিওর মতো (**৪.৯**), আর ব্যাটারির ব্রডকাস্ট নোটিশ বোর্ডের মতো (**৪.১০**)।

৪.৩ ধাপে ধাপে: একটি ক্রিকেট স্কোর অ্যাপ

একটি ক্রিকেট স্কোর অ্যাপের উদাহরণ নেওয়া যাক। সার্ভারে একটি ব্রোকার আছে, আর টপিকের নাম `score/bd-vs-ind`।

১. **সাবস্ক্রাইব:** রহিমের ফোন আর করিমের ফোন ব্রোকারকে জানায়, "আমরা score/bd-vs-ind চাই"। ব্রোকার তালিকায় লিখে রাখে: score/bd-vs-ind → রহিম, করিম।

২. **প্রকাশ:** মাঠের স্কোরার একটি চার হওয়ার পর বার্তা পাঠান: টপিক score/bd-vs-ind, বিষয়বস্তু "বাংলাদেশ ১৫০/৩"। তিনি কারও নাম লেখেন না, পাঠিয়েই পরের বলে মন দেন।

৩. **মেলানো:** ব্রোকার বার্তার টপিক দেখে তালিকা থেকে সাবস্ক্রাইবার খুঁজে পায়: রহিম ও করিম।

৪. **ফ্যান-আউট ও পৌঁছানো:** ব্রোকার দুজনের প্রত্যেককে বার্তার একটি করে কপি পাঠায়।

৫. **গ্রহণ:** প্রতিটি ফোন নিজের মতো বার্তা সামলায়: একটিতে নোটিফিকেশন দেখায়, অন্যটিতে স্কোরবোর্ড হালনাগাদ করে।

৬. **আনসাবস্ক্রাইব:** খেলা শেষে করিম অ্যাপ বন্ধ করলে তাঁর ফোন আনসাবস্ক্রাইব করে; এরপর বার্তা শুধু রহিমের কাছে যায়। স্কোরারকে এ ব্যাপারে কিছুই জানতে বা করতে হয় না।

বাস্তবে এ ধরনের ব্যবস্থায় MQTT, Redis Pub/Sub, Apache Kafka বা Google Cloud Pub/Sub ব্যবহার হয়। ফোনের পুশ নোটিফিকেশনও (যেমন Firebase Cloud Messaging-এর টপিক) একই নিয়মে চলে।

৪.৪ টপিক ও ফিল্টার

সাবস্ক্রাইবার সব বার্তা চায় না, নির্দিষ্ট কিছু চায়। কোন বার্তা কোন সাবস্ক্রাইবার পাবে, তা দুইভাবে ঠিক হয়:

- **টপিক-ভিত্তিক:** বার্তার গায়ে একটি টপিকের নাম থাকে; সাবস্ক্রাইবার যে টপিকে সাবস্ক্রাইব করেছে, শুধু সেটির বার্তা পায়। ওপরের ক্রিকেট উদাহরণ এরকম।
- **বিষয়বস্তু-ভিত্তিক (content-based):** সাবস্ক্রাইবার একটি শর্ত দেয়, যেমন "শুধু উইকেট পড়লে জানাও"; ব্রোকার বার্তার ভেতরের তথ্য দেখে শর্ত মেলায়।

Android-এ এই ফিল্টারের নাম IntentFilter। এটি মূলত টপিক-ভিত্তিক (Intent-এর অ্যাকশন মেলায়), তবে ক্যাটাগরি আর ডেটা দিয়েও শর্ত দেওয়া যায়। এই অ্যাপের দুটি ফিল্টারই একটি করে অ্যাকশন মেলায়:

কোডে কোথায় `CustomMessageReceiver.kt` › **`intentFilter()`** লাইন 40
`BatteryStatusLiveData.kt` › **`onActive()` (ব্যাটারির ফিল্টার)** লাইন 37–38

৪.৫ তিন দিক থেকে আলাদা থাকা (decoupling)

পাবলিশ/সাবস্ক্রাইব প্রেরক আর প্রাপককে তিন দিক থেকে আলাদা রাখে:

ধরন	মানে	উদাহরণ
পরিচয়ের দিক থেকে (space decoupling)	প্রকাশক আর সাবস্ক্রাইবার একে অপরকে চেনে না; শুধু টপিকের নাম জানে।	YouTube চ্যানেল তার দর্শকদের নাম জানে না।
অপেক্ষার দিক থেকে (synchronization decoupling)	প্রকাশক বার্তা পাঠিয়েই নিজের কাজে ফেরে; কারও গ্রহণের অপেক্ষা করে না।	স্কোরার পাঠিয়েই পরের বলে মন দেন; Android-এ `sendBroadcast()` সঙ্গে সঙ্গে ফিরে আসে।
সময়ের দিক থেকে (time decoupling)	দুই পক্ষের একই সময়ে উপস্থিত থাকার দরকার নেই; ব্রোকার বার্তা ধরে রাখতে পারে।	নোটিশ বোর্ড; Android-এর স্টিকি ব্রডকাস্ট।

খেয়াল করার বিষয়, রেডিওর মতো সাধারণ পাবলিশ/সাবস্ক্রাইবে সময়ের দিক থেকে আলাদা থাকা নেই: সাবস্ক্রাইবারকে পাঠানোর মুহূর্তে উপস্থিত থাকতে হয়। Android-এর সাধারণ ব্রডকাস্টও এমন, আর এটিই এই অ্যাপের একটি গুরুত্বপূর্ণ নকশার কারণ (**৮.৭**)।

৪.৬ দেরিতে আসা সাবস্ক্রাইবার: তিন রকম পৌঁছানো

পাবলিশ/সাবস্ক্রাইবের সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন: বার্তা প্রকাশের সময় যে সাবস্ক্রাইবার ছিল না, সে কি পরে বার্তাটি পাবে? তিন রকম উত্তর হতে পারে:

ধরন	কী হয়	দৈনন্দিন উদাহরণ	Android-এ
ক্ষণস্থায়ী (non-durable)	শুধু উপস্থিত সাবস্ক্রাইবার পায়; দেরিতে আসা কেউ পায় না।	রেডিও	সাধারণ ব্রডকাস্ট (`sendBroadcast()`)
শেষ বার্তা রাখা (retained)	ব্রোকার শেষ বার্তাটি রেখে দেয়; নতুন সাবস্ক্রাইবার সঙ্গে সঙ্গে সেটি পায়।	নোটিশ বোর্ড	স্টিকি ব্রডকাস্ট (`ACTION_BATTERY_CHANGED`); LiveData ও StateFlow
স্থায়ী সাবস্ক্রিপশন (durable)	সাবস্ক্রাইবার না থাকার সময়ের সব বার্তা জমা থাকে; ফিরে এলে সব পায়।	ইমেইলের ইনবক্স	এই অ্যাপে ব্যবহৃত হয়নি

এই পার্থক্য না বুঝলে বার্তা "নিঃশব্দে হারিয়ে যাওয়া"-র মতো বাগ (ভুল) হয়: প্রেরক ভাবে পাঠানো হয়ে গেছে, কিন্তু সাবস্ক্রাইবার তখনো প্রস্তুত ছিল না। কোনো ভুলের বার্তাও দেখা যায় না, কারণ ক্ষণস্থায়ী ব্যবস্থায় "কেউ শুনছে না" কোনো ভুল নয়।

৪.৭ অবজারভার প্যাটার্ন: একই প্রোগ্রামের ভেতরে পাবলিশ/সাবস্ক্রাইব

পাবলিশ/সাবস্ক্রাইবের ছোট, ব্রোকার-ছাড়া রূপ হলো অবজারভার প্যাটার্ন (Observer pattern)। এখানে একটি অবজেক্ট নিজেই তার অবজারভারদের (যারা নজর রাখছে) তালিকা রাখে, আর নিজের মান বদলালে সবাইকে জানায়। সবকিছু একই প্রসেসের ভেতরে ঘটে, তাই আলাদা ব্রোকার লাগে না।

Android Jetpack-এর LiveData আর StateFlow এই প্যাটার্নের উদাহরণ, আর দুটিই নোটিশ বোর্ডের মতো আচরণ করে: নতুন অবজারভার যুক্ত হওয়ামাত্র বর্তমান মানটি পায়, তারপর প্রতিটি পরিবর্তন। এই অ্যাপে:

- Activity 3 পাওয়া বার্তার LiveData-র ওপর নজর রাখে; ViewModel নতুন বার্তা রাখলেই স্ক্রিন হালনাগাদ হয় লাইন 76 ।
- Audio স্ক্রিন একটি StateFlow থেকে মান সংগ্রহ (collect) করে; ViewModel স্টেট বদলালেই `render()` চলে লাইন 39–43 ।
- `ZoomableImageView`-এর `onZoomChanged` কলব্যাক একজন মাত্র অবজারভার নিয়ে এই প্যাটার্নের সবচেয়ে ছোট রূপ; জুম বদলালেই লেবেল হালনাগাদ হয় লাইন 36 ।

LiveData-র একটি বাড়তি গুণ আছে: এটি অবজারভারের লাইফসাইকেল জানে, তাই স্ক্রিন দেখা গেলেই কেবল মান পৌঁছায়। ফলে সাবস্ক্রিপশনের মেয়াদ নিজে থেকেই স্ক্রিনের সঙ্গে মেলে (**৬.১**)।

৪.৮ Android-এর ব্রডকাস্ট: একটি পাবলিশ/সাবস্ক্রাইব ব্যবস্থা

Android-এর ব্রডকাস্ট ব্যবস্থা হুবহু ব্রোকার-ভিত্তিক পাবলিশ/সাবস্ক্রাইব। কোনো অ্যাপ যখন `registerReceiver()` বা `sendBroadcast()` ডাকে, তখন সে রিসিভারের সঙ্গে সরাসরি কথা বলে না; Binder (Android-এ এক প্রসেস থেকে আরেক প্রসেসে কথা বলার ব্যবস্থা) দিয়ে সিস্টেমের `system_server` প্রসেসের ActivityManagerService-কে অনুরোধ পাঠায়। এই সার্ভিসই ব্রোকার।

পাবলিশ/সাবস্ক্রাইবে	Android-এ
বার্তা	Intent

পাবলিশ/সাবস্ক্রাইবে	Android-এ
টপিক	Intent-এর অ্যাকশন (action), যেমন `android.intent.action.BATTERY_CHANGED`
বার্তার বিষয়বস্তু	Intent-এর এক্সট্রা (extras)
প্রকাশক	যে `sendBroadcast()` ডাকে, অথবা সিস্টেম নিজে
সাবস্ক্রাইবার	BroadcastReceiver, যার `onReceive()` বার্তা পায়
সাবস্ক্রাইব / আনসাবস্ক্রাইব	`registerReceiver()` / `unregisterReceiver()`
ফিল্টার	IntentFilter
ব্রোকার	ActivityManagerService

একটি ব্রডকাস্ট পাঁচটি ধাপে চলে:

১. **সাবস্ক্রাইব:** `registerReceiver(receiver, filter)` সিস্টেমের তালিকায় লিখে রাখে, "এই রিসিভার এই অ্যাকশনের বার্তা চায়"।

২. **প্রকাশ:** `sendBroadcast(intent)` Intent-টি সিস্টেমের হাতে দিয়ে সঙ্গে সঙ্গে ফিরে আসে; প্রেরক অপেক্ষা করে না।

৩. **মেলানো:** সিস্টেম Intent-টিকে নিবন্ধিত প্রতিটি IntentFilter-এর সঙ্গে মিলিয়ে উপযুক্ত রিসিভার খুঁজে বের করে।

৪. **পৌঁছানো:** বার্তাটি রিসিভারের প্রসেসে পৌঁছে সেই প্রসেসের মেইন থ্রেডের সারিতে জমা হয়। এখানেই পাবলিশ/সাবস্ক্রাইব আর মেসেজ কিউ মিলে যায়।

৫. **গ্রহণ:** মেইন থ্রেডের Looper বার্তাটিতে পৌঁছালে রিসিভারের `onReceive()` চলে।

রিসিভার দুইভাবে সাবস্ক্রাইব করতে পারে। **ম্যানিফেস্টে লেখা (manifest-declared)** রিসিভার `AndroidManifest.xml`-এ লেখা একটি স্থায়ী সাবস্ক্রিপশন; দরকার হলে সিস্টেম অ্যাপ চালু করেও এতে বার্তা দিতে পারে, তবে Android 8 থেকে নির্দিষ্ট অ্যাপের নাম ছাড়া পাঠানো (implicit) বেশিরভাগ ব্রডকাস্ট এভাবে আর পাঠানো হয় না। **কোডে রেজিস্টার করা (context-registered)** রিসিভার `registerReceiver()` দিয়ে তৈরি একটি অস্থায়ী সাবস্ক্রিপশন, যা শুধু রেজিস্টার থেকে আনরেজিস্টার পর্যন্ত বাঁচে, আর শেষে অবশ্যই আনরেজিস্টার করতে হয়। এই অ্যাপের দুটি রিসিভারই দ্বিতীয় ধরনের।

৪.৯ উদাহরণ ১: নিজের ব্রডকাস্টের পুরো যাত্রা

Activity 2-তে ব্যবহারকারী "Hello CSE489" লিখে Send চাপলেন। এরপর বার্তাটি কীভাবে রিসিভার পর্যন্ত যায়, প্রতিটি ধাপ কোডের অবস্থানসহ:

১. **এক-থেকে-এক হস্তান্তর:** লেখাটি একটি explicit Intent-এর এক্সট্রা হয়ে Activity 3-এ যায়; এখানে এখনো কোনো ব্রডকাস্ট নেই।

`BroadcastInputActivity.kt` › **`sendMessage()`** লাইন 82–90

২. **সাবস্ক্রাইবার তৈরি:** Activity 3 নিজের `CustomMessageReceiver` তৈরি করে।

`CustomReceiverActivity.kt` › **`showCustomReceiver()`** লাইন 74

৩. **সাবস্ক্রাইব:** onStart-এ রিসিভার রেজিস্টার হয়; ফিল্টার শুধু `ACTION_CUSTOM_MESSAGE` মেলায়।

`CustomReceiverActivity.kt › onStart()` লাইন 85–90

৪. **প্রকাশ:** রেজিস্টারের পরেই Activity 3 একটি Intent তৈরি করে (টপিক = `ACTION_CUSTOM_MESSAGE`, বিষয়বস্তু = লেখাটি, `setPackage()` দিয়ে শুধু এই অ্যাপ) আর `sendBroadcast()` ডাকে।

`CustomReceiverActivity.kt › sendCustomBroadcast()` লাইন 114–117

`CustomMessageReceiver.kt › createBroadcast()` লাইন 46–49

৫. **মেলানো:** ActivityManagerService নিবন্ধিত ফিল্টারগুলোর সঙ্গে অ্যাকশন মিলিয়ে আমাদের রিসিভার খুঁজে পায়। এটি Android নিজে করে, প্রজেক্টে এর কোড নেই।

৬. **পৌঁছানো:** বার্তাটি আমাদের প্রসেসের মেইন থ্রেডের সারিতে জমা হয়।

৭. **গ্রহণ:** Looper পৌঁছালে `onReceive()` চলে: অ্যাকশন যাচাই, লেখা বের করা, লগ লেখা, তারপর কলব্যাক।

`CustomMessageReceiver.kt › onReceive()` লাইন 23–28

৮. **ভেতরের অবজারভারে হস্তান্তর:** কলব্যাক ViewModel-এ বার্তাটি রাখে, আর LiveData-র অবজারভার স্ক্রিনে "Received via broadcast: Hello CSE489" দেখায়।

`CustomReceiverActivity.kt › onBroadcastReceived()` লাইন 119–122

`CustomReceiverViewModel.kt › onMessageReceived()` লাইন 29–32

৯. **আনসাবস্ক্রাইব:** স্ক্রিন থেকে চলে গেলে onStop-এ রিসিভার আনরেজিস্টার হয়।

`CustomReceiverActivity.kt › onStop()` লাইন 105–112

> **লক্ষ করার বিষয়**
>
> একটি বার্তা এখানে চার রকম পথ পার হলো: এক-থেকে-এক (Intent দিয়ে Activity 3-এ), পাবলিশ/সাবস্ক্রাইব (ব্রডকাস্ট), মেসেজ কিউ (মেইন থ্রেড), আর শেষে অবজারভার প্যাটার্ন (LiveData দিয়ে স্ক্রিনে)। আর এটি রেডিওর মতো ক্ষণস্থায়ী: রেজিস্টারের আগে পাঠালে বার্তাটি কেউ পেত না। সে কারণেই ধাপ ৩ আগে, ধাপ ৪ পরে।

৪.১০ উদাহরণ ২: ব্যাটারি, শেষ বার্তা রাখা একটি ব্রডকাস্ট

ব্যাটারির ক্ষেত্রে প্রকাশক সিস্টেম নিজে, আর বার্তাটি নোটিশ বোর্ডের মতো রেখে দেওয়া (retained):

১. সিস্টেমের ব্যাটারি সার্ভিস ব্যাটারির অবস্থা বদলালেই `ACTION_BATTERY_CHANGED` প্রকাশ করে, আর শেষ বার্তাটি রেখে দেয় (স্টিকি)। এটি Android-এর ভেতরে ঘটে।

২. Activity 2 দেখা গেলে `BatteryStatusLiveData`-র `onActive()` সাবস্ক্রাইব করে।

`BatteryStatusLiveData.kt › onActive()` লাইন 29–40

৩. `registerReceiver()` সঙ্গে সঙ্গে রেখে দেওয়া শেষ Intent-টি ফেরত দেয়, আর কোড সেখান থেকেই প্রথম মান বসায় লাইন 39 । তাই পরের পরিবর্তনের অপেক্ষা ছাড়াই স্ক্রিনে শতাংশ দেখা যায়, ঠিক যেমন দেরিতে আসা ছাত্র বোর্ডে সর্বশেষ নোটিশ দেখে।

৪. এরপর প্রতিটি নতুন ব্রডকাস্টে রিসিভারের `onReceive()` মান হালনাগাদ করে।

`BatteryStatusLiveData.kt › receiver` লাইন 22–27

৫. স্ক্রিন আর দেখা না গেলে `onInactive()` আনসাবস্ক্রাইব করে।

`BatteryStatusLiveData.kt` › **`onInactive()`** লাইন 42–44

এই ব্রডকাস্ট সুরক্ষিত (protected): শুধু সিস্টেম এটি প্রকাশ করতে পারে, কোনো অ্যাপ নকল ব্যাটারি-বার্তা পাঠাতে পারে না। পাবলিশ/সাবস্ক্রাইবের ভাষায়, এই টপিকের প্রকাশকের ওপর নিয়ন্ত্রণ আছে। বিস্তারিত **৮.৯** ও **৮.১০** অংশে।

৫ দুই ব্যবস্থার তুলনা ও এই অ্যাপের মানচিত্র

> **সহজ কথায়**
> মেসেজ কিউ কাজ পাঠায়: প্রতিটি কাজ একজনই করে, আর কেউ না থাকলে কাজটি অপেক্ষা করে। পাবলিশ/সাবস্ক্রাইব খবর ছড়ায়: যারা শুনছে তারা সবাই পায়। এই অধ্যায়ের তালিকাগুলো দেখায় কোন ধারণা এই অ্যাপের কোন কোডে কাজ করছে।

এই অধ্যায়ে মেসেজ কিউ আর পাবলিশ/সাবস্ক্রাইব পাশাপাশি রেখে তুলনা করা হবে, তারপর একটি মানচিত্রে দেখা হবে কোন ধারণা এই অ্যাপের কোন কোডে কাজ করছে।

৫.১ মেসেজ কিউ বনাম পাবলিশ/সাবস্ক্রাইব

বিষয়	মেসেজ কিউ	পাবলিশ/সাবস্ক্রাইব
একটি বার্তা কতজন পায়	ঠিক একজন গ্রাহক	টপিকের সব সাবস্ক্রাইবার (ফ্যান-আউট)
প্রেরক কি প্রাপককে চেনে	সারিটি চেনে, গ্রাহককে নয়	শুধু টপিকের নাম জানে
প্রাপক তখন না থাকলে	বার্তা সারিতে অপেক্ষা করে	সাধারণত হারিয়ে যায়, যদি না শেষ বার্তা রাখা (retained) বা স্থায়ী (durable) হয়
প্রাপক বাড়ালে	কাজ ভাগ হয়, দ্রুত শেষ হয়	প্রত্যেকে আলাদা কপি পায়
মূল ব্যবহার	কাজ ভাগ করা: “এই কাজটি কেউ একজন করো”	খবর জানানো: “এটা ঘটেছে, যার দরকার জেনে নাও”
দৈনন্দিন উদাহরণ	রেস্তোরাঁর অর্ডার স্লিপ, ব্যাংকের টোকেন	YouTube নোটিফিকেশন, রেডিও, নোটিশ বোর্ড
Android-এ	মেইন থ্রেডের Looper ও MessageQueue; explicit Intent	ব্রডকাস্ট; ভেতরে LiveData ও StateFlow (অবজারভার)
এই অ্যাপে	ছোঁয়া, লাইফসাইকেল, onPrepared, `delay(200)`	নিজের ব্রডকাস্ট, ব্যাটারির ব্রডকাস্ট

> **এক বাক্যে**
> **কাজ** পাঠাতে চাইলে সারি, **খবর** ছড়াতে চাইলে পাবলিশ/সাবস্ক্রাইব। সারি নিশ্চিত করে কাজটি একজনই একবার করবে; পাবলিশ/সাবস্ক্রাইব খবরটি পৌঁছে দেয় সেই মুহূর্তে যারা শুনছে তাদের সবার কাছে, প্রেরককে তাদের না চিনেই।

৫.২ Explicit Intent: এক-থেকে-এক বার্তা

Activity খোলার explicit Intent (যে Intent-এ গন্তব্যের ক্লাস সরাসরি লেখা থাকে) এক ধরনের এক-থেকে-এক বার্তা। প্রাপক নির্দিষ্ট, অর্থাৎ একটি নির্দিষ্ট Activity ক্লাস, আর বার্তাটি হারায় না, কারণ সিস্টেম প্রাপক Activity

তৈরি করে Intent-টি তার হাতেই দেয়; নতুন Activity যখনই তৈরি হোক, Intent সঙ্গে নিয়েই আসে। এই অ্যাপে প্রতিটি Activity-র একটি `newIntent()` ফাংশন আছে, যা বার্তার "খাম" তৈরি করে:

কোডে কোথায় `BroadcastInputActivity.kt` › **`newIntent()` (ধরনসহ)** লাইন 139–140
`CustomReceiverActivity.kt` › **`newIntent()` (ধরন ও লেখাসহ)** লাইন 167–170

ব্যবহারকারীর লেখা ব্রডকাস্টে নয়, explicit Intent-এ Activity 3-এ পাঠানোর কারণ এখানেই: এক-থেকে-এক পথে বার্তা হারায় না, কিন্তু ক্ষণস্থায়ী পাবলিশ/সাবস্ক্রাইবে দেরিতে আসা সাবস্ক্রাইবার বার্তা পায় না (**৮.৭**)।

৫.৩ বড় মানচিত্র: ধারণা থেকে কোড

নিচের তালিকা তত্ত্ব আর কোডের সেতু। বাঁ দিকে ধারণা, মাঝে Android-এ তার নাম, আর ডানে এই প্রজেক্টে ঠিক কোথায় সেটি আছে।

ধারণা	Android-এ	এই অ্যাপের কোডে
টপিক	Intent-এর অ্যাকশন	`CustomMessageReceiver.kt` › **`ACTION_CUSTOM_MESSAGE`** লাইন 34
সাবস্ক্রিপশনের ফিল্টার	IntentFilter	`CustomMessageReceiver.kt` › **`intentFilter()`** লাইন 40
সাবস্ক্রাইব	`registerReceiver()`	`CustomReceiverActivity.kt` › **`onStart()`** লাইন 85–90
আনসাবস্ক্রাইব	`unregisterReceiver()`	`CustomReceiverActivity.kt` › **`onStop()`** লাইন 105–112
প্রকাশ	`sendBroadcast()`	`CustomReceiverActivity.kt` › **`sendCustomBroadcast()`** লাইন 114–117
সাবস্ক্রাইবার	BroadcastReceiver	`CustomMessageReceiver.kt` › **`onReceive()`** লাইন 23–28
প্রাপক সীমিত করা	`setPackage()`	`CustomMessageReceiver.kt` › **`createBroadcast()`** লাইন 46–49
কে পাঠাতে পারে	`RECEIVER_NOT_EXPORTED`; সুরক্ষিত ব্রডকাস্ট	`CustomReceiverActivity.kt` › **`onStart()`** লাইন 85–90
শেষ বার্তা রাখা (retained)	স্টিকি ব্রডকাস্ট	`BatteryStatusLiveData.kt` › **`onActive()`** লাইন 29–40
একই বার্তা দুবার না পাঠানো	"আগে পাঠানো হয়েছে" ফ্ল্যাগ	`CustomReceiverViewModel.kt` › **`initialBroadcastSent`** লাইন 22–26
এক-থেকে-এক বার্তা	explicit Intent	`CustomReceiverActivity.kt` › **`newIntent()`** লাইন 167–170
সারি ও একজন গ্রাহক	Looper ও MessageQueue	সব কলব্যাক; যেমন `ZoomableImageView.kt` › **`onTouchEvent()`** লাইন 170–187

ধারণা	Android-এ	এই অ্যাপের কোডে
দেরিতে চলার বার্তা	`delay()`, `postDelayed()`	`AudioPlayerViewModel.kt` › **`startProgressUpdates()`** লাইন 125–133
অনুরোধ-উত্তর	`prepareAsync()` → onPrepared	`AudioPlayerViewModel.kt` › **`init`** লাইন 64–69
ভেতরের পাবলিশ/সাবস্ক্রাইব (অবজারভার)	LiveData, StateFlow	`CustomReceiverViewModel.kt` › **`lastReceived`** লাইন 19 `AudioPlayerViewModel.kt` › **`uiState`** লাইন 47–48
লাইফসাইকেলে বাঁধা সাবস্ক্রিপশন	onActive/onInactive; `repeatOnLifecycle`	`BatteryStatusLiveData.kt` › **`onActive() … onInactive()`** লাইন 29–44
এক খবর, অনেক শ্রোতা (ফ্যান-আউট)	NavController → টুলবার ও ড্রয়ার	`MainActivity.kt` › **`setUpNavigation()`** লাইন 59–71
একজন গ্রাহকই জেতে	OnBackPressedCallback	`MainActivity.kt` › **`closeDrawerOnBack`** লাইন 35–39

৫.৪ কোন ফিচারে কোন ব্যবস্থা

ফিচার	মেসেজ কিউ ও এক-থেকে-এক	পাবলিশ/সাবস্ক্রাইব ও অবজারভার
ড্রয়ার ও নেভিগেশন	ছোঁয়া ও ব্যাক বাটনের ঘটনা; ব্যাকের ক্ষেত্রে একজন গ্রাহক	NavController-এর পরিবর্তন টুলবার ও ড্রয়ারে ফ্যান-আউট
নিজের ব্রডকাস্ট	explicit Intent-এ লেখা; `onReceive()` মেইন সারি দিয়ে আসে	নিজের ব্রডকাস্ট (ক্ষণস্থায়ী); পাওয়া বার্তার LiveData
ব্যাটারির ব্রডকাস্ট	`onReceive()` মেইন সারি দিয়ে আসে	স্টিকি ব্রডকাস্ট (শেষ বার্তা রাখা); LiveData-র onActive/onInactive
Image Scale	ছোঁয়ার ঘটনার ধারা; অ্যানিমেশনের ফ্রেম	ছবির StateFlow (Loading/Success/Error); `onZoomChanged` কলব্যাক
Video	onPrepared, onError, onCompletion বার্তা	ড্রয়ারের লিসেনার (ড্রয়ার নড়লে কন্ট্রোল লুকানো)
Audio	`prepareAsync()` → onPrepared; `delay(200)`-এর লুপ	AudioUiState-এর StateFlow

৬ লাইফসাইকেল ও স্টেট

সহজ কথায়

Android নিজেই স্ক্রিন তৈরি করে, দেখায়, লুকায় আর ভেঙে ফেলে, আর প্রতিটি ধাপে কোডকে খবর দেয়; একে লাইফসাইকেল বলে। এই অ্যাপের নিয়ম: যে ধাপে শোনা শুরু, তার জোড়া ধাপে শোনা বন্ধ। আর ফোন ঘোরালে স্ক্রিন নতুন করে তৈরি হয়, তাই দরকারি তথ্য ViewModel বা সংরক্ষিত অবস্থায় রাখা হয়, যাতে কিছু না হারায়।

শোনা শুরু করার সঠিক সময় কখন, আর ফোন ঘোরালে তথ্য কোথায় থাকে, এই দুটি প্রশ্নের উত্তর পুরো অ্যাপে একই নিয়মে দেওয়া হয়েছে। এই অধ্যায়ে সেই নিয়মগুলো।

৬.১ লাইফসাইকেল: কখন সাবস্ক্রাইব, কখন আনসাবস্ক্রাইব

Activity আর Fragment-কে সিস্টেম তৈরি করে, দেখায়, লুকায় আর ভেঙে ফেলে, আর প্রতিটি ধাপ একটি লাইফসাইকেল কলব্যাক দিয়ে জানায়: onCreate → onStart → onResume → onPause → onStop → onDestroy। onStart থেকে onStop পর্যন্ত স্ক্রিন দেখা যায় (STARTED অবস্থা), আর onResume থেকে onPause পর্যন্ত স্ক্রিন সবার সামনে থেকে ছোঁয়া ও ইনপুট পায় (RESUMED অবস্থা)।

কলব্যাকগুলো জোড়ায় জোড়ায় আসে। এই প্রজেক্টের মূলনীতি: **যে কলব্যাকে সাবস্ক্রাইব, তার জোড়া কলব্যাকে আনসাবস্ক্রাইব**, যাতে কোনো সাবস্ক্রিপশন তার স্ক্রিনের চেয়ে বেশি দিন না বাঁচে।

জোড়া	কী শুরু ও শেষ হয়	কোডে কোথায়
onStart ↔ onStop	Activity 3-এর নিজস্ব রিসিভার	`CustomReceiverActivity.kt` › **`onStart()`** লাইন 79–103 `CustomReceiverActivity.kt` › **`onStop()`** লাইন 105–112
onActive ↔ onInactive (LiveData)	ব্যাটারির রিসিভার	`BatteryStatusLiveData.kt` › **`onActive() … onInactive()`** লাইন 29–44
`repeatOnLifecycle(STARTED)`	Image Scale ও Audio স্ক্রিনে StateFlow থেকে মান সংগ্রহ	`ImageScaleFragment.kt` › **`onViewCreated()`** লাইন 42–46 `AudioFragment.kt` › **`onViewCreated()`** লাইন 39–43
onViewCreated ↔ onDestroyView	Fragment-এর binding; Video স্ক্রিনের ড্রয়ার-লিসেনার	`VideoFragment.kt` › **`onViewCreated()`** লাইন 57–58 `VideoFragment.kt` › **`onDestroyView()`** লাইন 91–100
ViewModel তৈরি (init) ↔ onCleared	Audio-র MediaPlayer তৈরি ও মুক্ত করা	`AudioPlayerViewModel.kt` › **`init`** লাইন 53–70 `AudioPlayerViewModel.kt` › **`onCleared()`** লাইন 141–144

নিয়ম না মানলে কী হয়? ধরা যাক onCreate-এ রিসিভার রেজিস্টার করা হলো, কিন্তু আনরেজিস্টার করা হলো না। সিস্টেম রিসিভারটিকে ধরে রাখে, আর রিসিভার ধরে রাখে Activity-কে; ফলে বন্ধ হয়ে যাওয়া Activity মেমরিতে আটকে থাকে (মেমরি লিক), আর কেউ দেখছে না এমন স্ক্রিনের জন্যও `onReceive()` চলতেই থাকে। বার্তার ভাষায়: একটি পরিত্যক্ত সাবস্ক্রিপশন, যা তবুও বার্তা পেয়ে যাচ্ছে।

Fragment-এর ক্ষেত্রে আরেকটি সূক্ষ্মতা আছে: Fragment নিজে বেঁচে থাকলেও তার ভিউ (স্ক্রিনে দেখা অংশ) ভেঙে যেতে পারে, যেমন ব্যাক স্ট্যাকে থাকার সময়। তাই স্ক্রিন-সংক্রান্ত অবজারভারগুলো Fragment-এর নিজের লাইফসাইকেলের সঙ্গে নয়, `viewLifecycleOwner`-এর সঙ্গে বাঁধা হয়।

৬.২ কনফিগারেশন পরিবর্তন, ViewModel ও সংরক্ষিত অবস্থা

ফোন ঘোরালে, ডার্ক মোড বা ভাষা বদলালে (একে কনফিগারেশন পরিবর্তন বলে) Android সাধারণত চলমান Activity ভেঙে নতুন করে তৈরি করে, যাতে নতুন অবস্থার উপযোগী রিসোর্স লোড হয়, যেমন `layout-land`-এর আড়াআড়ি লেআউট। এতে Activity আর Fragment-এর ভেতরের সাধারণ ভেরিয়েবলগুলো হারিয়ে যায়। এই অ্যাপ `android:configChanges` দিয়ে ব্যাপারটি এড়িয়ে যায় না; বরং তিন স্তরে তথ্য রক্ষা করে:

১. **ViewModel** ফোন ঘোরানোর মধ্যেও বেঁচে থাকে; স্ক্রিন স্থায়ীভাবে চলে গেলে তবেই `onCleared()` ডাকা হয়। তাই দীর্ঘস্থায়ী জিনিস, যেমন চলমান ডাউনলোড বা MediaPlayer, এখানে রাখা হয়। উদাহরণ:
   `AudioPlayerViewModel.kt › player লাইন 50`

২. **SavedStateHandle** হলো ViewModel-এর ভেতরের একটি চাবি-মান (key-value) ভান্ডার, যার মান Bundle-এ সংরক্ষিত হয়। অ্যাপ পেছনে থাকার সময় সিস্টেম মেমরির জন্য প্রসেসটি বন্ধ করে দিলেও (process death) ফিরে এলে মানগুলো আবার পাওয়া যায়। ছোট কিন্তু গুরুত্বপূর্ণ মান, যেমন স্পিনারের বাছাই, পাওয়া বার্তা বা ভিডিওর অবস্থান, এখানে থাকে। উদাহরণ:
   `BroadcastViewModel.kt › selectedMode লাইন 16–17`

৩. **ভিউয়ের নিজস্ব সংরক্ষিত অবস্থা:** `onSaveInstanceState()` ও `onRestoreInstanceState()`। id থাকা ভিউ নিজের অবস্থা নিজেই রাখতে পারে, যেমন টেক্সট ফিল্ডের লেখা বা জুমের মাত্রা। উদাহরণ:
   `ZoomableImageView.kt › onSaveInstanceState() লাইন 268–275`

ViewModel থেকে স্ক্রিনে মান যায় নজর রাখা যায় এমন ধারকের (observable) মাধ্যমে: LiveData বা StateFlow। দুটিই আসলে "সর্বশেষ মানসহ একটি টপিক" (**৪.৭**): নতুন অবজারভার যুক্ত হওয়ামাত্র বর্তমান মানটি পায়, তারপর প্রতিটি পরিবর্তন। ফোন ঘোরানোর পর নতুন স্ক্রিন নজর রাখা শুরু করামাত্র আগের মান ফিরে পায়, এটিই এর বড় সুবিধা।

শেষ হাতিয়ারটি হলো Activity-র `isChangingConfigurations`। onPause বা onStop-এর সময় এটি বলে দেয় স্ক্রিনটি কি সত্যিই চলে যাচ্ছে, নাকি শুধু ঘুরছে। ভিডিও আর অডিও স্ক্রিন এর ওপর নির্ভর করে ঠিক করে, থামবে নাকি চলতে থাকবে:

কোডে কোথায় `VideoFragment.kt › onPause() লাইন 76–79` `AudioFragment.kt › onStop() লাইন 52`

৭ অ্যাপের কাঠামো: ড্রয়ার ও নেভিগেশন

> **সহজ কথায়**
> পুরো অ্যাপে একটিই মূল Activity, যার ভেতরে ওপরে টুলবার, মাঝে বদলানো যায় এমন স্ক্রিন-অংশ (Fragment), আর পাশে ড্রয়ার। ড্রয়ারের প্রতিটি আইটেমের id আর নেভিগেশন গ্রাফের প্রতিটি স্ক্রিনের id এক, তাই Android নিজেই বুঝে নেয় কোন আইটেম চাপলে কোন স্ক্রিন খুলবে। ড্রয়ার খোলা থাকলে ব্যাক বাটন আগে ড্রয়ার বন্ধ করে।

সংশ্লিষ্ট ফাইল: `MainActivity.kt` · `res/layout/activity_main.xml` · `res/navigation/nav_graph.xml` · `res/menu/drawer_menu.xml` · `util/WindowInsetsExt.kt` · `AndroidManifest.xml`

চারটি ফিচারের আগে সেই কাঠামোটি দেখা যাক, যা এগুলোকে ধরে রাখে: একটি Activity, একটি ড্রয়ার আর Navigation কম্পোনেন্ট।

৭.১ একটি Activity, চারটি Fragment

কোডে কোথায় `activity_main.xml › DrawerLayout` লাইন 7–49
`activity_main.xml › NavHostFragment` লাইন 32–39 · `activity_main.xml › NavigationView` লাইন 42–48
`MainActivity.kt › onCreate()` লাইন 41–50

ড্রয়ারের চারটি স্ক্রিনের জন্য অ্যাপে একটিই মূল Activity আছে: `MainActivity`। চারটি স্ক্রিন (Broadcast Receiver, Image Scale, Video ও Audio) প্রতিটি একটি Fragment। `MainActivity`-র লেআউট এভাবে সাজানো:

- সবার বাইরে **DrawerLayout**, যা বাঁ পাশ থেকে টেনে আনা ড্রয়ার সামলায়।
- মূল অংশে ওপরে **MaterialToolbar** (টুলবার), তার নিচে **NavHostFragment** (একটি FragmentContainerView); বর্তমান Fragment এখানেই দেখানো হয়।
- পাশে **NavigationView**, যা ড্রয়ারের মেনু আর তার মাথার অংশ (কোর্সের নাম) দেখায়।

এই এক-Activity কাঠামোর সুবিধা হলো টুলবার আর ড্রয়ার একবারই তৈরি হয়; স্ক্রিন বদলালে শুধু মাঝের Fragment বদলায়, আর Navigation কম্পোনেন্ট পেছনের ইতিহাস (ব্যাক স্ট্যাক), শিরোনাম ও চিহ্নিত আইটেম সামলায়। ব্রডকাস্টের দ্বিতীয় ও তৃতীয় ধাপ অবশ্য আলাদা Activity (`BroadcastInputActivity` ও `CustomReceiverActivity`), কারণ অ্যাসাইনমেন্টে এই ধাপগুলোর জন্য আলাদা Activity চাওয়া হয়েছে।

৭.২ নেভিগেশন গ্রাফ ও ড্রয়ার মেনুর সংযোগ

কোডে কোথায় `nav_graph.xml › নেভিগেশন গ্রাফ` লাইন 6–35 `drawer_menu.xml › ড্রয়ার মেনু` লাইন 7–20
`MainActivity.kt › setUpNavigation()` লাইন 59–71

`nav_graph.xml`-এ চারটি Fragment গন্তব্য (destination) আছে। শুরুর গন্তব্য হলো `broadcastFragment` লাইন 10, তাই অ্যাপ খুললে প্রথমে Broadcast Receiver স্ক্রিন আসে। প্রতিটি গন্তব্যের লেবেলই সেই স্ক্রিনে

টুলবারের শিরোনাম হয়।

`drawer_menu.xml`-এর প্রতিটি আইটেমের id হুবহু একটি গন্তব্যের id-এর সমান। এই মিলটিই আসল কৌশল: NavigationUI এই id দেখেই বোঝে কোন আইটেম চাপলে কোন স্ক্রিনে যেতে হবে, আর কোন স্ক্রিনে থাকলে কোন আইটেম চিহ্নিত থাকবে। মেনুর গ্রুপে `checkableBehavior="single"`, তাই একবারে একটিই আইটেম চিহ্নিত থাকে।

`setUpNavigation()` তিনটি কাজ করে:

১. NavHostFragment থেকে NavController (নেভিগেশনের নিয়ন্ত্রক) নেয়; FragmentContainerView ব্যবহার করলে এভাবেই নিতে হয়।

২. ড্রয়ার মেনু থেকে `AppBarConfiguration` তৈরি করে লাইন 68। ফলে চারটি স্ক্রিনই টপ-লেভেল গন্তব্য: কোনোটিতে পেছনে যাওয়ার তীর (←) নেই, সবগুলোতে হ্যামবার্গার আইকন (☰), যা ড্রয়ার খোলে। মেনু থেকে বানানোয় মেনুতে নতুন আইটেম যোগ করলে এই তালিকাও নিজে থেকে মিলে যায়।

৩. `setupWithNavController()` দিয়ে টুলবারকে (শিরোনাম ও আইকন) আর NavigationView-কে (আইটেমে ক্লিক ও চিহ্ন) NavController-এর সঙ্গে যুক্ত করে লাইন 69–70।

বার্তার ভাষায় NavController একজন প্রকাশক: গন্তব্য বদলালে টুলবার আর ড্রয়ার, দুই লিসেনারই খবর পায় আর নিজেদের হালনাগাদ করে (ফ্যান-আউট, **৪.১**)। এ কারণেই কোথাও আলাদা করে শিরোনাম বা চিহ্ন বদলানোর কোড লিখতে হয়নি।

এর একটি গুরুত্বপূর্ণ ফল আছে: এক ড্রয়ার আইটেম থেকে আরেকটিতে যাওয়ার সময় NavigationUI আগের স্ক্রিনের অবস্থা (তার ViewModel-সহ) সংরক্ষণ করে রাখে, যাতে ফিরে এলে স্ক্রিনটি আগের অবস্থায় পাওয়া যায়। এ কারণে Audio স্ক্রিন ছেড়ে গেলেও তার প্লেয়ার বেঁচে থাকে, আর গান থামাতে আলাদা ব্যবস্থা লাগে (দ্রষ্টব্য **১১.৭**)।

৭.৩ Fragment ও ViewBinding

কোডে কোথায় `BroadcastFragment.kt › _binding` লাইন 20–21
`BroadcastFragment.kt › onDestroyView()` লাইন 69–72
`BroadcastFragment.kt › viewLifecycleOwner` লাইন 32

প্রতিটি স্ক্রিন তার লেআউটের জন্য ViewBinding ব্যবহার করে। বিল্ডের সময় প্রতিটি লেআউট থেকে একটি binding ক্লাস তৈরি হয়, যেখানে id থাকা প্রতিটি ভিউয়ের জন্য একটি নিরাপদ (type-safe) ফিল্ড থাকে। ফলে `findViewById()` লাগে না, আর ভুল id বা ভুল ধরনের কারণে ক্র্যাশের ঝুঁকিও থাকে না।

Fragment-এ binding থাকে শুধু onViewCreated থেকে onDestroyView পর্যন্ত, আর onDestroyView-এ সেটি null করে দেওয়া হয়। কারণ Fragment তার ভিউয়ের চেয়ে বেশি দিন বাঁচতে পারে; binding ধরে রাখলে ভেঙে যাওয়া পুরোনো ভিউগুলো মেমরিতে আটকে থাকত। একইভাবে স্ক্রিনের অবজারভারগুলো `viewLifecycleOwner` দিয়ে যুক্ত, যাতে ভিউ চলে গেলে অবজারভারও নিজে থেকে সরে যায়। চারটি Fragment-ই এই একই ধাঁচে লেখা; ওপরের চিপগুলো Broadcast স্ক্রিনের উদাহরণ।

৭.৪ ব্যাক বাটন চাপলে আগে ড্রয়ার বন্ধ হয়

কোডে কোথায় `MainActivity.kt › closeDrawerOnBack` লাইন 35–39
`MainActivity.kt › setUpBackHandling()` লাইন 73–84
`MainActivity.kt › onRestoreInstanceState()` লাইন 52–57 `AndroidManifest.xml › manifest` লাইন 15

প্রত্যাশিত আচরণ হলো: ড্রয়ার খোলা অবস্থায় ব্যাক বাটন চাপলে অ্যাপ থেকে বের না হয়ে শুধু ড্রয়ার বন্ধ হবে। নতুন Android-এ (targetSdk 36 বা তার বেশি, Android 16+) পুরোনো `onBackPressed()` আর ডাকাই হয় না; ব্যাক বাটন সামলানোর স্বীকৃত উপায় হলো OnBackPressedDispatcher-এ একটি OnBackPressedCallback যোগ করা।

- `closeDrawerOnBack` নামের কলব্যাকটি শুরুতে নিষ্ক্রিয় থাকে।
- একটি DrawerListener ড্রয়ার খুললে কলব্যাকটি চালু করে, বন্ধ হলে আবার নিষ্ক্রিয় করে।
- ফলে ড্রয়ার খোলা থাকলে ব্যাক বাটন শুধু ড্রয়ার বন্ধ করে; অন্য সব সময় কলব্যাক নিষ্ক্রিয় থাকায় ব্যাক বাটন তার স্বাভাবিক কাজ করে, পেছনে যাওয়ার আগাম অ্যানিমেশনসহ (predictive back)।
- ফোন ঘোরানোর পর DrawerLayout খোলা ড্রয়ার আবার খুলে দেয়, কিন্তু লিসেনারকে জানায় না। তাই `onRestoreInstanceState()`-এ ড্রয়ার খোলা কি না দেখে কলব্যাকের অবস্থা মিলিয়ে নেওয়া হয়।

বার্তার ভাষায় ব্যাক বাটন চাপা একটি ঘটনা, যা ডিসপ্যাচার চালু কলব্যাকগুলোর মধ্যে সবচেয়ে শেষে যোগ হওয়াটিকে দেয়: একজন গ্রাহকই জেতে, ঠিক মেসেজ কিউয়ের মতো। ম্যানিফেস্টে `enableOnBackInvokedCallback="true"` Android 13 থেকে 15-এও এই আগাম অ্যানিমেশন চালু করে, যাতে সব সংস্করণে আচরণ একই থাকে।

৭.৫ পুরো স্ক্রিনজুড়ে নকশা (এজ-টু-এজ) ও ইনসেট

কোডে কোথায় `WindowInsetsExt.kt › applyWindowInsetsPadding()` লাইন 20–46
`MainActivity.kt › applyWindowInsets()` লাইন 86–95
`BroadcastInputActivity.kt › includeIme` লাইন 126–131
`AndroidManifest.xml › adjustResize` লাইন 41–43

targetSdk 35 বা তার বেশি হলে Android 15+ ফোনে প্রতিটি অ্যাপ পুরো স্ক্রিনজুড়ে আঁকা হয় (edge-to-edge): ওপরের স্ট্যাটাস বার আর নিচের নেভিগেশন বার স্বচ্ছ, আর অ্যাপের জানালা (window) এদের নিচ পর্যন্ত বিস্তৃত। এই অ্যাপের প্রতিটি Activity শুরুতেই `enableEdgeToEdge()` ডাকে লাইন 43, যাতে পুরোনো Android-এও একই চেহারা হয়। এর মানে, প্রতিটি স্ক্রিনকে নিজের টুলবার আর বাটনগুলো সিস্টেম বারের নিচ থেকে সরিয়ে আনতে হয়, নইলে সেগুলো ঢাকা পড়ে যায়।

ইনসেট (window insets) হলো সিস্টেম বার, ক্যামেরার খাঁজ (display cutout) বা অন-স্ক্রিন কীবোর্ড স্ক্রিনের কোন পাশে কতটা জায়গা ঢেকে রেখেছে তার মাপ। `applyWindowInsetsPadding()` নামের ফাংশনটি বেছে নেওয়া দিকগুলোতে XML-এ দেওয়া মূল প্যাডিংয়ের (ভেতরের ফাঁকা জায়গা) সঙ্গে ইনসেট যোগ করে:

- মূল প্যাডিং একবার মনে রাখে, যাতে ইনসেট বারবার এলেও প্যাডিং জমে বাড়তে না থাকে।
- ইনসেট খরচ করে ফেলে না (consume করে না), তাই পাশের বা ভেতরের অন্য ভিউও একই ইনসেট পায়।

- টুলবারের অংশ ওপর আর দুই পাশের ইনসেট নেয়: তার রং স্ট্যাটাস বারের নিচ পর্যন্ত যায়, কিন্তু লেখা থাকে বারের নিচে। Fragment-এর এলাকা নিচ আর দুই পাশের ইনসেট নেয়, কারণ খাড়া (portrait) অবস্থায় নেভিগেশন বার নিচে থাকে আর আড়াআড়ি (landscape) অবস্থায় পাশে। ড্রয়ারের মাথার অংশ নেয় ওপরের ইনসেট।
- দ্বিতীয় Activity-র টেক্সট ফিল্ডের জন্য `includeIme = true`: কীবোর্ডের (IME) উচ্চতাও প্যাডিংয়ে যোগ হয়, তাই কীবোর্ড খুললেও টেক্সট ফিল্ড আর Send বাটন তার ওপরে দেখা যায়। ম্যানিফেস্টের `adjustResize` সিস্টেমকে কীবোর্ডের ইনসেট পাঠাতে বলে, কিন্তু এজ-টু-এজ জানালা নিজে আর ছোট হয় না; তাই ইনসেটটি নিজেদেরই প্রয়োগ করতে হয়।

`ViewGroupCompat.installCompatInsetsDispatch()` পুরোনো ও নতুন Android-এ একইভাবে প্রতিটি ভেতরের ভিউয়ের কাছে ইনসেট পৌঁছানো নিশ্চিত করে।

৮ ফিচার A: Broadcast Receiver

> **সহজ কথায়**
> এই ফিচারে দুটি পথ। নিজের ব্রডকাস্টের পথে দ্বিতীয় স্ক্রিনে লেখা বার্তা তৃতীয় স্ক্রিনে যায়; সেখানে আগে একটি রিসিভার শোনা শুরু করে, তারপর বার্তাটি ব্রডকাস্ট করা হয়, আর রিসিভার সেটি পেয়ে দেখায়। ব্যাটারির পথে দ্বিতীয় স্ক্রিন ফোনের ব্যাটারির ব্রডকাস্ট শুনে শতাংশ দেখায়, শুধু স্ক্রিন খোলা থাকলেই। তৃতীয় স্ক্রিন তখন কিছুই করে না, যেমনটি অ্যাসাইনমেন্টে বলা।

সংশ্লিষ্ট ফাইল (`broadcast` প্যাকেজ): `BroadcastFragment` · `BroadcastViewModel` · `BroadcastMode` · `BroadcastInputActivity` · `CustomReceiverActivity` · `CustomMessageReceiver` · `CustomReceiverViewModel` · `BatteryStatusLiveData` · `BatteryViewModel` · `BatteryStatus`

এটি অ্যাপের সবচেয়ে বড় ফিচার, আর অধ্যায় ৪-এর পাবলিশ/সাবস্ক্রাইবের সরাসরি প্রয়োগ। তিনটি স্ক্রিন জুড়ে দুটি আলাদা পথ: একটিতে অ্যাপ নিজেই ব্রডকাস্ট পাঠায় ও গ্রহণ করে, অন্যটিতে ফোনের সিস্টেমের পাঠানো ব্যাটারির ব্রডকাস্ট শোনে।

৮.১ পুরো প্রবাহ এক নজরে

প্রথম স্ক্রিন (`BroadcastFragment`, ড্রয়ারের "Broadcast Receiver") থেকে ব্যবহারকারী একটি ধরন বেছে Proceed চাপেন। তারপর পথ দুই ভাগ হয়।

"Custom broadcast receiver" বেছে নিলে

১. Activity 2 (`BroadcastInputActivity`) একটি টেক্সট ফিল্ড দেখায়; ব্যবহারকারী একটি বার্তা লিখে "Send to receiver" চাপেন।
২. লেখাটি Intent-এর ভেতরে করে Activity 3-এ (`CustomReceiverActivity`) যায়।
৩. Activity 3 নিজের `CustomMessageReceiver` তৈরি ও রেজিস্টার করে, তারপর লেখাটি ব্রডকাস্ট করে।
৪. রিসিভার ব্রডকাস্টটি গ্রহণ করে; স্ক্রিনে "Received via broadcast: …", সময় আর একটি টোস্ট (ছোট ভাসমান বার্তা) দেখা যায়।

"System battery notification receiver" বেছে নিলে

১. Activity 2 সিস্টেমের `ACTION_BATTERY_CHANGED` ব্রডকাস্ট শোনে আর ব্যাটারির শতাংশ, চার্জ হচ্ছে কি না ও শেষ খবরের সময় সরাসরি দেখায়।
২. "Next" চাপলে Activity 3 খোলে, যা অ্যাসাইনমেন্ট অনুযায়ী কিছুই করে না, শুধু সেটি জানিয়ে দেয়।

৮.২ প্রথম স্ক্রিন: স্পিনার ও Proceed

কোডে কোথায় `BroadcastFragment.kt` › **`setUpSpinner()`** লাইন 42–62
`BroadcastFragment.kt` › **`onViewCreated()`** লাইন 25–40
`BroadcastFragment.kt` › **`openSelectedActivity()`** লাইন 65–67
`BroadcastMode.kt` › **`BroadcastMode`** লাইন 10–24
`BroadcastViewModel.kt` › **`BroadcastViewModel`** লাইন 13–30 `arrays.xml` › **`broadcast_types`** লাইন 4–7

স্ক্রিনে আছে "Select a broadcast type" লেখা, একটি স্পিনার (নিচে নামানো তালিকা), একটি Proceed বাটন, আর বাছাই করা ধরনটি পরের স্ক্রিনে কী করবে তার একটি ছোট বর্ণনা।

- **স্পিনারের আইটেম:** `arrays.xml`-এর একটি লেখার তালিকায় (string-array) ঠিক দুটি আইটেম আছে। `ArrayAdapter.createFromResource()` সেগুলো স্পিনারে বসায় লাইন 44–48, আর `item_spinner` লেআউট বন্ধ ও খোলা, দুই অবস্থার চেহারা ঠিক করে।
- **`BroadcastMode` enum:** দুটি মান, `CUSTOM` ও `BATTERY`, তালিকার একই ক্রমে সাজানো। তাই স্পিনারের অবস্থান (position) থেকে সরাসরি ধরন পাওয়া যায় (`fromPosition()`), আর সীমার বাইরের অবস্থান হলে আসে null।
- **বাছাই মনে রাখা:** ব্যবহারকারী যা বাছেন, তা `BroadcastViewModel`-এর SavedStateHandle-এ রাখা হয়। স্পিনার তৈরির সময় মনে রাখা বাছাইটি বসিয়ে দেওয়া হয় লাইন 54, তাই ফোন ঘোরানোর পর, এমনকি প্রসেস বন্ধ হয়ে আবার চালু হলেও, আগের বাছাইই দেখা যায়।
- **বর্ণনা হালনাগাদ:** `selectedMode` একটি LiveData। Fragment এই LiveData-র ওপর নজর রাখে লাইন 32–39, আর ধরন বদলালেই বর্ণনার লেখা বদলে যায়।
- **Proceed:** `BroadcastInputActivity.newIntent()` একটি explicit Intent তৈরি করে, যার এক্সট্রায় ধরনের নাম (যেমন `"CUSTOM"`) থাকে, আর `startActivity()` দিয়ে Activity 2 খোলে।

৮.৩ Explicit Intent: এক Activity থেকে আরেক Activity-তে

কোডে কোথায় `BroadcastInputActivity.kt` › **`newIntent()`** লাইন 139–140
`BroadcastInputActivity.kt` › **`onCreate()`** লাইন 42–50
`CustomReceiverActivity.kt` › **`newIntent()`** লাইন 167–170
`AndroidManifest.xml` › **`exported="false"`** লাইন 41–46

explicit Intent-এ গন্তব্য Activity-র ক্লাস সরাসরি বলে দেওয়া থাকে, তাই সিস্টেম ঠিক সেটিই খোলে। এটি এক-থেকে-এক বার্তা (৫.২): একজন প্রেরক, একজন নির্দিষ্ট প্রাপক, আর বার্তা হারানোর প্রশ্ন নেই।

- প্রতিটি Activity-র companion object-এ একটি `newIntent()` ফাংশন আছে। এক্সট্রার চাবিগুলো (`"mode"`, `"message"`) সেই Activity-র ভেতরে লুকানো (private), তাই বাইরে থেকে কেউ ভুল চাবি লিখতে পারে না।
- enum পাঠানো হয় তার নাম (String) হিসেবে, আর গ্রহণকারী `BroadcastMode.fromName()` দিয়ে ফিরিয়ে আনে। নাম না থাকলে বা অচেনা হলে null আসে; তখন Activity লগে সতর্কবার্তা লিখে `finish()` করে, ভুল তথ্যে ক্র্যাশ হয় না।
- Activity 2 ও 3 ম্যানিফেস্টে `exported="false"`, তাই শুধু এই অ্যাপই এগুলো খুলতে পারে।

- টুলবারের পেছনে যাওয়ার আইকন শুধু `finish()` ডাকে, ফলে আগের স্ক্রিন তার আগের অবস্থাতেই ফিরে আসে।

৮.৪ নিজের ব্রডকাস্টের পথ, Activity 2: বার্তা নেওয়া

কোডে কোথায় `BroadcastInputActivity.kt` › **`showCustomInput()`** লাইন 55–79
`BroadcastInputActivity.kt` › **`sendMessage()`** লাইন 82–90

`BroadcastInputActivity`-র লেআউটে দুটি অংশ আছে: বার্তা লেখার অংশ আর ব্যাটারির অংশ। Intent-এ আসা ধরন অনুযায়ী একটি অংশ দেখানো হয় আর টুলবারের শিরোনাম বদলানো হয়। বার্তার অংশে আছে "Message to broadcast" টেক্সট ফিল্ড আর "Send to receiver" বাটন।

- **পাঠানোর তিন উপায়, একই কাজ:** বাটন চাপা, অন-স্ক্রিন কীবোর্ডের Send কী (`IME_ACTION_SEND`), অথবা আলাদা (হার্ডওয়্যার) কীবোর্ডের Enter লাইন 63–77। Enter আসে `IME_NULL` হিসেবে, সঙ্গে একটি কী-ঘটনা, আর সেটি আসে দুবার: একবার কী চাপার সময়, একবার ছাড়ার সময়। তাই শুধু চাপার সময় একবার পাঠানো হয়, কিন্তু দুটি ঘটনাই অ্যাপ নিজে গ্রহণ করে নেয় (consume), যাতে বার্তা দুবার না যায়।
- **যাচাই:** `sendMessage()` লেখার দুই পাশের ফাঁকা অংশ ছেঁটে (trim) দেখে কিছু বাকি আছে কি না। ফাঁকা হলে টেক্সট ফিল্ডের নিচে "Please type a message first" ভুলের বার্তা দেখিয়ে থেমে যায়; আবার টাইপ শুরু করলেই বার্তাটি মুছে যায়।
- **লেখা ঠিক থাকলে:** এখানে কোনো ব্রডকাস্ট পাঠানো হয় না। লেখাটি Intent-এর ভেতরে করে Activity 3-এ পাঠানো হয় লাইন 89। কেন এমন নকশা, তা **৮.৭**-এ ব্যাখ্যা করা আছে।
- **কীবোর্ডের ওপরে রাখা:** ম্যানিফেস্টের `adjustResize` সিস্টেমকে কীবোর্ডের ইনসেট পাঠাতে বলে, আর স্ক্রল ভিউ সেই ইনসেট নিজের প্যাডিংয়ে যোগ করে (**৭.৫**)। টাইপ করা লেখাটি ফোন ঘোরালেও থাকে, কারণ id থাকা টেক্সট ফিল্ড নিজের লেখা নিজেই সংরক্ষণ করে।

৮.৫ CustomMessageReceiver: অ্যাপের নিজের সাবস্ক্রাইবার

কোডে কোথায় `CustomMessageReceiver.kt` › **`CustomMessageReceiver`** লাইন 19–51
`CustomMessageReceiver.kt` › **`onReceive()`** লাইন 23–28
`CustomMessageReceiver.kt` › **`ACTION_CUSTOM_MESSAGE, EXTRA_MESSAGE`** লাইন 34–37
`CustomMessageReceiver.kt` › **`intentFilter()`** লাইন 40
`CustomMessageReceiver.kt` › **`createBroadcast()`** লাইন 46–49

`CustomMessageReceiver` হলো BroadcastReceiver-এর একটি উপশ্রেণি (subclass)। এর কাজ একটাই: `onReceive()` এলে Intent থেকে লেখাটি বের করে তৈরির সময় পাওয়া কলব্যাকে পৌঁছে দেওয়া। রিসিভার নিজে স্ক্রিনের কিছু জানে না; লেখা পেয়ে কী করতে হবে তা ঠিক করে Activity। এতে দুটি অংশ আলাদা থাকে, আর আলাদাভাবে পরীক্ষা করা সহজ হয়।

- **টপিকের নাম:** `ACTION_CUSTOM_MESSAGE`-এর মান হলো অ্যাপের applicationId-এর পরে `.action.CUSTOM_MESSAGE`; লেখার এক্সট্রার নাম `EXTRA_MESSAGE`-ও একইভাবে applicationId দিয়ে শুরু। এই উপসর্গ নামটিকে পুরো ফোনে অনন্য রাখে, তাই অন্য কোনো অ্যাপের অ্যাকশনের সঙ্গে ভুল করে মিলে যায় না।

- **সাবস্ক্রিপশনের ফিল্টার:** `intentFilter()` এমন একটি IntentFilter দেয়, যা শুধু এই অ্যাকশন মেলায়।
- **বার্তা তৈরি:** `createBroadcast()` অ্যাকশন, লেখার এক্সট্রা আর `setPackage(packageName)` দিয়ে ব্রডকাস্টের Intent বানায়। `setPackage()` বার্তা পৌঁছানো এই অ্যাপের ভেতরেই সীমিত রাখে, তাই ব্যবহারকারীর লেখা অন্য কোনো অ্যাপ পায় না।
- **গ্রহণ:** `onReceive()` সাবধানতা হিসেবে আবার অ্যাকশন মিলিয়ে দেখে, এক্সট্রা না থাকলে চুপচাপ ফিরে যায়, তারপর Logcat-এ "Received via broadcast: ..." লিখে কলব্যাক ডাকে।
- **ম্যানিফেস্টে নেই কেন:** অ্যাসাইনমেন্ট চায় রিসিভারটি তৃতীয় Activity-তে তৈরি হোক। তাছাড়া এটি শুধু ওই স্ক্রিন খোলা থাকলেই দরকার; ম্যানিফেস্টের রিসিভার স্থায়ী সাবস্ক্রিপশন, যার দরকার এখানে নেই।

`onReceive()` মেইন থ্রেডে চলে (৩.৬), তাই এটি দ্রুত শেষ হওয়া উচিত; দীর্ঘক্ষণ আটকে থাকলে ANR হতে পারে। এখানে শুধু একটি মান হালনাগাদ হয় আর একটি টোস্ট দেখানো হয়, যা মুহূর্তের কাজ।

৮.৬ Activity 3: রেজিস্টার, প্রকাশ, গ্রহণ

কোডে কোথায় `CustomReceiverActivity.kt › onCreate()` লাইন 37–62
`CustomReceiverActivity.kt › showCustomReceiver()` লাইন 64–77
`CustomReceiverActivity.kt › onStart()` লাইন 79–103
`CustomReceiverActivity.kt › onStop()` লাইন 105–112
`CustomReceiverActivity.kt › sendCustomBroadcast() … renderReceived()` লাইন 114–139

`CustomReceiverActivity`-তে পুরো পাবলিশ/সাবস্ক্রাইব চক্রটি একটি স্ক্রিনেই ঘটে (একই যাত্রা পাবলিশ/সাবস্ক্রাইবের দৃষ্টিতে ৪.৯-এ দেখানো হয়েছে)। ক্রম অনুযায়ী:

১. **onCreate:** Intent থেকে ধরন আর লেখা পড়ে। নিজের ব্রডকাস্টের ধরনে লেখা না থাকলে `finish()`। থাকলে স্ক্রিন সাজায় (রিসিভারের অ্যাকশন দেখায়, "Broadcast again" বাটন যুক্ত করে) আর `CustomMessageReceiver` তৈরি করে, যার কলব্যাক হলো `onBroadcastReceived()` লাইন 74।

২. **onStart, সাবস্ক্রাইব:** `ContextCompat.registerReceiver()` দিয়ে, `RECEIVER_NOT_EXPORTED` ফ্ল্যাগসহ, রিসিভার রেজিস্টার হয় লাইন 85–90; স্ক্রিনে লেখা ওঠে "CustomMessageReceiver is registered"।

৩. **onStart, প্রকাশ:** প্রথম ব্রডকাস্ট আগে পাঠানো না হয়ে থাকলে "পাঠানো হয়েছে" ফ্ল্যাগ চালু করে `sendBroadcast()` দিয়ে লেখাটি প্রকাশ করা হয় লাইন 99–102। `sendBroadcast()` সঙ্গে সঙ্গে ফিরে আসে; Activity কারও গ্রহণের জন্য অপেক্ষা করে না।

৪. **মেলানো ও পৌঁছানো:** সিস্টেম Intent-টিকে নিবন্ধিত IntentFilter-গুলোর সঙ্গে মিলিয়ে আমাদের রিসিভার খুঁজে পায়, আর বার্তাটি আমাদের প্রসেসের মেইন থ্রেডের সারিতে রাখে।

৫. **গ্রহণ:** Looper বার্তাটিতে পৌঁছালে `onReceive()` চলে, যা `onBroadcastReceived()` ডাকে লাইন 119–122। এটি ViewModel-কে লেখা আর সময় দেয় (ViewModel পাওয়ার সংখ্যা এক বাড়িয়ে সবকিছু SavedStateHandle-এ রাখে) আর একটি টোস্ট দেখায়।

৬. **স্ক্রিন হালনাগাদ:** পাওয়া বার্তার LiveData-র অবজারভার স্ক্রিনে দেখায় "Received via broadcast: ...", সময় আর মোট কতবার পাওয়া গেছে লাইন 124–139। সংখ্যার সঙ্গে সঠিক বচনের জন্য plurals রিসোর্স ব্যবহার করা হয়।

৭. **আবার পাঠানো:** "Broadcast again" একই লেখা আবার প্রকাশ করে; প্রতিবার গণনা বাড়ে, যা প্রমাণ করে বার্তাটি প্রতিবার সত্যিই রিসিভারের মধ্য দিয়ে আসছে।

৮. **onStop, আনসাবস্ক্রাইব:** `unregisterReceiver()` ডাকা হয় লাইন 108 আর স্ক্রিনে "not registered" দেখায়।

দুটি বিষয় লক্ষ করার মতো। প্রথমত, onStart-এ ক্রমটি গুরুত্বপূর্ণ: আগে রেজিস্টার, তারপর পাঠানো। দ্বিতীয়ত, রেজিস্টার আর আনরেজিস্টার একটি লাইফসাইকেল জোড়ায় (onStart ↔ onStop) থাকায় রিসিভার কখনো মেমরিতে আটকে থাকে না (লিক হয় না), আর স্ক্রিন দেখা না গেলে কোনো বার্তা সামলানো হয় না।

> **RECEIVER_NOT_EXPORTED কেন**
>
> targetSdk 34 (Android 14) বা তার বেশি হলে, সিস্টেমের নয় এমন ব্রডকাস্টের জন্য কোডে রিসিভার রেজিস্টার করার সময় `RECEIVER_EXPORTED` বা `RECEIVER_NOT_EXPORTED` বলে দিতেই হয়; না দিলে SecurityException হয়। NOT_EXPORTED মানে অন্য কোনো অ্যাপ এই রিসিভারে ব্রডকাস্ট পাঠাতে পারবে না। `ContextCompat`-এর সংস্করণটি পুরোনো Android-এও একইভাবে কাজ করে।

৮.৭ হারিয়ে যাওয়া বার্তার সমস্যা

কোডে কোথায় `CustomReceiverActivity.kt` › **`onStart() (ব্যাখ্যাসহ মন্তব্য)`** লাইন 93–102

প্রথম দেখায় মনে হতে পারে, Activity 2-ই ব্রডকাস্ট পাঠাবে আর Activity 3 সেটি গ্রহণ করবে। কিন্তু এই নকশা কাজ করে না।

কারণ: Activity 2 যখন `sendBroadcast()` ডাকত, তখনো Activity 3 তৈরিই হয়নি, তার রিসিভারও রেজিস্টার হয়নি। সাধারণ (স্টিকি নয় এমন) ব্রডকাস্ট সিস্টেম জমিয়ে রাখে না: পাঠানোর মুহূর্তে যারা সাবস্ক্রাইব করা আছে, শুধু তারাই পায়, আর পরে আসা সাবস্ক্রাইবার কিছুই পায় না। এটি রেডিওর মতো ক্ষণস্থায়ী পৌঁছানো (৪.৬)। ফলে বার্তাটি নিঃশব্দে হারিয়ে যেত। তাছাড়া `startActivity()` আর `sendBroadcast()` দুটিই অপেক্ষা না করে ফিরে আসে (অ্যাসিঙ্ক্রোনাস), তাই "আগে Activity খুলি, তারপর পাঠাই" করলেও নিশ্চয়তা থাকত না যে রিসিভার ততক্ষণে প্রস্তুত।

সমাধান দুই ধাপে

১. লেখাটি Activity 2 থেকে Activity 3-এ যায় explicit Intent-এর ভেতরে, অর্থাৎ এক-থেকে-এক পথে, যেখানে পৌঁছানো নিশ্চিত।

২. Activity 3 আগে নিজের রিসিভার রেজিস্টার করে, তারপর একই লেখা ব্রডকাস্ট করে। তাই ব্রডকাস্ট পাঠানোর মুহূর্তে সাবস্ক্রাইবার নিশ্চিতভাবেই উপস্থিত।

এভাবে অ্যাসাইনমেন্টের শর্তও পূরণ হয়: লেখা নেওয়া হয় দ্বিতীয় Activity-তে, আর তৃতীয় Activity-তে তৈরি নিজস্ব রিসিভার সেটি গ্রহণ করে। স্টিকি ব্রডকাস্ট (`sendStickyBroadcast()`) এখানে বিকল্প নয়, কারণ এটি বাতিল (deprecated) আর নিরাপত্তার কারণে নিরুৎসাহিত। Activity 3-এর স্ক্রিনে "Why does this screen send the broadcast?" অংশে এই ব্যাখ্যাটি ব্যবহারকারীর জন্যও লেখা আছে।

৮.৮ ফোন ঘোরালে: বার্তা দুবার যায় না, কিছু হারায়ও না

কোডে কোথায় `CustomReceiverViewModel.kt` › **`CustomReceiverViewModel`** লাইন 16–38
`CustomReceiverViewModel.kt` › **`ReceivedMessage`** লাইন 45–49
`CustomReceiverActivity.kt` › **`onStart()`** লাইন 99–102

ফোন ঘোরালে Activity 3 ভেঙে নতুন করে তৈরি হয়, আর onStart আবার চলে। প্রতিবার onStart-এ ব্রডকাস্ট পাঠালে প্রতিবার ঘোরানোয় একটি বাড়তি বার্তা যেত আর গণনা অকারণে বাড়ত। আবার Activity-র সাধারণ ভেরিয়েবলে রাখলে পাওয়া বার্তাটিও মুছে যেত।

- **দুবার পাঠানো ঠেকানো:** `initialBroadcastSent` একটি ফ্ল্যাগ, যা ViewModel-এর SavedStateHandle-এ থাকে। প্রথমবার পাঠানোর আগেই এটি true করা হয়; পরে onStart ফ্ল্যাগ দেখে আর পাঠায় না। বার্তার ভাষায় একে de-duplication বলে: একই বার্তা একবারই সামলানো (**৩.৫**)।
- **কিছু না হারানো:** শেষ পাওয়া বার্তা (লেখা, সময় আর গণনা নিয়ে একটি `ReceivedMessage`) একই SavedStateHandle-এ থাকে। নতুন Activity নজর রাখা শুরু করামাত্র LiveData সেটি দিয়ে দেয়। `ReceivedMessage` Serializable, যাতে Bundle-এ রাখা যায় আর প্রসেস বন্ধ হয়ে আবার চালু হলেও ফিরে আসে।
- **সাবস্ক্রিপশন:** ঘোরানোর সময় পুরোনো Activity-র onStop-এ আনরেজিস্টার আর নতুনটির onStart-এ আবার রেজিস্টার হয়; সাবস্ক্রিপশন স্ক্রিনের প্রতিটি জীবনের সঙ্গে বাঁধা।

৮.৯ ব্যাটারির পথ: সিস্টেমের ব্রডকাস্ট

কোডে কোথায় `BatteryStatusLiveData.kt` › **`onActive() (মন্তব্যসহ)`** লাইন 29–40
`BroadcastInputActivity.kt` › **`showBatteryStatus()`** লাইন 94–105

`Intent.ACTION_BATTERY_CHANGED` সিস্টেম পাঠায় যখনই ব্যাটারির অবস্থা বদলায়: চার্জের মাত্রা, চার্জার লাগানো বা খোলা, তাপমাত্রা ইত্যাদি। এখানে প্রকাশক সিস্টেমের ব্যাটারি সার্ভিস; আমাদের অ্যাপ শুধু সাবস্ক্রাইবার। এই ব্রডকাস্টের তিনটি বিশেষ বৈশিষ্ট্য:

১. **স্টিকি:** সিস্টেম শেষ ব্রডকাস্টটি রেখে দেয়, আর `registerReceiver()` নিজেই সেই সর্বশেষ Intent ফেরত দেয়। তাই পরের পরিবর্তনের অপেক্ষা না করেই স্ক্রিন সঙ্গে সঙ্গে মান দেখাতে পারে। পাবলিশ/সাবস্ক্রাইবের ভাষায় এটি নোটিশ বোর্ডের মতো শেষ বার্তা রাখা (retained) ব্রডকাস্ট (**৪.১০**)।
২. **সুরক্ষিত (protected):** শুধু সিস্টেমই এটি পাঠাতে পারে; কোনো অ্যাপ নকল ব্যাটারি-বার্তা পাঠাতে পারে না। এ কারণে এটি Android 14-এর এক্সপোর্ট-ফ্ল্যাগের নিয়মের বাইরে, তাই ফ্ল্যাগ ছাড়াই রেজিস্টার করা যায়।
৩. **শুধু কোডে রেজিস্টার করা রিসিভার পায়:** ম্যানিফেস্টে লেখা রিসিভার এই ব্রডকাস্ট কখনো পায় না।

ব্যাটারির অংশে দেখানো হয় বড় করে শতাংশ, একটি গোল অগ্রগতির রিং, চার্জের অবস্থা ("Status: Charging" ইত্যাদি), শেষ খবরের সময়, আর একটি নোট যে স্ক্রিন দেখা গেলেই শুধু শোনা হয়। "Next" বাটন Activity 3 খোলে। আড়াআড়ি অবস্থার জন্য এই অংশের আলাদা লেআউট আছে।

৮.১০ BatteryStatusLiveData: শুধু দেখা গেলে শোনা

কোডে কোথায় `BatteryStatusLiveData.kt` › **`BatteryStatusLiveData`** লাইন 17–45
`BatteryViewModel.kt` › **`BatteryViewModel`** লাইন 14–17
`BroadcastInputActivity.kt` › **`observe(this)`** লাইন 104

ব্যাটারির রিসিভার সরাসরি Activity-তে না রেখে একটি বিশেষ LiveData-র ভেতরে রাখা হয়েছে: `BatteryStatusLiveData`। LiveData নিজেই জানে কখন তার অন্তত একজন সক্রিয় অবজারভার আছে (যার লাইফসাইকেল অন্তত STARTED): প্রথম সক্রিয় অবজারভার এলে `onActive()`, আর শেষজন চলে গেলে `onInactive()` ডাকা হয়।

- **`onActive()`, সাবস্ক্রাইব:** অ্যাপের context (application context) দিয়ে `ACTION_BATTERY_CHANGED`-এর জন্য রিসিভার রেজিস্টার করা হয়; ফেরত পাওয়া স্টিকি Intent থেকে সঙ্গে সঙ্গে প্রথম মান বসানো হয় লাইন 37–39।
- **`onReceive()`:** প্রতিটি নতুন ব্রডকাস্টে মান হালনাগাদ হয় লাইন 22–27। কোডে রেজিস্টার করা রিসিভার মেইন থ্রেডে চলে, তাই সেখান থেকে সরাসরি LiveData-র মান বসানো নিরাপদ।
- **`onInactive()`, আনসাবস্ক্রাইব:** `unregisterReceiver()` লাইন 42–44।

Activity 2 নিজেকে লাইফসাইকেলের মালিক হিসেবে দিয়ে LiveData-টির ওপর নজর রাখে। ফলে Activity-র onStart-এ LiveData সক্রিয় হয় (রিসিভার রেজিস্টার) আর onStop-এ নিষ্ক্রিয় হয় (আনরেজিস্টার)। হোম বাটন চাপলে, Next দিয়ে Activity 3-এ গেলে বা ফোন ঘোরানোর সময় শোনা বন্ধ থাকে; ফিরে এলে আবার শুরু হয়, আর স্টিকি মান থেকে সঙ্গে সঙ্গে সর্বশেষ অবস্থা পাওয়া যায়। Activity নিজে রেজিস্টার বা আনরেজিস্টার নিয়ে কিছুই জানে না, শুধু নজর রাখে।

- **অ্যাপের context কেন:** এটি পুরো অ্যাপের সমান দীর্ঘজীবী, তাই LiveData কোনো Activity-কে ধরে রাখে না আর লিক হতে পারে না লাইন 20।
- **`BatteryViewModel`:** একটি AndroidViewModel, যা LiveData-র একটিমাত্র কপি ফোন ঘোরানোর মধ্যেও ধরে রাখে; নতুন Activity নজর রাখা শুরু করামাত্র শেষ মানটি পায়।
- **দরকার হলে তবেই তৈরি:** ViewModel-টি প্রথম ব্যবহারের সময় তৈরি হয় (lazy) লাইন 31, তাই নিজের ব্রডকাস্টের পথে ব্যাটারির LiveData বা তার রিসিভার কখনো তৈরিই হয় না।

৮.১১ ব্যাটারির Intent থেকে শতাংশ ও অবস্থা

কোডে কোথায় `BatteryStatus.kt` › **`percentOf()`** লাইন 31–34
`BatteryStatus.kt` › **`fromIntent()`** লাইন 37–49 `BatteryStatus.kt` › **`ChargingStatus`** লাইন 54–71
`BroadcastInputActivity.kt` › **`renderBattery()`** লাইন 107–120
`TimeFormat.kt` › **`formatTimeOfDay()`** লাইন 25–29

`ACTION_BATTERY_CHANGED` Intent-এর এক্সট্রায় অনেক তথ্য থাকে; এখানে তিনটি ব্যবহার করা হয়: `EXTRA_LEVEL` (বর্তমান মাত্রা), `EXTRA_SCALE` (সর্বোচ্চ মাত্রা) আর `EXTRA_STATUS` (চার্জের অবস্থা)। `BatteryStatus.fromIntent()` এগুলো পড়ে একটি `BatteryStatus` data class তৈরি করে।

- **শতাংশের সূত্র:** শতাংশ = `EXTRA_LEVEL` × ১০০ ÷ `EXTRA_SCALE`। মাত্রাকে সরাসরি শতাংশ ধরে নেওয়া ভুল, কারণ সব ফোনে সর্বোচ্চ মাত্রা ১০০ নয়।
- **সুরক্ষা:** `percentOf()` মাত্রা না থাকলে (তখন `getIntExtra()` ডিফল্ট হিসেবে −১ দেয়) বা সর্বোচ্চ মাত্রা শূন্য বা ঋণাত্মক হলে null দেয়, যাতে শূন্য দিয়ে ভাগ না হয়; স্ক্রিনে তখন "—" দেখায়। ফল সবসময় ০ থেকে ১০০-এর মধ্যে আটকে রাখা হয়।
- **চার্জের অবস্থা:** `EXTRA_STATUS`-এর মান (চার্জ হচ্ছে, চার্জ কমছে, পূর্ণ, চার্জ হচ্ছে না) `ChargingStatus` enum-এ রূপান্তরিত হয়; অপ্রত্যাশিত যেকোনো মান হয় `UNKNOWN`। প্রতিটি মানের সঙ্গে স্ক্রিনে দেখানোর লেখার রিসোর্স যুক্ত।
- **সময়:** `receivedAtMillis` হলো অ্যাপ কখন ব্রডকাস্টটি পড়ল। `formatTimeOfDay()` ব্যবহারকারীর ১২ বা ২৪ ঘণ্টার সেটিং মেনে সেকেন্ডসহ সময় দেখায়, যাতে প্রতিটি নতুন খবর চোখে পড়ে।
- **অগ্রগতির রিং:** শতাংশ অনুযায়ী অ্যানিমেশনসহ গোল রিং ভরে ওঠে; মান অজানা হলে ০।

শতাংশ আর অবস্থা নির্ণয়ের হিসাব স্ক্রিনের কোড থেকে আলাদা ছোট ফাংশনে রাখা, তাই ইউনিট টেস্ট দিয়ে প্রান্তিক অবস্থাগুলো (সর্বোচ্চ মাত্রা শূন্য, মাত্রা না থাকা) যাচাই করা হয়েছে। এমুলেটরের ব্যাটারি কন্ট্রোল বা adb দিয়ে মাত্রা বদলালে স্ক্রিনের মান সঙ্গে সঙ্গে বদলায়।

৮.১২ ব্যাটারির পথ, Activity 3: কিছুই না করা

কোডে কোথায় `CustomReceiverActivity.kt` › **`showNothingToDo()`** লাইন 147–150
`CustomReceiverActivity.kt` › **`onStart()`** লাইন 81

অ্যাসাইনমেন্ট বলে, ব্যাটারি বেছে নিলে তৃতীয় Activity-তে কিছু করার নেই। `CustomReceiverActivity` এই ধরনে কোনো রিসিভার তৈরি বা রেজিস্টার করে না; রিসিভারের ভেরিয়েবল null থাকে, তাই onStart আর onStop কিছুই করে না। স্ক্রিনে শুধু "Nothing to do here" আর কারণটি লেখা থাকে: লাইভ শতাংশ আগের স্ক্রিনেই দেখানো হয়।

৮.১৩ নিরাপত্তা: কে পাঠাতে পারে, কে শুনতে পারে

ব্রডকাস্ট একটি খোলা ব্যবস্থা, তাই পাবলিশ/সাবস্ক্রাইবের প্রতিটি দিক আলাদাভাবে সুরক্ষিত করা হয়েছে:

ব্যবস্থা	কী নিয়ন্ত্রণ করে	কোডে কোথায়
`setPackage()`	নিজের ব্রডকাস্ট কে পাবে: শুধু এই অ্যাপের রিসিভার।	`CustomMessageReceiver.kt` › **`createBroadcast()`** লাইন 46–49
`RECEIVER_NOT_EXPORTED`	কে এই রিসিভারে পাঠাতে পারে: শুধু এই অ্যাপ (আর সিস্টেম)।	`CustomReceiverActivity.kt` › **`onStart()`** লাইন 85–90
সুরক্ষিত ব্রডকাস্ট	`ACTION_BATTERY_CHANGED` কে পাঠাতে পারে: শুধু সিস্টেম।	`BatteryStatusLiveData.kt` › **`onActive()`** লাইন 30–38
applicationId উপসর্গ	নিজের অ্যাকশনের নাম অনন্য রাখে; অন্য অ্যাপের নামের সঙ্গে মেলে না।	`CustomMessageReceiver.kt` › **`ACTION_CUSTOM_MESSAGE`** লাইন 34

ব্যবস্থা	কী নিয়ন্ত্রণ করে	কোডে কোথায়
`exported="false"`	Activity 2 ও 3 অন্য অ্যাপ সরাসরি খুলতে পারে না; শুধু লঞ্চারের `MainActivity` এক্সপোর্ট করা।	`AndroidManifest.xml` › **`manifest`** লাইন 25–46

৯ ফিচার B: Image Scale

> **সহজ কথায়**
> অ্যাপ ইন্টারনেট থেকে একটি বড় ছবি নামায়, আর ছবিটি দুই আঙুলে বড়-ছোট, এক আঙুলে সরানো আর ডাবল-ট্যাপে জুম করা যায়। ছবি কোথায় কত বড় আঁকা হবে, তা ঠিক করে একটি Matrix (গাণিতিক নিয়ম)। দুই আঙুলের মাঝের বিন্দু ঘিরে জুম হয়, তাই আঙুলের নিচের অংশ আঙুলের নিচেই থাকে। ফোন ঘোরালেও জুম আর দেখা অংশ একই থাকে।

সংশ্লিষ্ট ফাইল (`image` প্যাকেজ): `ImageScaleFragment` · `ImageScaleViewModel` · `ZoomableImageView` · `ZoomMath` · `res/layout/fragment_image_scale.xml`

এই স্ক্রিনে দুটি আলাদা কাজ: ইন্টারনেট থেকে একটি ছবি নামানো, আর পিঞ্চ করে সেটি জুম করা। জুমের জন্য কোনো লাইব্রেরি নেই; ছোঁয়ার ঘটনা থেকে শুরু করে Matrix-এর গণিত পর্যন্ত সবকিছু নিজস্ব `ZoomableImageView`-এ লেখা।

৯.১ ইন্টারনেট থেকে ছবি: Coil 3

কোডে কোথায় `ImageScaleViewModel.kt › load()` লাইন 45–64
`ImageScaleViewModel.kt › ImageUiState` লাইন 20–24 · `ImageScaleViewModel.kt › init` লাইন 40–42
`ImageScaleViewModel.kt › IMAGE_URL` লাইন 68 · `AndroidManifest.xml › INTERNET পারমিশন` লাইন 6

ছবিটি Lorem Picsum থেকে নেওয়া ২০০০ × ১৩০০ পিক্সেলের একটি ছবি। ছবি নামাতে ব্যবহার হয় Coil 3, Kotlin কোরুটিন-ভিত্তিক একটি ছবি লোড করার লাইব্রেরি। ইন্টারনেট থেকে নামানো (OkHttp দিয়ে), ডিকোড করা আর মেমরি ও ডিস্কে জমিয়ে রাখা (cache), সব এটি নিজে সামলায়, আর ভারী কাজগুলো পেছনের থ্রেডে করে।

- **অনুরোধ:** `load()` একটি ImageRequest তৈরি করে, সঙ্গে `Size.ORIGINAL` লাইন 54 । Coil সাধারণত ছবিকে ভিউয়ের মাপে ছোট করে ডিকোড করে, কিন্তু সেই ছোট ছবিতে জুম করলে ছবি ঝাপসা দেখাত। তাই মূল রেজোলিউশন চাওয়া হয়, যাতে জুম করলেও ছবি যথেষ্ট পরিষ্কার থাকে।
- **অনুরোধ-উত্তর:** ViewModel-এর নিজস্ব কোরুটিন-পরিসরে (viewModelScope) চলা একটি কোরুটিনে `imageLoader.execute(request)` ডাকা হয় লাইন 56 । এটি একটি সাসপেন্ড ফাংশন: অনুরোধ পাঠিয়ে কোরুটিন থেমে অপেক্ষা করে, কিন্তু মেইন থ্রেড আটকায় না; উত্তর এলে কোরুটিন যেখানে থেমেছিল সেখান থেকেই আবার চলে। মেসেজ কিউয়ের ভাষায়: ধীর কাজ অন্য থ্রেডে, ফল আবার মেইন থ্রেডে (৩.৮)।
- **ফল একটি স্টেটে:** `ImageUiState`-এর তিনটি রূপ: Loading, Success (ছবিসহ) আর Error। Coil সফল হলে Success, ব্যর্থ হলে লগে সতর্কবার্তা লিখে Error। এই স্টেট থাকে একটি StateFlow-এ, যা স্ক্রিনের জন্য নোটিশ বোর্ডের মতো শেষ মান ধরে রাখে (৪.৭)।
- **ফোন ঘোরালে আবার ডাউনলোড নয়:** `load()` চলে ViewModel তৈরির সময়। ঘোরানোর সময় ViewModel বেঁচে থাকে, তাই ডাউনলোড আবার শুরু হয় না; নতুন Fragment শুধু আগের স্টেট সংগ্রহ করে।

- **Retry:** `load()` আগের কাজ (job) বাতিল করে, স্টেটকে আবার Loading করে আর নতুন করে চেষ্টা করে।

৯.২ স্ক্রিনে স্টেট দেখানো

কোডে কোথায় `ImageScaleFragment.kt › onViewCreated()` লাইন 32–47
`ImageScaleFragment.kt › render()` লাইন 49–60 `ImageScaleFragment.kt › shownImage` লাইন 30

`ImageScaleFragment` স্টেট সংগ্রহ করে `repeatOnLifecycle(STARTED)`-এর ভেতরে: onStart-এ সংগ্রহ শুরু, onStop-এ বাতিল; অর্থাৎ স্ক্রিন দেখা গেলেই শুধু সাবস্ক্রাইব। প্রতিটি নতুন স্টেটে `render()` চলে:

- Loading হলে একটি ঘুরন্ত অগ্রগতির চিহ্ন;
- Error হলে "Couldn't load the image…" বার্তা আর Retry বাটন;
- Success হলে ছবি, বর্তমান জুমের লেবেল (যেমন "1.0x") আর পরামর্শ "Pinch to zoom · double-tap to reset"।

একটি সূক্ষ্ম বিষয় আছে: StateFlow প্রতিটি নতুন সংগ্রাহককে সর্বশেষ মানটি আবার দেয়, যেমন অন্য অ্যাপ থেকে ফিরে এলে onStart-এ। একই ছবি আবার ভিউতে বসালে জুম শুরুর অবস্থায় ফিরে যেত। তাই Fragment বর্তমান ছবিটি (`shownImage`) মনে রাখে, আর শুধু সত্যিই নতুন ছবি এলে বসায়। জুম বদলালে `ZoomableImageView`-এর `onZoomChanged` কলব্যাক লেবেলটি হালনাগাদ করে।

৯.৩ ছবির Matrix ও "ফিট"

কোডে কোথায় `ZoomableImageView.kt › init (scaleType = MATRIX)` লাইন 70–73
`ZoomableImageView.kt › refit()` লাইন 104–118 `ZoomMath.kt › fitScale()` লাইন 22–23
`ZoomableImageView.kt › setImageDrawable()` লাইন 80–83

`ZoomableImageView` হলো AppCompatImageView-এর একটি উপশ্রেণি, যার scaleType হলো `MATRIX`। এই অবস্থায় ImageView ছবিকে কোথায়, কত বড় করে আঁকবে, তা পুরোপুরি একটি Matrix ঠিক করে।

Matrix হলো ৩×৩ সংখ্যার একটি গাণিতিক রূপান্তর (affine transformation), যা ছবির প্রতিটি বিন্দুর স্থানাঙ্ককে ভিউয়ের স্থানাঙ্কে নিয়ে যায়। এতে বড়-ছোট করা (scale) আর সরানো (translate), দুটিই থাকে। `postScale()` আর `postTranslate()` বর্তমান রূপান্তরের পরে নতুন একটি রূপান্তর জুড়ে দেয়। `mapRect()` আর `mapPoints()` দিয়ে জানা যায় ছবির কোনো বিন্দু বা আয়তক্ষেত্র ভিউয়ের কোথায় আঁকা হচ্ছে, আর উল্টো (inverse) Matrix দিয়ে উল্টো দিকে, অর্থাৎ ভিউয়ের বিন্দু থেকে ছবির বিন্দুতে, যাওয়া যায়।

- **ফিট:** পুরো ছবি না বাঁকিয়ে ভিউয়ের ভেতরে দেখাতে স্কেল হয় (ভিউয়ের প্রস্থ ÷ ছবির প্রস্থ) আর (ভিউয়ের উচ্চতা ÷ ছবির উচ্চতা), এই দুইয়ের মধ্যে যেটি ছোট, অর্থাৎ যে দিকে জায়গা কম সেদিক অনুযায়ী। তারপর ছবিকে মাঝখানে আনতে সরানো হয়। এটি ImageView-এর FIT_CENTER-এর মতোই।
- **জুমের সংজ্ঞা:** জুম = বর্তমান স্কেল ÷ ফিটের স্কেল। তাই জুম ১ মানে পুরো ছবি দেখা যাচ্ছে; ২.৫ মানে ফিটের আড়াই গুণ বড়।
- **আবার ফিট (`refit()`):** ছবি বা ভিউয়ের মাপ বদলালেই ফিট নতুন করে হিসাব হয়। এটি জরুরি, কারণ ছবি ইন্টারনেট থেকে পরে আসে, সাধারণত ভিউ প্রথমবার আঁকার পরে। ছবি আর মাপ দুটোই না জানা পর্যন্ত `refit()` কিছুই করে না।

- **প্যাডিং:** সব হিসাব ভিউয়ের প্যাডিং বাদ দিয়ে আঁকার এলাকার ওপর করা হয়।

৯.৪ পিঞ্চ জুম: ScaleGestureDetector

কোডে কোথায় `ZoomableImageView.kt` › **`ScaleListener`** লাইন 196–211
`ZoomableImageView.kt` › **`zoomBy()`** লাইন 126–133 `ZoomMath.kt` › **`clampZoom()`** লাইন 16
`ZoomableImageView.kt` › **`scaleDetector`** লাইন 58–61

ScaleGestureDetector হলো Android-এর তৈরি একটি অঙ্গভঙ্গি-শনাক্তকারী (gesture detector), যা দুই আঙুলের ছোঁয়ার ঘটনা বিশ্লেষণ করে প্রতি মুহূর্তে দুটি জিনিস দেয়: **scaleFactor**, অর্থাৎ আগের ঘটনা থেকে দুই আঙুলের দূরত্ব কত গুণ বদলেছে (১-এর বেশি হলে আঙুল ছড়াচ্ছে), আর **ফোকাল পয়েন্ট** (focal point), অর্থাৎ দুই আঙুলের মাঝের বিন্দু।

১. `onScaleBegin()`: শুরুর ফোকাল পয়েন্ট মনে রাখে।
২. `onScale()`: প্রথমে ফোকাল পয়েন্ট যতটা সরেছে, ছবিকেও ততটা সরায়, ফলে দুই আঙুলে টেনেও ছবি সরানো যায়। তারপর `zoomBy()` ডাকে, scaleFactor আর ফোকাল পয়েন্ট দিয়ে।
৩. `zoomBy()`: লক্ষ্য জুম = বর্তমান জুম × গুণক, যা ১ থেকে ৫ গুণের মধ্যে আটকানো। আসলে প্রয়োগ করা হয় লক্ষ্য ÷ বর্তমান জুম; সীমায় পৌঁছালে এই অনুপাত ১ হয়ে যায়, তাই ছবি আর বড় বা ছোট হয় না। এরপর ফোকাল পয়েন্ট ঘিরে `postScale()`, তারপর কিনারা ঠিক করা আর Matrix প্রয়োগ।

ফোকাল পয়েন্ট ঘিরে স্কেল কেন: কোনো বিন্দু f-কে কেন্দ্র করে s গুণ বড় করলে ছবির যেকোনো বিন্দু p চলে যায় f + s × (p − f)-এ। p যদি ঠিক f হয়, তবে সে f-এই থাকে। অর্থাৎ আঙুলের নিচে ছবির যে অংশ ছিল, জুমের পরেও সেটি আঙুলের নিচেই থাকে; ব্যবহারকারীর মনে হয় তিনি ছবির ঠিক সেই জায়গাটিই টেনে বড় করছেন।

ডিটেক্টরের quick scale (ডাবল-ট্যাপ করে আঙুল টেনে জুম) বন্ধ রাখা হয়েছে, কারণ এটি ডাবল-ট্যাপের সঙ্গে গুলিয়ে যেত। জুম শুধু দুই আঙুলের পিঞ্চ দিয়েই হয়।

৯.৫ এক আঙুলে সরানো ও কিনারা সামলানো

কোডে কোথায় `ZoomableImageView.kt` › **`onScroll()`** লাইন 216–221
`ZoomableImageView.kt` › **`panBy()`** লাইন 135–139
`ZoomableImageView.kt` › **`clampTranslation()`** লাইন 142–150
`ZoomMath.kt` › **`edgeCorrection()`** লাইন 31–39

দ্বিতীয় ডিটেক্টর, GestureDetector, এক আঙুলের টানা (scroll), ডাবল-ট্যাপ আর একক ট্যাপ চেনে। `onScroll()` জানায় আগের অবস্থান থেকে আঙুল কতটা সরেছে (উল্টো চিহ্নে), তাই চিহ্ন উল্টে নিয়ে ছবিকে ঠিক ততটা সরানো হয়; ফলে ছবি আঙুলের সঙ্গে সঙ্গে চলে। সরানো যায় শুধু তখনই, যখন জুম ১-এর বেশি আর পিঞ্চ চলছে না; জুম ১-এ পুরো ছবি আগেই দেখা যাচ্ছে, সরানোর কিছু নেই।

প্রতিটি জুম বা সরানোর পর `clampTranslation()` চলে। এটি ছবির চার কোনা Matrix দিয়ে ভিউয়ের স্থানাঙ্কে রূপান্তর করে, তারপর আড়াআড়ি আর খাড়া, প্রতিটি অক্ষে আলাদাভাবে `edgeCorrection()` হিসাব করে:

- ছবি ওই অক্ষে ভিউয়ের চেয়ে ছোট হলে: মাঝখানে আনা;
- বড় হলে আর শুরুর দিকে (বাঁয়ে বা ওপরে) ফাঁক থাকলে: ফাঁক বন্ধ হওয়া পর্যন্ত টেনে আনা;

- শেষের দিকে (ডানে বা নিচে) ফাঁক থাকলে: সেদিকে টেনে আনা;
- নইলে কিছু না করা।

ফলে জুম করা ছবি যতই সরানো হোক, কিনারায় কখনো খালি ফাঁক দেখা যায় না। `ZoomMath`-এর এই ফাংশনগুলো Android-এর কোনো ক্লাস ব্যবহার করে না, তাই সাধারণ ইউনিট টেস্ট দিয়ে সহজে যাচাই করা যায়; ভিউয়ের কোড শুধু ছোঁয়ার ঘটনা আর Matrix সামলায়।

৯.৬ ডাবল-ট্যাপ

কোডে কোথায় `ZoomableImageView.kt › onDoubleTap()` লাইন 223–227
`ZoomableImageView.kt › animateZoomTo()` লাইন 158–168
`ZoomableImageView.kt › DOUBLE_TAP_ZOOM, ZOOM_EPSILON, ZOOM_ANIMATION_MS` লাইন 294–296

- **লক্ষ্য ঠিক করা:** `onDoubleTap()`-এ বর্তমান জুম ১-এর চেয়ে বেশি হলে লক্ষ্য ১ (শুরুর অবস্থা), নইলে ২.৫ গুণ, ট্যাপ করা বিন্দুকে কেন্দ্র করে। "বেশি" যাচাইয়ে ০.০১-এর একটি ছোট ছাড় রাখা হয়, কারণ দশমিকের হিসাবে জুম ঠিক ১.০ না হয়ে সামান্য বেশি হতে পারে।
- **মসৃণ অ্যানিমেশন:** একটি ValueAnimator ২৫০ মিলিসেকেন্ড ধরে বর্তমান জুম থেকে লক্ষ্য পর্যন্ত মান তৈরি করে, DecelerateInterpolator দিয়ে (শুরুতে দ্রুত, শেষে ধীর)। প্রতিটি ফ্রেমে একই `zoomBy()` ডাকা হয়, তাই সীমা আর কিনারার নিয়ম অ্যানিমেশনের সময়ও বহাল থাকে। প্রতিটি ফ্রেম মেইন থ্রেডের সারিতে একটি করে বার্তা।
- **নিয়ন্ত্রণ ব্যবহারকারীর হাতে:** নতুন ছোঁয়া শুরু হলে চলমান অ্যানিমেশন সঙ্গে সঙ্গে বাতিল হয় লাইন 172। ভিউ জানালা থেকে সরে গেলেও অ্যানিমেশন বাতিল হয়।

৯.৭ ছোঁয়ার ঘটনা: একটি বার্তার ধারা

কোডে কোথায় `ZoomableImageView.kt › onTouchEvent()` লাইন 170–187
`ZoomableImageView.kt › requestDisallowInterceptTouchEvent` লাইন 178–180

আঙুলের প্রতিটি নড়াচড়া একটি MotionEvent হিসেবে মেইন থ্রেডের সারি দিয়ে আসে (৩.৭): `ACTION_DOWN` (প্রথম আঙুলের ছোঁয়া), `ACTION_POINTER_DOWN` (দ্বিতীয় আঙুল), অনেকগুলো `ACTION_MOVE`, তারপর `ACTION_POINTER_UP` আর `ACTION_UP`। একটি অঙ্গভঙ্গি আসলে এই বার্তাগুলোর একটি ধারা।

- `onTouchEvent()` প্রতিটি ঘটনা দুই ডিটেক্টরকেই দেয়; প্রত্যেকে ধারাটি থেকে নিজের অঙ্গভঙ্গি (পিঞ্চ, টানা বা ডাবল-ট্যাপ) চিনে নেয়।
- true ফেরত দেওয়া মানে ঘটনাটি এই ভিউ নিজে গ্রহণ করে নিল (consume): এই ভিউই অঙ্গভঙ্গির মালিক। মেসেজ কিউয়ের মতোই, একটি ঘটনা একজনই গ্রহণ করে।
- **ওপরের ভিউকে আটকানো:** DrawerLayout পাশ থেকে টানাকে ড্রয়ার খোলার জন্য নিজের দিকে নিতে চায়। জুম ১-এর বেশি হলে বা একাধিক আঙুল থাকলে `requestDisallowInterceptTouchEvent(true)` ডাকা হয়, ফলে ওপরের (parent) ভিউ এই অঙ্গভঙ্গি কেড়ে নিতে পারে না, আর সরানো ও পিঞ্চ নির্বিঘ্নে চলে।

- **অ্যাক্সেসিবিলিটি:** সাধারণ একক ট্যাপে এখনো `performClick()` ডাকা হয়, যা TalkBack-এর মতো সহায়ক সেবার (accessibility service) জন্য দরকারি।
- ছবি না থাকলে ভিউটি সাধারণ ImageView-এর মতো আচরণ করে।

৯.৮ ফোন ঘোরালে জুম ধরে রাখা

কোডে কোথায় `ZoomableImageView.kt › captureState()` লাইন 243–256
`ZoomableImageView.kt › restore()` লাইন 259–266
`ZoomableImageView.kt › onSaveInstanceState()` লাইন 268–275
`ZoomableImageView.kt › onRestoreInstanceState()` লাইন 277–291
`ZoomableImageView.kt › onSizeChanged()` লাইন 85–98

ফোন ঘোরালে Fragment-এর ভিউ নতুন করে তৈরি হয়, অর্থাৎ নতুন একটি `ZoomableImageView` আসে, আর তার মাপও বদলায় (খাড়া থেকে আড়াআড়ি)। শুধু জুমের সংখ্যা রাখলে যথেষ্ট নয়; ছবির কোন অংশটি দেখা যাচ্ছিল, তাও রাখতে হয়।

- **কী রাখা হয়:** জুম, আর ভিউয়ের ঠিক মাঝখানে ছবির কোন বিন্দু ছিল, ছবির প্রস্থ ও উচ্চতার ভগ্নাংশ (০ থেকে ১) হিসেবে। পিক্সেলে না রেখে ভগ্নাংশে রাখায় ভিন্ন মাপের ভিউতেও ছবির একই অংশ আবার দেখানো যায়।
- **মাপা:** `captureState()` উল্টো Matrix দিয়ে ভিউয়ের কেন্দ্রবিন্দুকে ছবির স্থানাঙ্কে নিয়ে যায়, তারপর ছবির মাপ দিয়ে ভাগ করে।
- **সংরক্ষণ:** `onSaveInstanceState()` ওপরের ক্লাসের (superclass) সংরক্ষিত অবস্থার সঙ্গে জুম আর কেন্দ্রের দুই ভগ্নাংশ একটি Bundle-এ রাখে। ভিউয়ের id থাকায় সিস্টেম নিজেই এটি ডাকে।
- **ফিরিয়ে আনা:** `onRestoreInstanceState()` মানগুলো সঙ্গে সঙ্গে প্রয়োগ না করে অপেক্ষমাণ হিসেবে রাখে, কারণ তখনো ছবি বা ভিউয়ের মাপ না-ও জানা থাকতে পারে। `refit()` যখন দুটোই পায়, তখন আগে ফিট করে, তারপর সংরক্ষিত জুম প্রয়োগ করে, শেষে সংরক্ষিত বিন্দুটি ভিউয়ের মাঝখানে আনতে ছবি সরায় (`restore()`)।
- **মাপ বদল:** আগে থেকেই আঁকা ভিউয়ের মাপ বদলালে (যেমন স্ক্রিন ভাগ করে দুটি অ্যাপ চালালে) `onSizeChanged()` পুরোনো মাপের হিসাবে অবস্থা নিয়ে নতুন মাপে আবার প্রয়োগ করে; জুম আর কেন্দ্র অটুট থাকে।

এদিকে ছবিটি নিজে ViewModel-এর StateFlow-এ থাকায় ফোন ঘোরানোর পর আবার ডাউনলোড হয় না; নতুন ভিউ সঙ্গে সঙ্গে ছবি পায়, আর `refit()` সংরক্ষিত জুম ফিরিয়ে আনে।

১০ ফিচার C: Video

> **সহজ কথায়**
> ভিডিও চলে VideoView দিয়ে, আর নিচের play/pause ও সামনে-পেছনে যাওয়ার কন্ট্রোল দেয় MediaController। স্ক্রিন সরে যেতে শুরু করলেই (onPause) ভিডিও কোথায় ছিল আর চলছিল কি না, তা মনে রাখা হয়। শুধু ফোন ঘোরালে ভিডিও আবার নিজে থেকে চলে; সত্যিই স্ক্রিন ছেড়ে গেলে থেমে থাকে।

সংশ্লিষ্ট ফাইল (`video` প্যাকেজ): `VideoFragment` · `VideoViewModel` · `res/layout/fragment_video.xml` · `res/raw/sample_video.mp4`

ভিডিও চালানো সহজ শোনালেও আসল চ্যালেঞ্জ হলো স্ক্রিনের লাইফসাইকেলের সঙ্গে প্লেয়ারকে মেলানো: ফোন ঘোরালে চলতে থাকা, স্ক্রিন ছাড়লে থামা, আর ফিরে এলে ঠিক একই জায়গা থেকে শুরু করা।

১০.১ VideoView ও MediaController

কোডে কোথায় `VideoFragment.kt` › **`onViewCreated()`** লাইন 40–59
`VideoFragment.kt` › **`setMediaController()`** লাইন 50 · `VideoFragment.kt` › **`setVideoURI()`** লাইন 54

VideoView হলো একটি SurfaceView (ছবি আঁকার একটি বিশেষ পৃষ্ঠ), যার ভেতরে `android.media.MediaPlayer` কাজ করে: ভিডিও ডিকোড করে প্রতিটি ফ্রেম সেই পৃষ্ঠে আঁকে আর শব্দ বাজায়। ভিডিওর প্রস্থ-উচ্চতার অনুপাত (aspect ratio) অনুযায়ী নিজের মাপ ঠিক করে, তাই ছবি টেনে লম্বা বা চ্যাপ্টা হয় না।

ভিডিও ফাইলটি অ্যাপের ভেতরেই থাকে: `res/raw/sample_video.mp4` (Blender Foundation-এর Sintel ট্রেলার)। তাই ইন্টারনেট লাগে না। VideoView-কে এর ঠিকানা দেওয়া হয় একটি `android.resource://` URI হিসেবে, যেখানে অ্যাপের প্যাকেজের নাম আর রিসোর্সের id থাকে।

MediaController হলো Android-এর তৈরি নিয়ন্ত্রণ প্যানেল: play/pause, আগে-পিছে যাওয়া, সামনে-পেছনে যাওয়ার দাগ (seek bar), আর চলমান ও মোট সময়। `setMediaController()` দিয়ে যুক্ত করলে VideoView এটিকে নিজের ওপরের ভিউয়ের (কালো ফ্রেমের) নিচের কিনারায় বসায়। MediaController নিজে কোনো ভিডিও জানে না; সে VideoView-কে MediaPlayerControl ইন্টারফেসের মাধ্যমে চালায়: চালু করা, থামানো, নির্দিষ্ট জায়গায় যাওয়া, বর্তমান অবস্থান জানা ইত্যাদি। ভিডিওতে ট্যাপ করলে কন্ট্রোল দেখা যায় বা লুকায়, আর কয়েক সেকেন্ড পর নিজে থেকেই লুকিয়ে যায়।

১০.২ প্লেয়ারের তিনটি ঘটনা

কোডে কোথায় `VideoFragment.kt` › **লিসেনার যুক্ত করা** লাইন 51–53
`VideoFragment.kt` › **`onVideoPrepared()`** লাইন 102–112
`VideoFragment.kt` › **`onVideoError()`** লাইন 131–136
`VideoFragment.kt` › **`onVideoCompleted()`** লাইন 114–118

ভিডিওর ঠিকানা দেওয়ার পর VideoView অপেক্ষা না করে পেছনে সেটি প্রস্তুত (prepare) করে; ততক্ষণ একটি লোডিং চিহ্ন দেখা যায়। এরপর প্লেয়ার তিনটি ঘটনার খবর দেয়, আর প্রতিটিই মেইন থ্রেডের সারি দিয়ে আসা একটি বার্তা (৩.৭):

- **onPrepared:** ভিডিও প্রস্তুত বলে চিহ্নিত হয়, লোডিং লুকানো হয়, আর ViewModel-এ রাখা অবস্থানে যাওয়া হয় (seek)। আগে চলছিল (আবার চালানোর কথা) হলে ভিডিও আবার চালু হয়; নইলে কন্ট্রোল কিছুক্ষণ দেখানো হয়, যাতে play বাটন সহজে চোখে পড়ে।
- **onError:** লগে ভুল লিখে লোডিং লুকানো হয়, আর একটি সহজ বার্তা দেখানো হয় ("Sorry, this video can't be played on this device.")। true ফেরত দেওয়া মানে ভুলটি সামলানো হয়েছে, তাই সিস্টেমের "Can't play this video" ডায়ালগ আসে না।
- **onCompletion:** ভিডিও শেষ হলে ViewModel-এ অবস্থান ০ করা হয় আর আবার চালানো বন্ধ রাখা হয়, যাতে পরেরবার শুরু থেকে চলে; আর কন্ট্রোল আবার দেখানো হয়, যাতে আবার চালানো এক ট্যাপ দূরে থাকে।

১০.৩ থামানো ও মনে রাখা: onPause-এ কেন

কোডে কোথায় `VideoFragment.kt › onPause()` লাইন 67–82 `VideoFragment.kt › onStop()` লাইন 84–89 `VideoFragment.kt › onStart()` লাইন 61–65

স্ক্রিন যখনই চলে যেতে শুরু করে (ফোন ঘোরানো, হোম বাটন চাপা, বা অন্য ড্রয়ার আইটেম), `VideoFragment`-এর onPause দুটি কাজ করে: ভিডিও প্রস্তুত থাকলে বর্তমান অবস্থান আর "আবার চালাতে হবে কি না" ViewModel-এ রাখে, তারপর ভিডিও থামায়।

- **আবার চালানোর শর্ত:** ভিডিও চলছিল **এবং** `isChangingConfigurations` সত্য লাইন 78। অর্থাৎ শুধু ফোন ঘোরালে নতুন স্ক্রিনে ভিডিও নিজে থেকেই আবার চলতে থাকে; কিন্তু ব্যবহারকারী সত্যিই স্ক্রিন ছেড়ে গেলে ফিরে এসে দেখবেন ভিডিও থেমে একই জায়গায় অপেক্ষা করছে।
- **onStop নয় কেন:** সাম্প্রতিক Android-এ onStop চলার আগেই VideoView-এর পৃষ্ঠ (surface), আর তার সঙ্গে MediaPlayer, ছেড়ে দেওয়া হয়। তখন অবস্থান পড়লে ০ পাওয়া যেত। onPause-এর সময় প্লেয়ার তখনো চালু থাকে।
- **onStop-এ যা হয়:** "প্রস্তুত" চিহ্নটি মুছে ফেলা হয়। অ্যাপ পেছনে গেলে VideoView তার প্লেয়ার ছেড়ে দেয়; ফিরে এলে ভিডিও আবার প্রস্তুত হয় আর onPrepared আবার আসে। তাই onStart-এ ভিডিও প্রস্তুত না থাকলে লোডিং দেখায়।

১০.৪ VideoViewModel: অবস্থান কোথায় থাকে

কোডে কোথায় `VideoViewModel.kt › VideoViewModel` লাইন 14–37 `VideoViewModel.kt › savePlayback()` লাইন 25–28

অবস্থান VideoView নিজে মনে রাখতে পারে না: ফোন ঘোরালে ভিউটি ভেঙে যায়, আর অ্যাপ পেছনে গেলে তার প্লেয়ার মুক্ত হয়ে যায়। তাই অবস্থান (মিলিসেকেন্ডে) আর "চলছিল কি না" থাকে `VideoViewModel`-এর SavedStateHandle-এ, যা ফোন ঘোরানো আর প্রসেস বন্ধ হওয়া, দুটোতেই টিকে থাকে (৬.২)।

অডিওর মতো এখানে প্লেয়ারটি ViewModel-এ রাখা হয়নি কেন? কারণ VideoView একটি ভিউ, আর তার প্লেয়ার স্ক্রিনের পৃষ্ঠের সঙ্গে বাঁধা; ভিউ চলে গেলে প্লেয়ারও যায়। তাই এখানে পুরো প্লেয়ার নয়, শুধু "কোথায় ছিলাম" আর "চলছিল কি না", এই দুটি তথ্য রাখা হয়, আর নতুন প্লেয়ার প্রস্তুত হলে সেখান থেকে শুরু হয়।

১০.৫ কন্ট্রোল ও ড্রয়ারের সমন্বয়

কোডে কোথায় `VideoFragment.kt › hideControlsWhileDrawerMoves` লাইন 147–151
`VideoFragment.kt › showControlsBriefly()` লাইন 126–128
`VideoFragment.kt › onDestroyView()` লাইন 91–100

- **ড্রয়ার নড়লে কন্ট্রোল লুকানো:** MediaController Activity-র ওপরে নিজের আলাদা জানালায় আঁকা হয়। ড্রয়ার খুললে ড্রয়ারটি কন্ট্রোলের নিচ দিয়ে চলে যেত, আর কন্ট্রোল ড্রয়ারের ওপরে ভেসে থাকত। তাই একটি DrawerListener ড্রয়ারের অবস্থা স্থির (idle) থেকে বদলালেই, অর্থাৎ টানা শুরু হলেই, কন্ট্রোল লুকিয়ে দেয়। এটি অবজারভার প্যাটার্ন: ড্রয়ার তার অবস্থার পরিবর্তন জানায়, আর Fragment শুনে সাড়া দেয়।
- **সবসময় দেখানো হয় না কেন:** কন্ট্রোলটি একটি আলাদা জানালা, যা কীবোর্ডের মনোযোগ (focus) নিয়ে নেয়; এটি খোলা থাকলে টুলবার বা হ্যামবার্গার আইকন কীবোর্ডের মনোযোগ পায় না। তাই কন্ট্রোল সবসময় না রেখে ডিফল্ট সময়সীমা (কয়েক সেকেন্ড) রাখা হয়েছে। TalkBack চালু থাকলে MediaController নিজেই সময়সীমা উপেক্ষা করে কন্ট্রোল দেখিয়ে রাখে।
- **গুছিয়ে বন্ধ করা:** ভিউ ভাঙার সময় ড্রয়ারের লিসেনার সরানো হয় (নইলে Activity-র ড্রয়ার ভেঙে যাওয়া Fragment-কে ধরে রাখত), কন্ট্রোল লুকানো হয় (যাতে তার জানালা আটকে না থাকে), আর `stopPlayback()` দিয়ে MediaPlayer মুক্ত করা হয়।

১১ ফিচার D: Audio

> **সহজ কথায়**
> গান বাজায় MediaPlayer, আর এটি রাখা থাকে ViewModel-এ, তাই ফোন ঘোরালেও গান থামে না। MediaPlayer-এর কয়েকটি নির্দিষ্ট অবস্থা আছে, আর প্রতিটি কাজ শুধু ঠিক অবস্থায় করা যায়। স্ক্রিনের সব অবস্থা এক জায়গায় (AudioUiState) রাখা, আর প্রতি ২০০ মিলিসেকেন্ডে গানের অবস্থান হালনাগাদ হয়। অন্য স্ক্রিনে গেলে গান থেমে যায়।

সংশ্লিষ্ট ফাইল (`audio` প্যাকেজ): `AudioPlayerViewModel` · `AudioFragment` · `util/TimeFormat.kt` · `res/layout/fragment_audio.xml` (ও `layout-land`) · `res/raw/sample_audio.mp3`

অডিও স্ক্রিনে MediaPlayer সরাসরি ব্যবহার করা হয়েছে, কোনো তৈরি কন্ট্রোল ছাড়া। তাই এখানে প্লেয়ারের অবস্থা, স্ক্রিনের স্টেট আর লাইফসাইকেল, তিনটিই নিজেদের হাতে সামলাতে হয়। মেসেজ কিউয়ের ব্যবহারও এখানে সবচেয়ে স্পষ্ট (**৩.৮**)।

১১.১ MediaPlayer একটি স্টেট মেশিন

কোডে কোথায় `AudioPlayerViewModel.kt` › **অবস্থার পথ (মন্তব্য)** লাইন 35–43

MediaPlayer সবসময় নির্দিষ্ট কয়েকটি অবস্থার একটিতে থাকে, আর প্রতিটি মেথড শুধু নির্দিষ্ট অবস্থায় ডাকা যায়; একে স্টেট মেশিন (state machine) বলে। ভুল অবস্থায় ডাকলে IllegalStateException বা ভুল হয়। এই অ্যাপ যে পথে চলে:

বর্তমান অবস্থা	কাজ বা ঘটনা	পরের অবস্থা
Idle (নতুন তৈরি)	`setDataSource()`	Initialized
Initialized	`prepareAsync()`	Preparing
Preparing	onPrepared (প্লেয়ার জানায়)	Prepared
Prepared, Paused বা PlaybackCompleted	`start()`	Started
Started	`pause()`	Paused
Started বা Paused	`stop()`	Stopped
Stopped	`prepareAsync()`	Preparing, তারপর Prepared
Started	গান শেষ	PlaybackCompleted
যেকোনো অবস্থা	`release()`	End

সবচেয়ে গুরুত্বপূর্ণ নিয়ম: Stopped অবস্থা থেকে সরাসরি চালু (start) করা যায় না, আগে আবার প্রস্তুত (prepare) করতে হয়। তাই অ্যাপের `stop()` নিজেই শেষে `prepareAsync()` ডাকে।

১১.২ প্লেয়ার কেন ViewModel-এ

কোডে কোথায় `AudioPlayerViewModel.kt › player লাইন 50`
`AudioPlayerViewModel.kt › init লাইন 53–70` `AudioPlayerViewModel.kt › onCleared() লাইন 141–144`

MediaPlayer তৈরি হয় `AudioPlayerViewModel`-এর ভেতরে, Fragment-এ নয়। ফোন ঘোরালে Fragment আর তার ভিউ নতুন করে তৈরি হয়, কিন্তু ViewModel আর তার প্লেয়ার একই থাকে; তাই গান থামে না, এক মুহূর্তের বিরতি ছাড়াই চলতে থাকে। স্ক্রিন স্থায়ীভাবে চলে গেলে `onCleared()`-এ অগ্রগতির কাজ বাতিল করে `release()` ডাকা হয়, যা ডিকোডার আর অডিওর সম্পদ মুক্ত করে।

ViewModel-টি AndroidViewModel, কারণ অ্যাপের ভেতরে রাখা (raw) ফাইল খুলতে অ্যাপের context লাগে; Activity-র context রাখলে ফোন ঘোরানোর পর পুরোনো Activity মেমরিতে আটকে থাকত। ViewModel তৈরির সময় প্লেয়ার প্রস্তুত করার ধাপগুলো:

১. AudioAttributes (`USAGE_MEDIA`, `CONTENT_TYPE_MUSIC`) দিয়ে সিস্টেমকে জানানো যে এটি গান বা মিডিয়া, ফলে মিডিয়ার ভলিউম আর শব্দের গন্তব্য (স্পিকার, হেডফোন) ঠিক থাকে। এটি প্রস্তুত করার আগেই দিতে হয় লাইন 55–60 ।

২. `res/raw/sample_audio.mp3` ফাইলটি খুলে `setDataSource()`, অর্থাৎ Idle থেকে Initialized লাইন 61–63 ।

৩. onPrepared, onCompletion আর onError লিসেনার যুক্ত করা লাইন 64–66 ।

৪. `prepareAsync()`, অর্থাৎ Initialized থেকে Preparing লাইন 69 ।

`prepareAsync()` সঙ্গে সঙ্গে ফিরে আসে; ফাইল প্রস্তুত হলে প্লেয়ার মেইন থ্রেডে onPrepared পাঠায়। এটি অনুরোধ-উত্তর ধরন: অনুরোধ এখন, উত্তর পরে একটি বার্তা হিসেবে। সাধারণ `prepare()` ব্যবহার করলে প্রস্তুত হওয়া পর্যন্ত মেইন থ্রেড আটকে থাকত।

১১.৩ স্টেট ওপরে ওঠে, নির্দেশ নিচে নামে

কোডে কোথায় `AudioPlayerViewModel.kt › AudioUiState লাইন 20–26`
`AudioPlayerViewModel.kt › togglePlayPause(), play(), pause() লাইন 72–88`
`AudioFragment.kt › onViewCreated() লাইন 30–44` `AudioFragment.kt › render() লাইন 76–103`

পুরো স্ক্রিনের অবস্থা একটি অপরিবর্তনীয় (immutable) data class-এ রাখা হয়: `AudioUiState`, যার পাঁচটি ফিল্ড বলে দেয়: প্রস্তুত কি না, চলছে কি না, বর্তমান অবস্থান, মোট সময় আর কোনো ভুল আছে কি না। এটি একটি StateFlow-এ প্রকাশ হয়, আর ViewModel-ই এর একমাত্র লেখক।

- **নির্দেশ নিচে (স্ক্রিন থেকে ViewModel-এ):** Fragment ব্যবহারকারীর কাজকে ViewModel-এর মেথডে রূপ দেয়: `togglePlayPause()`, `stop()`, `seekTo()`। Fragment নিজে কখনো প্লেয়ার ছোঁয় না।
- **স্টেট ওপরে (ViewModel থেকে স্ক্রিনে):** Fragment `repeatOnLifecycle(STARTED)`-এর ভেতরে StateFlow থেকে মান সংগ্রহ করে, আর প্রতিটি নতুন স্টেটে `render()` দিয়ে পুরো স্ক্রিন আঁকে। এই StateFlow অবজারভার প্যাটার্নে শেষ মান ধরে রাখা একটি টপিকের মতো কাজ করে (**৪.৭**)।
- **আগে শর্ত যাচাই:** প্রতিটি নির্দেশ আগে স্টেট দেখে নেয়: চালানো শুধু প্রস্তুত আর না-চলা অবস্থায়, থামানো শুধু চলা অবস্থায়, Stop আর সামনে-পেছনে যাওয়া শুধু প্রস্তুত অবস্থায়। এতে স্টেট মেশিনের ভুল

অবস্থায় কোনো মেথড ডাকা হয় না।

- **অপরিবর্তনীয় হালনাগাদ:** প্রতিবার পুরোনো স্টেট কপি করে শুধু দরকারি ফিল্ড বদলানো হয়; StateFlow নতুন মানটি সব সংগ্রাহকের কাছে পৌঁছে দেয়।

এই ধরনকে একমুখী ডেটা প্রবাহ (unidirectional data flow) বলে: সত্যের উৎস একটিই (ViewModel-এর স্টেট), আর স্ক্রিন শুধু সেই স্টেটের প্রতিচ্ছবি। `render()` স্টেট থেকেই সব ঠিক করে: Play/Pause বাটনের লেখা ও আইকন; কোন বাটন চাপা যাবে (প্রস্তুত না হলে Play চাপা যায় না, আর Stop শুধু তখনই চাপা যায় যখন গান চলছে বা অবস্থান শূন্যের বেশি); স্লাইডার; সময়; আর অবস্থার লেখা: Preparing the track..., Ready to play, Playing, Paused, বা ভুলের বার্তা।

১১.৪ অগ্রগতি: কোরুটিনের একটি লুপ

কোডে কোথায় `AudioPlayerViewModel.kt › startProgressUpdates()` লাইন 125–133
`AudioPlayerViewModel.kt › stopProgressUpdates()` লাইন 135–138
`AudioPlayerViewModel.kt › PROGRESS_INTERVAL_MS` লাইন 148

গান চলার সময় স্লাইডার আর সময় এগোতে হবে, কিন্তু MediaPlayer নিজে থেকে অবস্থান জানায় না; জিজ্ঞেস করে নিতে হয়। তাই গান চালু হলে ViewModel-এর কোরুটিন-পরিসরে একটি কোরুটিন চালু হয়: যতক্ষণ সক্রিয়, ততক্ষণ প্লেয়ারের বর্তমান অবস্থান স্টেটে বসায়, তারপর ২০০ মিলিসেকেন্ড অপেক্ষা করে (`delay(200)`); অর্থাৎ সেকেন্ডে প্রায় পাঁচবার।

এই অপেক্ষা থ্রেড আটকায় না। মেইন থ্রেডে `delay()` আসলে একটি দেরিতে চলার বার্তা, যা ২০০ মিলিসেকেন্ড পরে সারিতে চলার উপযোগী হয় (**৩.৫**); মাঝের সময়ে মেইন থ্রেড ছোঁয়া, আঁকা ইত্যাদি অন্য কাজ করে। বার্তার ভাষায় এটি একটি নিয়মিত প্রেরক (periodic producer)। Pause, Stop, গান শেষ বা কোনো ভুল হলে কাজটি বাতিল হয়, আর `onCleared()`-এও।

১১.৫ স্লাইডার ও সময়ের লেখা

কোডে কোথায় `AudioFragment.kt › setUpSeekSlider()` লাইন 55–74
`AudioFragment.kt › render() (স্লাইডারের অংশ)` লাইন 84–92 `TimeFormat.kt › formatMmSs()` লাইন 14–19

- **স্লাইডারের সীমা:** Material Slider-এর সর্বোচ্চ মান হলো মোট সময় (মিলিসেকেন্ডে), তবে কমপক্ষে ১; কারণ স্লাইডারে সর্বোচ্চ মান অবশ্যই সর্বনিম্ন মানের (০) চেয়ে বড় হতে হয়, নইলে স্লাইডার exception ছোড়ে আর অ্যাপ ক্র্যাশ করে। বর্তমান মানও সবসময় সীমার ভেতরে আটকে রাখা হয়।
- **আঙুলের সঙ্গে টানাটানি না করা:** স্লাইডার টানার সময় প্রতি ২০০ মিলিসেকেন্ডের হালনাগাদ স্লাইডারকে পুরোনো অবস্থানে টেনে নিত। তাই `isUserSeeking` ফ্ল্যাগ: আঙুল রাখার মুহূর্ত থেকে ছাড়া পর্যন্ত স্টেট থেকে স্লাইডারের মান বসানো হয় না, শুধু সময়ের লেখা আঙুলের মান দেখায়। আঙুল ছাড়লে তবেই গান সেই জায়গায় যায়।
- **কীবোর্ড ও অ্যাক্সেসিবিলিটি:** এসব উপায়ে মান বদলালে কোনো ছোঁয়ার পর্ব থাকে না, তাই সঙ্গে সঙ্গে গান সেই জায়গায় যায়। বদলটি ব্যবহারকারীর, নাকি কোডের, তা লিসেনারের `fromUser` থেকে বোঝা যায়।
- **মিনিট:সেকেন্ড:** `formatMmSs()` মিলিসেকেন্ডকে মিনিট:সেকেন্ডে রূপ দেয়; যেমন ৬৫,০০০ মিলিসেকেন্ড হয় "01:05"। ঋণাত্মক মান ০ ধরা হয়; এক ঘণ্টার বেশি হলে মিনিট গুনতেই থাকে

("75:00")। `Locale.ROOT` ব্যবহার করায় ফোনের যেকোনো ভাষার সেটিংয়ে একই অঙ্ক দেখায়। স্লাইডার টানার সময় ভেসে ওঠা লেবেলও একই ফাংশন ব্যবহার করে।

১১.৬ Stop, গান শেষ ও ভুল

কোডে কোথায় `AudioPlayerViewModel.kt` › **`stop()`** লাইন 91–97
`AudioPlayerViewModel.kt` › **`onCompleted()`** লাইন 111–115
`AudioPlayerViewModel.kt` › **`onError()`** লাইন 117–122

- **Stop:** প্লেয়ার থামে (Stopped অবস্থা), অগ্রগতির লুপ বন্ধ হয়, স্টেটে "প্রস্তুত নয়, চলছে না, অবস্থান ০" বসে, তারপর `prepareAsync()`। প্রস্তুত হলে onPrepared আবার "প্রস্তুত" করে দেয়। ফলে Stop মানে "থামো আর শুরুতে ফিরে যাও"।
- **গান শেষ:** অগ্রগতির লুপ বন্ধ, অবস্থান ০-তে যাওয়া, "চলছে না" আর অবস্থান ০; আবার Play চাপলেই শুরু থেকে বাজে।
- **ভুল:** লগ, অগ্রগতির লুপ বন্ধ, স্টেটে ভুল চিহ্নিত, আর true ফেরত, যাতে এর পরে onCompletion-ও না ডাকা হয় (false ফেরত দিলে MediaPlayer onCompletion-ও ডাকত)।

১১.৭ অডিও আর ভিডিও কখনো একসঙ্গে বাজে না

কোডে কোথায় `AudioFragment.kt` › **`onStop()`** লাইন 46–53
`AudioPlayerViewModel.kt` › **`pause()`** লাইন 83–88

সমস্যা: অন্য ড্রয়ার আইটেমে যাওয়ার সময় Navigation অডিও স্ক্রিনের ViewModel বাঁচিয়ে রাখে (**৭.২**)। প্লেয়ার ViewModel-এ, তাই কিছু না করলে Video স্ক্রিনে গেলেও পেছনে গান বাজতে থাকত আর ভিডিওর শব্দের সঙ্গে মিশে যেত।

সমাধান: `AudioFragment`-এর onStop-এ, যদি `isChangingConfigurations` সত্য না হয় (অর্থাৎ এটি ফোন ঘোরানো নয়, সত্যিই চলে যাওয়া: অন্য ড্রয়ার আইটেম, হোম বাটন ইত্যাদি), তবে ViewModel-কে গান থামাতে বলা হয়।

ফল: ফোন ঘোরালে গান চলতেই থাকে; স্ক্রিন ছাড়লে থেমে যায়; ফিরে এসে থামা জায়গা থেকেই Play করা যায়। আর ভিডিও স্ক্রিনও নিজের onPause-এ থামে। তাই দুটি একসঙ্গে বাজার কোনো সুযোগ থাকে না।

১২ সব স্ক্রিনে প্রযোজ্য বিষয়

> **সহজ কথায়**
> ফোন ঘোরালে প্রতিটি স্ক্রিন নতুন করে তৈরি হয়, তবু কোনো তথ্য হারায় না; নিচের তালিকায় আছে কোন স্ক্রিন কী ধরে রাখে আর কীভাবে। এছাড়া অ্যাপটি হালকা ও গাঢ় দুই থিমে চলে, সব লেখা রিসোর্স ফাইলে রাখা, আর যাঁরা চোখে কম দেখেন তাঁদের সহায়ক সেবার (TalkBack) কথাও মাথায় রাখা হয়েছে।

কিছু আচরণ কোনো একটি ফিচারের নয়, পুরো অ্যাপের। এগুলোর বেশিরভাগ আগের অধ্যায়গুলোতে টুকরো টুকরো এসেছে; এখানে একসঙ্গে দেখা যাক।

১২.১ ফোন ঘোরালে: কোন স্ক্রিন কী ধরে রাখে

অ্যাপ `android:configChanges` ব্যবহার করে না, তাই ফোন ঘোরালে সিস্টেম প্রতিটি স্ক্রিন নতুন করে তৈরি করে আর আড়াআড়ি অবস্থার উপযোগী রিসোর্স লোড করে (ব্যাটারির অংশ আর অডিও স্ক্রিনের আলাদা আড়াআড়ি লেআউট আছে)। তবু কোনো তথ্য হারায় না (**৬.২**):

স্ক্রিন	যা টিকে থাকে	যেভাবে ও কোডে কোথায়
Broadcast স্ক্রিন ১	স্পিনারের বাছাই	SavedStateHandle `BroadcastViewModel.kt` › **`selectedMode`** লাইন 16–17
Activity 2 (নিজের বার্তা)	টাইপ করা লেখা	টেক্সট ফিল্ডের নিজস্ব সংরক্ষিত অবস্থা (id থাকায়); আলাদা কোড লাগেনি
Activity 2 (ব্যাটারি)	সর্বশেষ ব্যাটারির তথ্য	ViewModel-এর LiveData আর স্টিকি ব্রডকাস্ট `BatteryViewModel.kt` › **`BatteryViewModel`** লাইন 14–17
Activity 3 (নিজের বার্তা)	পাওয়া বার্তা; “পাঠানো হয়েছে” ফ্ল্যাগ	SavedStateHandle `CustomReceiverViewModel.kt` › **`CustomReceiverViewModel`** লাইন 16–38
Image Scale	নামানো ছবি; জুম আর দেখা অংশ	StateFlow আর ভিউয়ের সংরক্ষিত অবস্থা `ImageScaleViewModel.kt` › **`uiState`** লাইন 35–36 `ZoomableImageView.kt` › **`onSave/onRestoreInstanceState()`** লাইন 268–291
Video	অবস্থান; চলছিল কি না	SavedStateHandle `VideoViewModel.kt` › **`savePlayback()`** লাইন 25–28
Audio	পুরো প্লেয়ার; গান চলতেই থাকে	ViewModel-এ প্লেয়ার `AudioPlayerViewModel.kt` › **`player`** লাইন 50
ড্রয়ার	খোলা না বন্ধ	DrawerLayout-এর সংরক্ষিত অবস্থা; ব্যাক কলব্যাক মিলিয়ে নেওয়া `MainActivity.kt` › **`onRestoreInstanceState()`** লাইন 52–57

১২.২ থিম, রিসোর্স ও অ্যাক্সেসিবিলিটি

- **হালকা ও গাঢ় থিম:** অ্যাপ Material 3-এর DayNight থিম ব্যবহার করে, তাই ফোনে ডার্ক মোড চালু করলে সব স্ক্রিন নিজে থেকেই গাঢ় রঙে চলে যায়।
- **সব লেখা ও মাপ রিসোর্সে:** লেখা `strings.xml`-এ, মাপ `dimens.xml`-এ (আড়াআড়ি অবস্থার জন্য `values-land`); লেআউটে সরাসরি লেখা বা মাপ বসানো নেই। ফলে অনুবাদ করা বা ভিন্ন মাপের স্ক্রিনে মানিয়ে নেওয়া সহজ। সংখ্যার সঙ্গে সঠিক বচনের জন্য plurals রিসোর্স ব্যবহার করা হয়।
- **অ্যাক্সেসিবিলিটি:** প্রতিটি ছোঁয়ার জায়গা কমপক্ষে ৪৮dp; ছবি, ভিডিও, ব্যাটারির রিং আর স্লাইডারের বর্ণনা (content description) আছে; জুম ভিউতে একক ট্যাপে `performClick()` লাইন 182–185; TalkBack চালু থাকলে MediaController কন্ট্রোল দেখিয়ে রাখে।
- **এজ-টু-এজ:** প্রতিটি স্ক্রিন খাড়া ও আড়াআড়ি দুই অবস্থাতেই সিস্টেম বার আর ক্যামেরার খাঁজ এড়িয়ে চলে (**৭.৫**)।

১৩ দ্রুত পুনরালোচনা

সহজ কথায়

এই অধ্যায় শুধু মনে রাখার তালিকা: প্রথমে দুই বার্তা ব্যবস্থা, তারপর প্রতিটি ফিচারের মূল কথা। পরীক্ষা বা ভাইভার আগে এটুকু চোখ বুলিয়ে নিলেই মূল বিষয়গুলো মনে পড়ে যাবে।

১৩.১ মেসেজ কিউ ও পাবলিশ/সাবস্ক্রাইবের মূল কথা

মেসেজ কিউ

- প্রেরক বার্তা সারিতে রাখে, গ্রাহক একটি একটি করে তোলে; সাধারণত FIFO।
- প্রতিটি বার্তা ঠিক একজন গ্রাহক সামলায় (এক-থেকে-এক); গ্রাহক ব্যস্ত থাকলে বার্তা অপেক্ষা করে, হারায় না।
- উদাহরণ: রেস্তোরাঁর অর্ডার স্লিপ, ব্যাংকের টোকেন, অনলাইন দোকানের ইমেইলের সারি।
- Android-এ: মেইন থ্রেডের Looper (গ্রাহক) আর MessageQueue; Handler বার্তা রাখে। গ্রাহক একজন, তাই কোনো কলব্যাক ধীর হলে ANR।
- এই অ্যাপে: ছোঁয়ার ঘটনা, লাইফসাইকেল কলব্যাক, `onReceive()`, onPrepared, `delay(200)`-এর লুপ, অ্যানিমেশনের ফ্রেম।

পাবলিশ/সাবস্ক্রাইব

- প্রকাশক টপিকে বার্তা দেয়; ব্রোকার টপিকের সব সাবস্ক্রাইবারকে একটি করে কপি দেয় (ফ্যান-আউট); প্রকাশক সাবস্ক্রাইবারদের চেনে না।
- দেরিতে আসা সাবস্ক্রাইবার: ক্ষণস্থায়ী হলে কিছু পায় না (রেডিও), শেষ বার্তা রাখা হলে শেষটি পায় (নোটিশ বোর্ড), স্থায়ী হলে সব পায় (ইমেইলের ইনবক্স)।
- Android-এ: Intent = বার্তা, অ্যাকশন = টপিক, IntentFilter = ফিল্টার, BroadcastReceiver = সাবস্ক্রাইবার, ActivityManagerService = ব্রোকার।
- এই অ্যাপে: নিজের ব্রডকাস্ট (ক্ষণস্থায়ী, তাই আগে রেজিস্টার, পরে পাঠানো); ব্যাটারি (স্টিকি, অর্থাৎ শেষ বার্তা রাখা)।
- ভেতরের পাবলিশ/সাবস্ক্রাইব: LiveData ও StateFlow (অবজারভার প্যাটার্ন), যা শেষ মান ধরে রাখে।

১৩.২ প্রতিটি ফিচারের মূল কথা

ড্রয়ার ও নেভিগেশন

- একটি `MainActivity`; চারটি স্ক্রিন চারটি Fragment, যা NavHostFragment-এ দেখানো হয়।
- মেনু আইটেমের id = গন্তব্যের id; এই মিল দিয়েই NavigationUI স্ক্রিন খোলে আর আইটেম চিহ্নিত করে।
- মেনু থেকে AppBarConfiguration, তাই চারটিই টপ-লেভেল আর সবখানে হ্যামবার্গার আইকন।
- ব্যাক কলব্যাক শুধু ড্রয়ার খোলা থাকলে চালু; তখন ব্যাক বাটন আগে ড্রয়ার বন্ধ করে।

নিজের ব্রডকাস্ট (Custom)

- লেখা Activity 2 থেকে Activity 3-এ যায় explicit Intent-এ (এক-থেকে-এক, নিশ্চিত পৌঁছানো)।
- Activity 3: onStart-এ রেজিস্টার (`RECEIVER_NOT_EXPORTED`), তারপর `sendBroadcast()` (`setPackage()`-সহ); `onReceive()` মেইন থ্রেডে চলে, ViewModel হয়ে স্ক্রিনে; onStop-এ আনরেজিস্টার।
- Activity 2 থেকে ব্রডকাস্ট করলে বার্তা হারাত, কারণ সাধারণ ব্রডকাস্ট ক্ষণস্থায়ী আর তখন রিসিভার ছিলই না।
- SavedStateHandle-এর ফ্ল্যাগ দিয়ে ফোন ঘোরালে দুবার পাঠানো ঠেকানো হয়; পাওয়া বার্তাও সেখানেই থাকে।

ব্যাটারির ব্রডকাস্ট

- `ACTION_BATTERY_CHANGED` স্টিকি (সর্বশেষ মান সঙ্গে সঙ্গে পাওয়া যায়) আর সুরক্ষিত (শুধু সিস্টেম পাঠায়), আর শুধু কোডে রেজিস্টার করা রিসিভার এটি পায়।
- LiveData-র onActive-এ রেজিস্টার, onInactive-এ আনরেজিস্টার, তাই শুধু স্ক্রিন দেখা গেলেই শোনা হয়।
- শতাংশ = মাত্রা × ১০০ ÷ সর্বোচ্চ মাত্রা; সর্বোচ্চ মাত্রা শূন্য বা মাত্রা না থাকলে "—"।

Image Scale

- Coil 3 দিয়ে মূল মাপে (`Size.ORIGINAL`) ছবি; ViewModel-এর StateFlow-এ Loading, Success বা Error; Retry।
- scaleType MATRIX; জুম = বর্তমান স্কেল ÷ ফিটের স্কেল, সীমা ১ থেকে ৫ গুণ।
- ScaleGestureDetector-এর ফোকাল পয়েন্ট ঘিরে স্কেল, তাই আঙুলের নিচের বিন্দু স্থির থাকে।
- এক আঙুলে সরানো, কিনারায় ফাঁক না রাখা, ডাবল-ট্যাপে ২.৫ গুণ বা শুরুর অবস্থা (২৫০ মিলিসেকেন্ডের অ্যানিমেশন)।
- জুম আর কেন্দ্রবিন্দু ভগ্নাংশে সংরক্ষণ, তাই ফোন ঘোরানোর পরও একই দৃশ্য।

Video

- VideoView (ভেতরে MediaPlayer আর আঁকার পৃষ্ঠ) আর MediaController; ফাইল `res/raw`-এ।
- onPause-এ অবস্থান রাখা হয় (onStop-এ দেরি হয়ে যায়); আবার চলে শুধু তখনই, যখন ভিডিও চলছিল আর কারণটি ফোন ঘোরানো।
- ভুল হলে নিজস্ব বার্তা, সিস্টেমের ডায়ালগ নয়; ড্রয়ার নড়লে কন্ট্রোল লুকানো।

Audio

- MediaPlayer একটি স্টেট মেশিন; Stop-এর পর আবার `prepareAsync()` লাগে।
- প্লেয়ার ViewModel-এ, তাই ফোন ঘোরালেও গান চলে; `onCleared()`-এ মুক্ত।
- একটি `AudioUiState` StateFlow: স্টেট ওপরে ওঠে, নির্দেশ নিচে নামে।
- ২০০ মিলিসেকেন্ড পরপর কোরুটিনে অগ্রগতি; স্লাইডার টানার সময় কোড তার মান বদলায় না; onStop-এ থামানো (ফোন ঘোরানো ছাড়া)।

১৪ ভাইভা প্রশ্নোত্তর

> **সহজ কথায়**
> ভাইভায় যে প্রশ্নগুলো আসতে পারে, সেগুলোর ছোট ও সহজ উত্তর। প্রতিটি প্রশ্নের নিচে ইংরেজিতে প্রশ্নটি দেওয়া আছে, কারণ ভাইভা ইংরেজিতে হতে পারে। উত্তর মুখস্থ না করে কারণটা বুঝে নেওয়া ভালো; প্রতিটি উত্তরের শেষে বিস্তারিত ব্যাখ্যার অংশের নম্বর দেওয়া আছে।

১৪.১ অ্যাপের কাঠামো ও নেভিগেশন

১. অ্যাপে কয়টি Activity আর কয়টি Fragment আছে?

How many Activities and Fragments does the app have?

তিনটি Activity: `MainActivity` (ড্রয়ারসহ মূল Activity), `BroadcastInputActivity` (Activity 2) আর `CustomReceiverActivity` (Activity 3)। ড্রয়ারের চারটি স্ক্রিন চারটি Fragment: `BroadcastFragment`, `ImageScaleFragment`, `VideoFragment` আর `AudioFragment`। (৭.১)

২. Activity আর Fragment-এর পার্থক্য কী?

What is the difference between an Activity and a Fragment?

Activity হলো একটি পূর্ণ স্ক্রিন, যাকে সিস্টেম সরাসরি খোলে আর যার নিজস্ব জানালা আছে। Fragment হলো Activity-র ভেতরে বসানো স্ক্রিন-অংশ, যার নিজস্ব লাইফসাইকেল আর ভিউ আছে, কিন্তু সে একা চলতে পারে না। এই অ্যাপে একটি Activity-র ভেতরে চারটি Fragment বদলে বদলে দেখানো হয়, ফলে টুলবার আর ড্রয়ার একবারই তৈরি হয়। (৭.১)

৩. ড্রয়ারের আইটেম চাপলে সঠিক স্ক্রিন খোলে কীভাবে?

How does tapping a drawer item open the right screen?

`drawer_menu.xml`-এর প্রতিটি আইটেমের id আর `nav_graph.xml`-এর প্রতিটি গন্তব্যের id এক। `setupWithNavController()` দিয়ে NavigationView-কে NavController-এর সঙ্গে যুক্ত করায় NavigationUI এই id মিলিয়ে নিজেই সেই স্ক্রিনে যায়, আর চিহ্নিত আইটেম ও টুলবারের শিরোনাম বদলায়। (৭.২)

৪. প্রতিটি স্ক্রিনে হ্যামবার্গার আইকন দেখা যায় কেন, পেছনে যাওয়ার তীর কেন নয়?

Why does every screen show the hamburger icon instead of a back arrow?

`AppBarConfiguration` ড্রয়ার মেনু থেকে তৈরি, তাই চারটি গন্তব্যই টপ-লেভেল। টপ-লেভেল গন্তব্যে টুলবার পেছনে যাওয়ার তীর না দেখিয়ে ড্রয়ার খোলার হ্যামবার্গার আইকন দেখায়। (৭.২)

৫. ড্রয়ার খোলা থাকলে ব্যাক বাটন চাপলে কী হয়, আর কীভাবে?

What happens when Back is pressed with the drawer open, and how is it done?

অ্যাপ বন্ধ না হয়ে শুধু ড্রয়ার বন্ধ হয়। OnBackPressedDispatcher-এ একটি OnBackPressedCallback যোগ করা আছে, যা শুধু ড্রয়ার খোলা থাকলে চালু থাকে; একটি DrawerListener ড্রয়ার খুললে তাকে চালু আর বন্ধ হলে নিষ্ক্রিয় করে। নতুন Android-এ পুরোনো `onBackPressed()` আর ডাকা হয় না, তাই এটিই সঠিক উপায়। (৭.৪)

৬. ViewBinding কী, আর Fragment-এ binding null করা হয় কেন?

What is ViewBinding, and why is the binding set to null in a Fragment?

ViewBinding প্রতিটি লেআউট থেকে একটি ক্লাস বানায়, যেখানে id থাকা প্রতিটি ভিউয়ের একটি নিরাপদ ফিল্ড থাকে; `findViewById()` লাগে না। Fragment তার ভিউয়ের চেয়ে বেশি দিন বাঁচতে পারে, তাই onDestroyView-এ binding null না করলে ভেঙে যাওয়া পুরোনো ভিউ মেমরিতে আটকে থাকত। (৭.৩)

১৪.২ মেসেজ কিউ ও পাবলিশ/সাবস্ক্রাইব

৭. মেসেজ কিউ কী? একটি উদাহরণসহ।

What is a message queue? Give an example.

একটি সারি, যেখানে প্রেরক (producer) বার্তা রাখে আর গ্রাহক (consumer) একটি একটি করে তুলে কাজ করে, সাধারণত FIFO ক্রমে। প্রতিটি বার্তা একজন গ্রাহকই সামলায়, আর গ্রাহক ব্যস্ত থাকলে বার্তা অপেক্ষা করে, হারায় না। উদাহরণ: রেস্তোরাঁয় ওয়েটার অর্ডারের স্লিপ তারে ঝোলান, আর রাঁধুনি একে একে রান্না করেন। (৩.২)

৮. পাবলিশ/সাবস্ক্রাইব কী? একটি উদাহরণসহ।

What is publish/subscribe? Give an example.

প্রকাশক (publisher) একটি টপিকে বার্তা প্রকাশ করে, আর ব্রোকার সেই টপিকের সব সাবস্ক্রাইবারকে একটি করে কপি পৌঁছে দেয়; প্রকাশক জানে না কারা শুনছে। উদাহরণ: YouTube চ্যানেলে নতুন ভিডিও দিলে সব সাবস্ক্রাইবার নোটিফিকেশন পায়। (৪.১)

৯. মেসেজ কিউ আর পাবলিশ/সাবস্ক্রাইবের মূল পার্থক্য কী?

What is the main difference between a message queue and publish/subscribe?

মেসেজ কিউতে একটি বার্তা যায় ঠিক একজন গ্রাহকের কাছে, আর কেউ না থাকলে বার্তা সারিতে অপেক্ষা করে। পাবলিশ/সাবস্ক্রাইবে একটি বার্তার কপি যায় সব সাবস্ক্রাইবারের কাছে, আর সাধারণত তখন উপস্থিত না থাকলে বার্তাটি আর পাওয়া যায় না। কাজ ভাগ করতে কিউ, খবর ছড়াতে পাবলিশ/সাবস্ক্রাইব। (৫.১)

১০. Android-এর ব্রডকাস্ট কীভাবে পাবলিশ/সাবস্ক্রাইব?

How is an Android broadcast a publish/subscribe system?

Intent হলো বার্তা, তার অ্যাকশন টপিক, BroadcastReceiver সাবস্ক্রাইবার, IntentFilter সাবস্ক্রিপশনের ফিল্টার, `registerReceiver()` সাবস্ক্রাইব, `sendBroadcast()` প্রকাশ, আর সিস্টেমের ActivityManagerService ব্রোকার। পাঠানো অ্যাপ জানে না কোন রিসিভার বার্তাটি পাবে। (৪.৮)

১১. Android-এ মেসেজ কিউ কোথায় আছে?

Where is a message queue used inside Android?

প্রতিটি অ্যাপের মেইন থ্রেডে। Handler বার্তা রাখে, MessageQueue বার্তাগুলো সময় অনুযায়ী সাজিয়ে রাখে, আর Looper একে একে সেগুলো চালায়। ছোঁয়ার ঘটনা, লাইফসাইকেল কলব্যাক, `onReceive()`, MediaPlayer-এর onPrepared, সবই এই সারি দিয়ে আসে। (৩.৬)

১২. ANR কী, আর কেন হয়?

What is an ANR, and what causes it?

ANR মানে Application Not Responding। মেইন থ্রেডের সারির গ্রাহক একজনই (Looper), তাই কোনো কলব্যাক অনেকক্ষণ আটকে থাকলে পেছনের সব বার্তা, যেমন ছোঁয়া, অপেক্ষা করে; কয়েক সেকেন্ড পরে সিস্টেম "অ্যাপ সাড়া দিচ্ছে না" বার্তা দেখায়। তাই এই অ্যাপে ছবি নামানো (Coil) আর গান প্রস্তুত করা (`prepareAsync()`) মেইন থ্রেডকে আটকায় না। (**৩.৬**)

১৩. ক্ষণস্থায়ী (non-durable) আর শেষ বার্তা রাখা (retained) পৌঁছানোর পার্থক্য কী?

What is the difference between non-durable and retained delivery?

ক্ষণস্থায়ী হলে শুধু তখন উপস্থিত সাবস্ক্রাইবার বার্তা পায়, দেরিতে আসা কেউ পায় না, রেডিওর মতো; Android-এর সাধারণ ব্রডকাস্ট এমন। শেষ বার্তা রাখা হলে ব্রোকার শেষ বার্তাটি রেখে দেয়, আর নতুন সাবস্ক্রাইবার সঙ্গে সঙ্গে সেটি পায়, নোটিশ বোর্ডের মতো; স্টিকি ব্রডকাস্ট, LiveData আর StateFlow এমন। (**৪.৬**)

১৪.৩ Broadcast Receiver

১৪. BroadcastReceiver কী?

What is a BroadcastReceiver?

Android-এর একটি অংশ, যা নির্দিষ্ট অ্যাকশনের ব্রডকাস্ট শোনে; ব্রডকাস্ট এলে তার `onReceive()` মেইন থ্রেডে চলে। এই অ্যাপে দুটি রিসিভার: নিজের বার্তার জন্য `CustomMessageReceiver`, আর ব্যাটারির জন্য `BatteryStatusLiveData`-র ভেতরের রিসিভার। (**৮.৫**)

১৫. ম্যানিফেস্টে লেখা রিসিভার আর কোডে রেজিস্টার করা রিসিভারের পার্থক্য কী? এই অ্যাপে কোনটি?

What is the difference between a manifest-declared and a context-registered receiver? Which does the app use?

ম্যানিফেস্টে লেখা রিসিভার স্থায়ী; দরকার হলে সিস্টেম অ্যাপ চালু করেও তাকে বার্তা দেয়, তবে Android 8 থেকে বেশিরভাগ implicit ব্রডকাস্ট সে আর পায় না। কোডে রেজিস্টার করা রিসিভার শুধু `registerReceiver()` থেকে `unregisterReceiver()` পর্যন্ত বাঁচে। এই অ্যাপের দুটি রিসিভারই কোডে রেজিস্টার করা; ব্যাটারির ব্রডকাস্ট তো ম্যানিফেস্টের রিসিভার পায়ই না। (**৪.৮**)

১৬. Activity 2 নিজে ব্রডকাস্ট না পাঠিয়ে Activity 3 কেন পাঠায়?

Why does Activity 3, not Activity 2, send the custom broadcast?

সাধারণ ব্রডকাস্ট ক্ষণস্থায়ী: পাঠানোর মুহূর্তে যে রিসিভার রেজিস্টার করা নেই, সে আর কখনো বার্তাটি পায় না। Activity 2 পাঠালে তখনো Activity 3-এর রিসিভার তৈরিই হয়নি, তাই বার্তা হারিয়ে যেত। তাই লেখাটি explicit Intent-এ Activity 3-এ যায়, আর Activity 3 আগে রিসিভার রেজিস্টার করে তারপর ব্রডকাস্ট পাঠায়। (**৮.৭**)

১৭. রিসিভার onStart-এ রেজিস্টার আর onStop-এ আনরেজিস্টার করা হয় কেন?

Why is the receiver registered in onStart and unregistered in onStop?

শোনা শুরু আর শেষ একটি লাইফসাইকেল জোড়ায় থাকলে রিসিভার কখনো মেমরিতে আটকে থাকে না, আর স্ক্রিন দেখা না গেলে কোনো বার্তা সামলানো হয় না। আনরেজিস্টার না করলে সিস্টেম রিসিভারকে, আর রিসিভার Activity-কে ধরে রাখত (মেমরি লিক)। (**৬.১**)

১৮. `RECEIVER_NOT_EXPORTED` আর `setPackage()` কী কাজ করে?

What do RECEIVER_NOT_EXPORTED and setPackage() do?

`RECEIVER_NOT_EXPORTED` ঠিক করে কে এই রিসিভারে পাঠাতে পারে: শুধু এই অ্যাপ (আর সিস্টেম)। targetSdk 34 থেকে সিস্টেমের নয় এমন ব্রডকাস্টের জন্য কোডে রিসিভার রেজিস্টার করার সময় এই ধরনের ফ্ল্যাগ দিতেই হয়। `setPackage()` ঠিক করে কে ব্রডকাস্টটি পাবে: শুধু এই অ্যাপের রিসিভার, তাই ব্যবহারকারীর লেখা অন্য অ্যাপ পায় না। (**৮.১৩**)

১৯. স্টিকি ব্রডকাস্ট কী? `ACTION_BATTERY_CHANGED`-এর বিশেষত্ব কী?

What is a sticky broadcast, and what is special about ACTION_BATTERY_CHANGED?

স্টিকি ব্রডকাস্টের শেষ কপি সিস্টেম রেখে দেয়, আর `registerReceiver()` সেটি সঙ্গে সঙ্গে ফেরত দেয়; তাই ব্যাটারির স্ক্রিন খোলামাত্র শতাংশ দেখা যায়। `ACTION_BATTERY_CHANGED` স্টিকি, সুরক্ষিত (শুধু সিস্টেম পাঠাতে পারে), আর শুধু কোডে রেজিস্টার করা রিসিভার এটি পায়। (**৮.৯**)

২০. ব্যাটারির শতাংশ কীভাবে হিসাব করা হয়?

How is the battery percentage calculated?

শতাংশ = `EXTRA_LEVEL` × ১০০ ÷ `EXTRA_SCALE`, কারণ সব ফোনে সর্বোচ্চ মাত্রা ১০০ নয়। মাত্রা না থাকলে, বা সর্বোচ্চ মাত্রা শূন্য বা ঋণাত্মক হলে, শূন্য দিয়ে ভাগ এড়াতে ফল null আর স্ক্রিনে "—"। ফল সবসময় ০ থেকে ১০০-এর মধ্যে রাখা হয়। (**৮.১১**)

২১. ব্যাটারির রিসিভার শুধু স্ক্রিন দেখা গেলেই শোনে কীভাবে?

How does the battery receiver listen only while the screen is visible?

রিসিভারটি `BatteryStatusLiveData`-র ভেতরে। LiveData প্রথম সক্রিয় অবজারভার পেলে `onActive()`-এ রিসিভার রেজিস্টার করে, আর শেষজন চলে গেলে `onInactive()`-এ আনরেজিস্টার করে। Activity 2 নিজেকে লাইফসাইকেলের মালিক হিসেবে দিয়ে নজর রাখে, তাই onStart-এ শোনা শুরু আর onStop-এ বন্ধ। (**৮.১০**)

২২. ফোন ঘোরালে Activity 3 দ্বিতীয়বার ব্রডকাস্ট পাঠায় না কেন?

Why doesn't Activity 3 send the broadcast again after rotation?

প্রথমবার পাঠানোর আগেই `initialBroadcastSent` ফ্ল্যাগ true করে ViewModel-এর SavedStateHandle-এ রাখা হয়। ঘোরানোর পর নতুন Activity-র onStart ফ্ল্যাগটি দেখে আর পাঠায় না। পাওয়া বার্তাও একই জায়গায় থাকে, তাই স্ক্রিনে আগের বার্তাই দেখা যায়। (**৮.৮**)

১৪.৪ Image Scale

২৩. ইন্টারনেট থেকে ছবি কীভাবে নামানো হয়, আর `Size.ORIGINAL` কেন?

How is the image downloaded, and why Size.ORIGINAL?

ViewModel-এর একটি কোরুটিনে Coil 3-এর `imageLoader.execute()` ছবিটি নামায়, তাই মেইন থ্রেড আটকায় না; ফল Loading, Success বা Error হিসেবে একটি StateFlow-এ যায়। Coil সাধারণত ছবিকে ভিউয়ের মাপে ছোট করে; সেই ছোট ছবিতে জুম করলে ঝাপসা দেখাত, তাই মূল মাপ চাওয়া হয়। (**৯.১**)

২৪. পিঞ্চ জুম কীভাবে কাজ করে?

How does pinch-to-zoom work?

ScaleGestureDetector দুই আঙুলের ছোঁয়ার ঘটনা থেকে scaleFactor আর ফোকাল পয়েন্ট দেয়। `zoomBy()` জুমকে ১ থেকে ৫ গুণের মধ্যে আটকে রেখে ফোকাল পয়েন্ট ঘিরে ছবির Matrix-এ `postScale()` করে, তারপর কিনারা ঠিক করে। (**৯.৪**)

২৫. জুমের সময় আঙুলের নিচের অংশ আঙুলের নিচেই থাকে কেন?

Why does the point under the fingers stay under the fingers while zooming?

স্কেল করা হয় ফোকাল পয়েন্ট f ঘিরে। s গুণ বড় করলে ছবির বিন্দু p যায় f + s × (p − f)-এ; p = f হলে বিন্দুটি নড়ে না। তাই দুই আঙুলের মাঝের অংশ জায়গায় থেকে চারপাশ বড় হয়। (**৯.৪**)

২৬. ছবির Matrix কী?

What is the image Matrix?

৩×৩ সংখ্যার একটি রূপান্তর, যা ছবির প্রতিটি বিন্দু ভিউয়ের কোথায় আর কত বড় করে আঁকা হবে তা ঠিক করে; এতে বড়-ছোট করা (scale) আর সরানো (translate) থাকে। scaleType `MATRIX` হওয়ায় ImageView পুরোপুরি এই Matrix মেনে আঁকে। জুম = বর্তমান স্কেল ÷ পুরো ছবি ফিট করার স্কেল। (**৯.৩**)

২৭. জুম করা ছবি সরালে কিনারায় খালি জায়গা দেখা যায় না কেন?

Why is there never an empty gap at the edges when panning?

প্রতিটি জুম বা সরানোর পর `clampTranslation()` ছবির কোনাগুলো ভিউয়ের স্থানাঙ্কে এনে প্রতিটি অক্ষে `edgeCorrection()` হিসাব করে: ছবি ভিউয়ের চেয়ে ছোট হলে মাঝখানে আনে, বড় হলে যেদিকে ফাঁক সেদিকে টেনে ফাঁক বন্ধ করে। (**৯.৫**)

২৮. ফোন ঘোরালে জুম কীভাবে থেকে যায়?

How is the zoom kept after rotation?

`ZoomableImageView` নিজের সংরক্ষিত অবস্থায় জুম আর ভিউয়ের মাঝখানের বিন্দুটি ছবির প্রস্থ-উচ্চতার ভগ্নাংশ (০ থেকে ১) হিসেবে রাখে। নতুন ভিউতে ছবি আর মাপ দুটোই জানা হলে আগে ফিট, তারপর সেই জুম, তারপর সেই বিন্দুকে মাঝখানে আনা হয়। ভগ্নাংশ বলে মাপ বদলালেও একই অংশ দেখা যায়। (**৯.৮**)

১৪.৫ Video ও Audio

২৯. VideoView আর MediaController কী কাজ করে?

What do VideoView and MediaController do?

VideoView ভেতরের MediaPlayer দিয়ে ভিডিও ডিকোড করে নিজের পৃষ্ঠে আঁকে আর শব্দ বাজায়। MediaController হলো তৈরি নিয়ন্ত্রণ প্যানেল (play/pause, সামনে-পেছনে যাওয়ার দাগ, সময়), যা MediaPlayerControl ইন্টারফেস দিয়ে VideoView-কে চালায়। (**১০.১**)

৩০. ভিডিওর অবস্থান onStop-এ না রেখে onPause-এ রাখা হয় কেন?

Why is the video position saved in onPause rather than onStop?

সাম্প্রতিক Android-এ onStop চলার আগেই VideoView-এর পৃষ্ঠ আর তার MediaPlayer ছেড়ে দেওয়া হয়; তখন অবস্থান পড়লে ০ পাওয়া যেত। onPause-এ প্লেয়ার তখনো চালু, তাই সঠিক অবস্থান পাওয়া যায়। (**১০.৩**)

৩১. ফোন ঘোরালে ভিডিও চলতে থাকে, কিন্তু স্ক্রিন ছাড়লে থেমে থাকে কেন?

Why does the video keep playing after rotation but stay paused after leaving the screen?

onPause-এ "আবার চালাতে হবে" রাখা হয় শুধু তখন, যখন ভিডিও চলছিল আর `isChangingConfigurations` সত্য, অর্থাৎ কারণটি ফোন ঘোরানো। সত্যিই স্ক্রিন ছেড়ে গেলে এটি মিথ্যা, তাই ফিরে এলে ভিডিও একই জায়গায় থেমে থাকে। (১০.৩)

৩২. MediaPlayer-কে স্টেট মেশিন বলা হয় কেন?

Why is MediaPlayer called a state machine?

এটি সবসময় Idle, Initialized, Preparing, Prepared, Started, Paused, Stopped, PlaybackCompleted বা End-এর মতো কোনো একটি অবস্থায় থাকে, আর প্রতিটি মেথড শুধু ঠিক অবস্থায় ডাকা যায়; ভুল অবস্থায় ডাকলে IllegalStateException হয়। যেমন Stopped থেকে সরাসরি start করা যায় না, আগে আবার prepare করতে হয়। (১১.১)

৩৩. `prepare()` না দিয়ে `prepareAsync()` কেন?

Why prepareAsync() instead of prepare()?

`prepare()` ফাইল প্রস্তুত না হওয়া পর্যন্ত মেইন থ্রেড আটকে রাখত। `prepareAsync()` সঙ্গে সঙ্গে ফিরে আসে, আর প্রস্তুত হলে প্লেয়ার মেইন থ্রেডের সারিতে onPrepared পাঠায়; এটি অনুরোধ-উত্তর ধরন। (১১.২)

৩৪. MediaPlayer ViewModel-এ রাখা হয়েছে কেন?

Why is the MediaPlayer kept in a ViewModel?

ফোন ঘোরালে Fragment আর তার ভিউ নতুন করে তৈরি হয়, কিন্তু ViewModel একই থাকে; তাই প্লেয়ার আর গান থামে না। স্ক্রিন স্থায়ীভাবে চলে গেলে `onCleared()`-এ `release()` করে প্লেয়ার মুক্ত করা হয়। (১১.২)

৩৫. গান চলার সময় স্লাইডার কীভাবে এগোয়?

How does the slider move while the music plays?

গান চালু হলে ViewModel-এ একটি কোরুটিন লুপ চলে: প্লেয়ারের বর্তমান অবস্থান স্টেটে বসায়, তারপর `delay(200)` । StateFlow নতুন স্টেট দিলে Fragment-এর `render()` স্লাইডার আর সময় হালনাগাদ করে। `delay()` মেইন থ্রেড আটকায় না, শুধু একটি দেরিতে চলার বার্তা রাখে। (১১.৪)

৩৬. অডিও আর ভিডিও একসঙ্গে বাজে না কেন?

Why do audio and video never play at the same time?

অন্য ড্রয়ার আইটেমে গেলে Navigation অডিওর ViewModel আর তার প্লেয়ার বাঁচিয়ে রাখে, তাই `AudioFragment`-এর onStop-এ, ফোন ঘোরানো না হলে, গান থামানো হয়। ভিডিও নিজেও onPause-এ থামে। তাই একটি স্ক্রিন ছাড়লেই সেটির শব্দ বন্ধ হয়। (১১.৭)

১৪.৬ লাইফসাইকেল ও স্টেট

৩৭. ফোন ঘোরালে কী হয়, আর এই অ্যাপ কীভাবে তথ্য বাঁচায়?

What happens on rotation, and how does the app keep its data?

সিস্টেম Activity আর Fragment ভেঙে নতুন করে তৈরি করে, তাই সাধারণ ভেরিয়েবল হারায়। এই অ্যাপ তিন উপায়ে তথ্য রাখে: ViewModel (ঘোরানোতে টেকে, যেমন MediaPlayer), SavedStateHandle (প্রসেস বন্ধ হলেও টেকে, যেমন স্পিনারের বাছাই আর ভিডিওর অবস্থান), আর ভিউয়ের নিজস্ব সংরক্ষিত অবস্থা (যেমন জুম আর টেক্সট ফিল্ডের লেখা)। (৬.২)

৩৮. ViewModel আর SavedStateHandle-এর পার্থক্য কী?

What is the difference between a ViewModel and a SavedStateHandle?

ViewModel মেমরিতে থাকে; ফোন ঘোরালে টেকে, কিন্তু অ্যাপ পেছনে থাকার সময় সিস্টেম প্রসেস বন্ধ করে দিলে হারায়। SavedStateHandle ViewModel-এর ভেতরের একটি চাবি-মান ভান্ডার, যার মান Bundle-এ সংরক্ষিত হয়, তাই প্রসেস বন্ধ হয়ে আবার চালু হলেও ফিরে আসে। তাই বড় জিনিস (প্লেয়ার) ViewModel-এ, আর ছোট দরকারি মান SavedStateHandle-এ। (**৬.২**)

৩৯. LiveData আর StateFlow কী, এই অ্যাপে কোথায় ব্যবহার হয়েছে?

What are LiveData and StateFlow, and where are they used here?

দুটিই নজর রাখা যায় এমন মানের ধারক, যা সবসময় সর্বশেষ মান ধরে রাখে আর মান বদলালে অবজারভারদের জানায়। LiveData নিজেই লাইফসাইকেল বোঝে; StateFlow Kotlin কোরুটিনের অংশ, তাই এখানে `repeatOnLifecycle` দিয়ে সংগ্রহ করা হয়। এই অ্যাপে LiveData: স্পিনারের বাছাই, পাওয়া বার্তা আর ব্যাটারির অবস্থা; StateFlow: ছবির স্টেট আর অডিওর স্টেট। (**৪.৭**)

৪০. `isChangingConfigurations` কী কাজে লাগে?

What is isChangingConfigurations used for?

onPause বা onStop-এর সময় এটি বলে দেয় স্ক্রিনটি শুধু ঘুরছে (true), নাকি সত্যিই চলে যাচ্ছে (false)। ভিডিও এটি দেখে ঠিক করে ঘোরানোর পর আবার চলবে কি না, আর অডিও ঠিক করে onStop-এ গান থামাবে কি না। (**৬.২**)

১৫ পরিভাষা

সহজ কথায়

এই বইয়ে ব্যবহৃত পারিভাষিক শব্দের তালিকা: বাঁয়ে ইংরেজি শব্দ (ভাইভায় যেটি শোনা যেতে পারে), মাঝে এই বইয়ে যে বাংলা শব্দ ব্যবহার করা হয়েছে, আর ডানে তার অর্থ।

১৫.১ বার্তা ব্যবস্থার শব্দ

ইংরেজি	এই বইয়ে	অর্থ
Message	বার্তা	একটি কাজ বা খবরের প্যাকেট।
Message Queue	মেসেজ কিউ, বার্তার সারি	বার্তার সারি; প্রতিটি বার্তা একজন গ্রাহক ক্রম অনুযায়ী সামলায়।
Producer / Consumer	প্রেরক / গ্রাহক	যে বার্তা সারিতে রাখে / যে সারি থেকে তুলে কাজ করে।
FIFO	FIFO	First In, First Out: যে আগে এসেছে, সে আগে বের হয়।
Point-to-point	এক-থেকে-এক	একটি বার্তা, একজন নির্দিষ্ট প্রাপক।
Competing consumers	প্রতিযোগী গ্রাহক	একাধিক গ্রাহক একই সারি থেকে কাজ ভাগ করে নেয়; একই বার্তা দুজন পায় না।
Acknowledgement (ack)	প্রাপ্তিস্বীকার	গ্রাহকের "কাজ শেষ" জানানো; তবেই সারি বার্তাটি মোছে।
Load leveling	চাপ সমান করা	ভিড়ের সময়ের বার্তা সারিতে জমে থাকে, গ্রাহক নিজের গতিতে শেষ করে।
Asynchronous	অ্যাসিঙ্ক্রোনাস	পাঠিয়েই ফিরে আসা; ফলের জন্য অপেক্ষা না করা।
Publish/Subscribe	পাবলিশ/সাবস্ক্রাইব	প্রকাশক টপিকে বার্তা দেয়; ব্রোকার সেটি সব সাবস্ক্রাইবারের কাছে পৌঁছায়।
Publisher / Subscriber	প্রকাশক / সাবস্ক্রাইবার	যে বার্তা প্রকাশ করে / যে টপিকে আগ্রহ জানিয়ে বার্তা পায়।
Topic	টপিক	বার্তার শ্রেণির নাম; Android-এ Intent-এর অ্যাকশন।
Broker	ব্রোকার	মাঝখানের ব্যবস্থা, যে সাবস্ক্রিপশনের তালিকা রাখে আর বার্তা পৌঁছায়; Android-এ ActivityManagerService।
Fan-out	ফ্যান-আউট	একটি বার্তার কপি একসঙ্গে অনেক সাবস্ক্রাইবারের কাছে যাওয়া।
Non-durable	ক্ষণস্থায়ী	শুধু তখন উপস্থিত সাবস্ক্রাইবার পায়; দেরিতে আসা কেউ পায় না।
Retained message	শেষ বার্তা রাখা	ব্রোকার শেষ বার্তাটি রেখে দেয়; নতুন সাবস্ক্রাইবার সঙ্গে সঙ্গে পায়।

ইংরেজি	এই বইয়ে	অর্থ
Durable subscription	স্থায়ী সাবস্ক্রিপশন	সাবস্ক্রাইবার না থাকার সময়ের সব বার্তা জমা থাকে; ফিরে এলে সব পায়।
Decoupling	আলাদা রাখা	প্রেরক আর প্রাপককে একে অপরের ওপর নির্ভর না করানো।
Observer pattern	অবজারভার প্যাটার্ন	একই প্রোগ্রামের ভেতরে ব্রোকার-ছাড়া পাবলিশ/সাবস্ক্রাইব; যেমন LiveData ও StateFlow।

১৫.২ Android-এর শব্দ

ইংরেজি	এই বইয়ে	অর্থ
Activity	Activity	একটি পূর্ণ স্ক্রিন আর তার লাইফসাইকেলের মালিক Android অংশ।
Fragment	Fragment	Activity-র ভেতরে বসানো স্ক্রিন-অংশ, যার নিজস্ব লাইফসাইকেল ও ভিউ আছে।
View	ভিউ	স্ক্রিনে দেখা যায় এমন একটি অংশ, যেমন বাটন, ছবি বা লেখা।
Intent	Intent	Android-এর অংশগুলোর মধ্যে পাঠানো বার্তা: অ্যাকশন, এক্সট্রা আর দরকার হলে গন্তব্য।
Explicit Intent	explicit Intent	গন্তব্যের ক্লাস নির্দিষ্ট করে দেওয়া Intent; একটি এক-থেকে-এক বার্তা।
Broadcast	ব্রডকাস্ট	একটি অ্যাকশনসহ Intent, যা সিস্টেম মিলে যাওয়া সব রিসিভারের কাছে পৌঁছায়।
BroadcastReceiver	রিসিভার	ব্রডকাস্ট গ্রহণকারী অংশ; এর `onReceive()` মেইন থ্রেডে চলে।
Register / Unregister	রেজিস্টার / আনরেজিস্টার	রিসিভারকে শোনার তালিকায় তোলা / তালিকা থেকে সরানো।
IntentFilter	IntentFilter	রিসিভার কোন অ্যাকশন গ্রহণ করবে তার শর্ত।
Sticky broadcast	স্টিকি ব্রডকাস্ট	যে ব্রডকাস্টের শেষ কপি সিস্টেম রেখে দেয়।
Protected broadcast	সুরক্ষিত ব্রডকাস্ট	যে ব্রডকাস্ট শুধু সিস্টেম পাঠাতে পারে।
Main thread	মেইন থ্রেড	যে একমাত্র কাজের ধারায় অ্যাপের স্ক্রিনের সব কাজ চলে।
Looper / MessageQueue / Handler	Looper / MessageQueue / Handler	মেইন থ্রেডের গ্রাহক লুপ / বার্তার সারি / সারিতে বার্তা রাখার উপায়।
Callback	কলব্যাক	কোনো ঘটনা ঘটলে সিস্টেম বা অন্য কোড যে ফাংশন ডেকে খবর দেয়।
Listener	লিসেনার	কোনো ঘটনার খবর পাওয়ার জন্য যুক্ত করা অংশ, যেমন ক্লিকের লিসেনার।

ইংরেজি	এই বইয়ে	অর্থ
ANR	ANR	Application Not Responding; মেইন থ্রেড অনেকক্ষণ আটকে থাকলে সিস্টেমের সতর্কবার্তা।
Lifecycle	লাইফসাইকেল	Activity বা Fragment-এর তৈরি থেকে ভাঙা পর্যন্ত ধাপগুলো।
Configuration change	কনফিগারেশন পরিবর্তন	ফোন ঘোরানো, ভাষা বা থিম বদল; সাধারণত Activity নতুন করে তৈরি হয়।
Process death	প্রসেস বন্ধ হওয়া	অ্যাপ পেছনে থাকার সময় সিস্টেম মেমরির জন্য অ্যাপের প্রসেস বন্ধ করে দেওয়া।
Memory leak	মেমরি লিক	দরকার শেষ হওয়া জিনিস মেমরিতে আটকে থাকা।
ViewModel	ViewModel	ফোন ঘোরানোর পরও বেঁচে থাকা, স্ক্রিনের তথ্যের ধারক।
SavedStateHandle	SavedStateHandle	ViewModel-এর চাবি-মান ভান্ডার; প্রসেস বন্ধ হলেও মান ফিরে আসে।
LiveData	LiveData	লাইফসাইকেল বোঝে এমন, নজর রাখা যায় এমন মান; শুধু সক্রিয় অবজারভারকে জানায়।
StateFlow	StateFlow	কোরুটিন-ভিত্তিক, নজর রাখা যায় এমন মান; সবসময় একটি সর্বশেষ মান রাখে।
Coroutine	কোরুটিন	থ্রেড না আটকে থামতে আর আবার চলতে পারে এমন হালকা কাজ।
State	স্টেট, অবস্থা	কোনো মুহূর্তে স্ক্রিন বা প্লেয়ারের অবস্থা।
State machine	স্টেট মেশিন	নির্দিষ্ট কয়েকটি অবস্থা আর এক অবস্থা থেকে আরেকটিতে যাওয়ার নিয়ম।
Unidirectional data flow	একমুখী ডেটা প্রবাহ	স্টেট ViewModel থেকে স্ক্রিনে যায়, নির্দেশ স্ক্রিন থেকে ViewModel-এ।
Matrix	Matrix	বিন্দুকে বড়-ছোট আর স্থানান্তর করে এক স্থানাঙ্ক থেকে আরেকটিতে নেওয়ার ৩×৩ রূপান্তর।
Focal point	ফোকাল পয়েন্ট	পিঞ্চের দুই আঙুলের মাঝের বিন্দু; জুম এই বিন্দুকে ঘিরে হয়।
MediaPlayer	MediaPlayer	অডিও আর ভিডিও চালানোর Android ক্লাস; একটি স্টেট মেশিন।
Window insets	ইনসেট	সিস্টেম বার, ক্যামেরার খাঁজ বা কীবোর্ড স্ক্রিনের কতটা ঢেকেছে তার মাপ।
Edge-to-edge	এজ-টু-এজ	অ্যাপের জানালা স্ট্যাটাস বার আর নেভিগেশন বারের নিচ পর্যন্ত বিস্তৃত থাকা।